

Agent Harness: A Practitioner's Textbook

A fact-based synthesis of agent-harness engineering. Substantive factual claims are cited where they appear, not only in the bibliography.

24 sections · generated from Markdown sources

Preface

This book examines the systems that surround a language model when the model is asked to do real work. The chapters that follow bring the agent-harness literature into one engineering narrative. Substantive factual claims cite their sources where they appear so that readers can verify both the evidence and its scope.

The field starts from a simple premise. As Vivek Trivedy of LangChain puts it: "Agent = Model + Harness. **If you're not the model, you're the harness.**" ([LangChain — The Anatomy of an Agent Harness](#)). That sentence is a useful responsibility boundary, but it is deliberately coarse. This book will distinguish the model-facing *agent harness* from the durable runtime, the user-facing product, the fleet platform or control plane, and the separate evaluation harness. Anthropic uses the same narrower distinction when it defines an agent harness as the system that processes inputs and orchestrates tool calls, while an evaluation harness runs trials, records them, grades them, and aggregates results ([Anthropic — Demystifying Evals for AI Agents](#)).

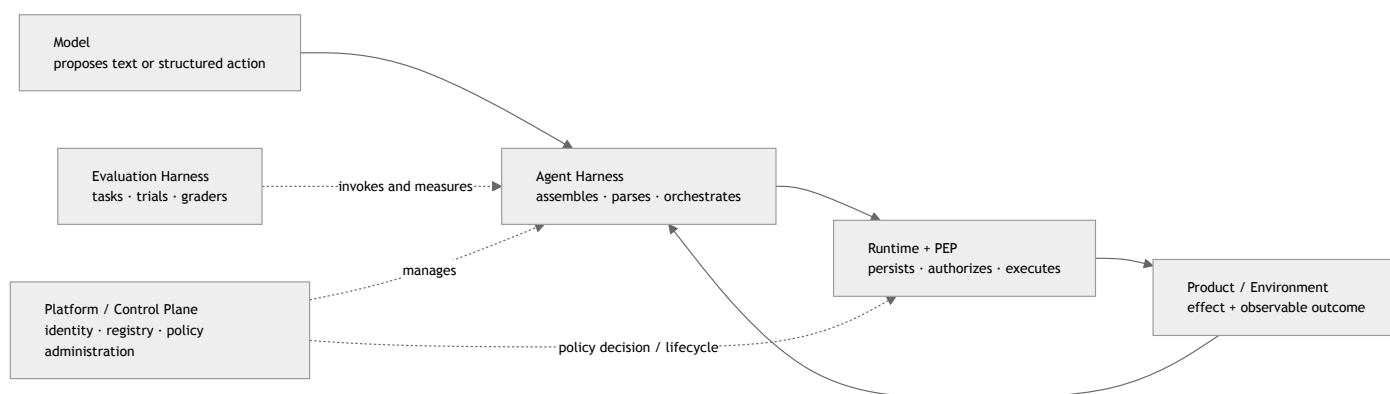
Throughout the book, think of the model as a component that consumes a current input representation and produces text or structured output. A tool call is a proposal, not an executed side effect: application code validates and runs the named tool, then returns a tool result to the model ([Anthropic — How Tool Use Works](#)). This distinction between model output and real-world effect explains why later chapters devote so much attention to context, tools, state, permissions, tests, sandboxes, and evaluation.

The companion stage-1 volume covers the model behavior this book relies on: tokens, attention, the per-request KV cache, provider prompt caching, the context window, sampling, retrieval, and the model side of tool calling. It ends with a six-part boundary: the model predicts; the external system owns context selection, durable state, tool execution, permissions, verification, and consequences ([LLM Foundations, Chapter 14](#)). This volume implements those external responsibilities. It does not re-derive the model mechanics.

Before You Start

This is the second book in a two-part sequence. It assumes you have read [LLM Foundations for Harness Engineering](#) and are comfortable with tokens, attention, context windows, sampling, post-training, retrieval, model-generated tool calls, prompt injection, and pass@k/pass^k. When this book uses a Foundations concept, it points back to the relevant chapter instead of deriving it again. Readers who have not completed the first volume can still follow the narrative, but should treat the short recaps as pointers rather than complete explanations. Provider prices, cache lifetimes, model capabilities, and benchmark scores change; whenever they appear here, read them as dated examples rather than permanent definitions.

The Core Equation



Key Takeaways

- A language model call does not own durable application state, execute real tools, or enforce permissions; external software provides those capabilities.
- Harness engineering is distinct from prompt engineering: it improves the entire system, not just individual prompts.
- "Harness" is a broad responsibility shorthand; later chapters separate the agent harness, runtime, product, platform, and evaluation harness when architecture depends on the distinction.
- Time-sensitive provider and product claims are labeled as dated examples rather than definitions.

- Substantive factual claims are cited in the chapter text so readers can inspect their original scope.

Further Reading

- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- *Awesome Harness Engineering* reading list: <https://github.com/walkinglabs/awesome-harness-engineering>

Chapter 1: What Is an Agent Harness?

1.1 From a Useful Shorthand to a Precise Boundary

The companion *LLM Foundations* volume uses **harness** as a deliberately coarse shorthand for the external system around a model. At that resolution, the central equation is useful:

Agent behavior = model behavior + surrounding-system behavior

A model receives the representation assembled for the current inference and generates tokens or structured output. It does not, by itself, preserve durable application state, grant permissions, run a shell command, write a database row, or verify that a requested real-world change occurred. Those responsibilities sit outside the model in the **model-system responsibility boundary**.

This book needs a finer vocabulary because “everything outside the model” is too broad for designing a production system. Anthropic, for example, defines an **agent harness** (or scaffold) as the system that enables a model to act as an agent by processing inputs, orchestrating tool calls, and returning results; it separately defines the **evaluation harness** that creates and runs tests around that system ([Anthropic — Demystifying Evals for AI Agents](#)). We adopt that distinction and add explicit runtime, product, and platform layers below.

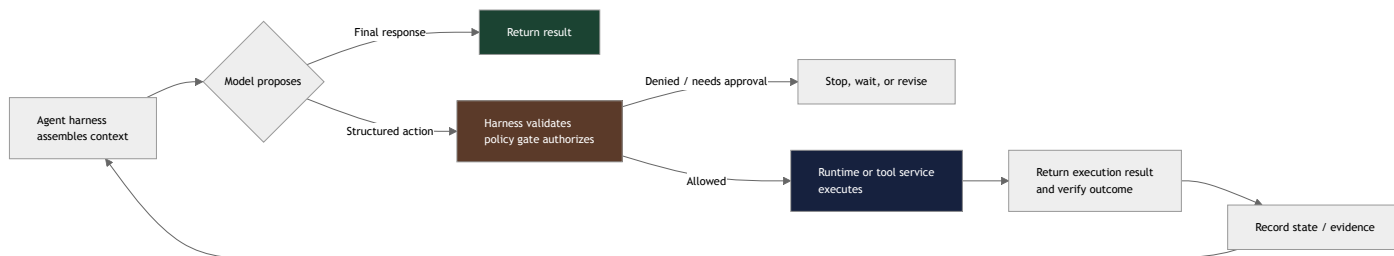
Accordingly, this book uses **agent harness** for the model-adjacent loop controller, and **surrounding system** when a claim applies more broadly. The coarse “model + harness” equation remains a teaching aid, not a claim that one process, library, or team owns every external responsibility.

1.2 The Minimum Agent Loop: Proposal Is Not Effect

LLM Foundations introduces model-side tool use and the minimal loop ([Foundations ch 12](#)). The engineering boundary is worth making explicit:

1. The agent harness **assembles context** from instructions, selected history or memory, tool schemas, retrieved data, and current state.
2. The model **proposes** a final response or a structured action such as a tool call.
3. The harness **parses and validates** that proposal, then routes it through any required authorization or approval gate.
4. A runtime or tool service **executes** an allowed action.
5. The surrounding system receives an execution result and, where consequences matter, checks the resulting environment or business outcome.
6. State is recorded and the next model input is assembled, or the run stops.

This is a contract, not a metaphor. In Anthropic's documented client-tool loop, the model emits a structured `tool_use` request, application code executes it, and a `tool_result` returns to the conversation. Provider-executed tools move the executor to provider infrastructure, but they still do not make the model itself the executor ([Anthropic — How Tool Use Works](#)).



The decisive boundary is between **proposal** and **effect**. A schema-valid request is not automatically authorized, and a successful API response is not always proof of the intended outcome. The model may explain, request, or recommend an action; it cannot grant itself authority. Later chapters turn the validation, authorization, execution, and outcome-confirmation stages into concrete designs.

1.3 Context Does Not Have to Grow Monotonically

In the simplest append-only implementation, each loop iteration adds the model's request and the tool result to a transcript. That transcript grows monotonically. A production harness, however, does not have to resend the entire raw transcript on every call. It may trim messages, offload artifacts, compact older material, reset a context, retrieve

selected memory, or expose tools only when needed. Anthropic describes context engineering as curating the token set available during inference rather than merely accumulating conversation text ([Anthropic – Effective Context Engineering for AI Agents](#)).

The accurate invariant is therefore not “context always grows,” but **the run keeps producing information that must be managed**. The next context is a selected, serialized view of that information. Selection and compression can lose evidence or change behavior, so the system must preserve durable state, artifacts, provenance, and unresolved uncertainty outside the model input. Context is finite input to one call, not the system's database or event history; the underlying mechanism is covered in [Foundations ch 9](#).

1.4 Six System Layers Used in This Book

The following levels are the book's architectural convention. Real products may combine several levels in one service or split one level across vendors, but the responsibilities remain distinguishable.

Layer	Definition in this book	Typical responsibilities
Model	The probabilistic component that consumes the current representation and generates tokens or structured output	Language and multimodal inference, action proposals, classifications, plans
Agent harness	The model-adjacent system that assembles inputs, parses outputs, and drives one or more agent loops	Prompt/context assembly, tool exposure, loop control, handoff coordination
Runtime	The execution substrate that makes work durable and runs allowed operations	Scheduling, queues, checkpoints, retry semantics, sandboxes, worker lifecycle
Product / application	The user-facing workflow and domain system in which the agent is embedded	UX, domain logic, business state, review surfaces, user communication
Platform / control plane	The management layer for many tenants, versions, agents, and runtimes	Registry, identity, policy administration and decision, rollout, fleet lifecycle
Evaluation harness	The test system that creates trials, invokes the system under test, captures evidence, and applies graders	Task environments, isolation, repeated trials, transcripts, outcomes, reports

These are not concentric boxes in every deployment. For example, an authorization decision may come from a platform service while an enforcement point in the runtime blocks the action. Likewise, a provider-executed tool may supply part of the runtime even though the product owns the workflow.

A **framework** is different from these layers: it is a set of abstractions or libraries used to implement one or more of them. Calling something a framework says how developers build with it, not which production responsibility it owns.

1.5 Who Owns the Six External Responsibilities?

The Foundations volume closes with six responsibilities that must remain outside model prediction: context, state, tools, permissions, verification, and consequences. At the finer resolution used here, ownership looks like this:

Responsibility	Primary owner in the surrounding system	Model's legitimate role
Context	Agent harness selects and serializes the model-visible view	Consume the supplied context; suggest information that may be needed
State	Runtime persists execution state; product owns domain and business state	Propose state transitions; never serve as the sole durable record
Tools	Agent harness exposes contracts; runtime or tool service executes them	Select a tool and propose arguments
Permissions	Product/platform policy decides; a non-bypassable runtime enforcement point applies the decision	Request access or explain intent; never self-grant authority
Verification	Runtime, product, and evaluation harness apply tests, environment checks, graders, or human review	Offer a critique or hypothesis as one fallible signal
Consequences	Runtime and integrated external systems produce and record effects; product owns their meaning	Predict or describe intended effects

“Harness owns these responsibilities” is correct at the Foundations book's coarse boundary. In this book, the table prevents that shorthand from hiding which subsystem must actually implement a guarantee.

1.6 Agent Harness and Evaluation Harness Are Different Systems

An **agent harness** is part of the system being evaluated. An **evaluation harness** surrounds that system during a test: it provisions a task and environment, invokes the model-plus-agent-harness combination, records a trial, inspects its transcript or final environment state, applies graders, and aggregates results. Anthropic's evaluation terminology makes the same separation and notes that evaluating an “agent” evaluates the model and its agent harness together ([Anthropic — Demystifying Evals for AI Agents](#)).

The distinction matters operationally. A production trace may supply evidence to an evaluation, but production observability is not automatically an evaluation harness. Conversely, a test runner may imitate tools and environments without being the runtime that serves users. Keeping these systems separate makes it possible to change the agent harness while holding the test infrastructure stable enough to measure the change.

1.7 Two Useful Maps, Neither a Universal Standard

Practitioners use several overlapping maps for the surrounding system. They answer different questions and should not be mistaken for canonical layers.

Birgitta Böckeler proposes three concentric scopes for coding agents: the model; a **builder harness** shipped by a model or agent vendor; and a **user harness** containing repository instructions, hooks, skills, and review agents added by the adopting team ([Thoughtworks / Martin Fowler — Harness Engineering](#)). This is an ownership and customization map. A builder harness or user harness may each contain pieces of the agent-harness, runtime, product, and evaluation layers defined above.

The 2026 manuscript *Agent Harness Engineering: A Survey* proposes **ETCLOVG** as a map of engineering concerns: Execution environment, Tool interface, Context, Lifecycle, Observability, Verification, and Governance ([OpenReview — Agent Harness Engineering: A Survey](#)). We use it as a coverage checklist, not as an industry standard or a deployment architecture:

- **Execution environment:** where allowed actions become effects.
- **Tool interface:** schemas, protocols, registries, and selection policies.
- **Context:** assembly, retrieval, memory injection, and compaction.
- **Lifecycle:** startup, checkpoints, recovery, handoffs, and termination.
- **Observability:** traces, telemetry, latency, cost, and failure evidence.
- **Verification:** outcome checks, test suites, graders, and release gates.
- **Governance:** identity, permissions, approvals, policy, and audit evidence.

The six system levels answer **where a responsibility lives**. ETCLOVG asks **which concerns must be covered**. The builder/user rings ask **who supplies or customizes a component**. A single component can occupy one level, cover several concerns, and be customized by more than one party.

1.8 Framework, Runtime, Harness, and Product Labels

Vendor and practitioner literature does not use these labels uniformly. One LangChain taxonomy describes frameworks as developer abstractions, runtimes as durable execution infrastructure, and harnesses as more opinionated, “batteries-included” agent systems; it also acknowledges overlapping cases ([LangChain — Agent Frameworks, Runtimes, and Harnesses, Oh My!](#)). That taxonomy is useful for comparing products, but it is not a protocol specification.

This book therefore classifies a component by the responsibility it performs, not by the label on its website. A library called an “agent runtime” may contain agent-harness logic. A product sold as an “agent platform” may bundle runtime, control-plane, evaluation, and application features. The six-layer vocabulary lets us discuss those functions without forcing every vendor into one box.

1.9 Prompt, Context, Harness, and Loop Engineering

Prompt engineering, context engineering, harness engineering, and loop engineering are best treated as complementary lenses:

- **Prompt engineering** shapes model-visible instructions and examples.
- **Context engineering** selects and maintains the information available for a particular inference; Anthropic explicitly frames it as curating the token set used during inference ([Anthropic — Effective Context Engineering for AI Agents](#)).
- **Harness engineering** designs the model-adjacent control and interfaces that turn proposals into governed work.

- **Loop engineering** is a practitioner term for designing repeated invocation, feedback, verification, budgets, and stop rules ([Addy Osmani — Loop Engineering](#)).

These terms do not describe a mandatory historical ladder, and none replaces the others. A reliable system usually needs decisions at all four scopes.

1.10 Why the Boundary Persists as Models Improve

More capable models can reduce failures and absorb behavior that previously required elaborate prompting or orchestration. They do not erase the distinction between a probabilistic proposal and an authorized, recorded external effect. As long as an agent can spend money, modify data, communicate externally, or operate infrastructure, some surrounding system must still own credentials, policy enforcement, execution, durable state, verification, and recovery.

What changes is the placement and complexity of those mechanisms. A provider may execute a tool server-side; a product may move planning from explicit workflow code into the model; a runtime may make recovery transparent. The implementation boundary can move, but the responsibility boundary remains visible. That is the organizing principle for the rest of this book.

Key Takeaways

- **“Model + harness” is a coarse equation:** it separates model prediction from the surrounding system, but production design needs finer layers.
- **An agent harness is model-adjacent:** it assembles inputs, interprets proposals, and drives loops; it is not automatically the whole product or platform.
- **Proposal is not effect:** validation, authorization, execution, and outcome confirmation occur outside the model.
- **Context is assembled, not inevitably appended forever:** monotonic growth describes a naive transcript, not every harness.
- **Runtime, product, platform, and evaluation harness have distinct jobs** even when one vendor bundles them.
- **Context, state, tools, permissions, verification, and consequences all have external owners;** the model contributes proposals and fallible judgments.
- **ETCLOVG and builder/user rings are useful maps, not universal standards.**
- **Better models may move implementation boundaries, but they do not assume authority over real-world consequences.**

Further Reading

- Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- Anthropic, *Demystifying Evals for AI Agents*, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Anthropic, *Effective Context Engineering for AI Agents*, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses*, *Oh My!*, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
- *Agent Harness Engineering: A Survey*, OpenReview manuscript, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>

Chapter 2: System Prompts, Instructions, and Policy Boundaries

The *LLM Foundations* volume established two facts that this chapter turns into engineering practice. First, role markers, delimiters, and instructions are serialized into model input; post-training can make the model respond differently to them, but does not turn them into infallible control channels. Second, a model can propose an action but cannot grant itself permission or produce the external effect ([Foundations, Chapter 8](#), [Foundations, Chapter 14](#)).

The instruction layer therefore has an important but bounded job: shape model behavior. It is not the place where hard authorization is enforced. This chapter explains how to compose, version, cache, and evaluate model-visible instructions while keeping executable policy in the harness and runtime.

2.1 Instructions Are Model Input, Not Executable Policy

An agent's system prompt is not merely the question typed into a chat box. It is a maintained, model-facing artifact that may define the agent's role, objectives, tool-use guidance, response contract, and rules for handling uncertainty. HumanLayer's phrase *own your prompts* captures the operational lesson: do not let important behavior disappear inside unversioned framework defaults ([HumanLayer — 12-Factor Agents](#)).

Calling the prompt “application code” is useful as an ownership metaphor, but it must not erase a crucial difference. Code in the runtime executes deterministic checks. Prompt text conditions a probabilistic model. A sentence such as “never send data outside this tenant” can reduce unsafe proposals, but it cannot inspect credentials, block network egress, or make a dispatch path non-bypassable. Those controls belong outside the model.

The prompt is not persistent state inside the model either. The harness selects and assembles instructions for a call, the provider serializes the applicable roles and content, and the model conditions its next output on that representation. Later calls see an instruction only if the surrounding system supplies it again.

2.2 Instruction Priority Is a Behavioral Defense

Agents receive content from several sources: platform or system instructions, developer and project instructions, user requests, examples, tool results, and retrieved documents. Providers can train models to follow an intended priority among these sources. *The Instruction Hierarchy* formalizes this as a training objective: higher-priority instructions should dominate conflicting lower-priority text ([Wallace et al. — The Instruction Hierarchy](#)).

That is a behavioral defense, not a hard boundary. Role names and exact priority orders are provider-specific, and even models trained for hierarchy following can fail under conflict. IHEval found substantial degradation when instructions at different priority levels conflicted, which is evidence against treating placement as an enforcement guarantee ([IHEval: Evaluating Language Models on Following the Instruction Hierarchy](#)).

Three concepts should remain separate:

- **Instruction priority** is the behavior the model is intended to exhibit when model-visible texts conflict.
- **Source trust** is the harness's classification of who supplied a segment and what that source is allowed to influence.
- **Authorization** is an executable decision about whether a particular identity may perform a particular action on a resource.

Placing trusted instructions in the provider's higher-priority role can improve compliance. It does not give the model authority. Conversely, content returned by a tool or retrieved from a document should be classified as untrusted data even when it contains imperative language. Headings, XML tags, quoted blocks, and phrases such as “treat this as data” can reduce ordinary ambiguity, but they do not isolate that content like a parser or access-control boundary ([Foundations, Chapter 8](#)).

Prompt injection exploits this residual ambiguity. Hierarchy training and explicit data labeling can lower the chance that the model follows injected text; the executable controls in the next section limit consequences when the behavioral defense fails.

2.3 Model-Visible Policy and Executable Policy

Production systems normally need both kinds of policy, but they serve different purposes:

Model-visible policy	Executable policy
Describes desired behavior and decision criteria	Computes and enforces allow, deny, redact, constrain, or require-approval decisions
Helps the model choose a tool, abstain, ask a question, or propose escalation	Controls tool availability, arguments, credentials, files, network destinations, budgets, and side effects
Can be followed imperfectly	Must sit on a non-bypassable dispatch path for the guarantee it claims
Is evaluated as model behavior	Is tested as software and security policy

For example, the instruction “ask before sending an email” is useful model-visible guidance. The hard guarantee is a runtime gate that intercepts every email dispatch, checks identity and policy, and requires a valid approval record when the action is in scope. A model-generated statement that approval exists is not proof of approval. The same separation applies to file deletion, purchases, credential use, data export, and access to tenant-scoped retrieval.

[Chapter 7](#) develops sandboxing and policy enforcement points, [Chapter 14](#) distinguishes consultation from mandatory approval, and [Chapter 19](#) covers policy administration and distributed enforcement. The responsibility boundary is the same one summarized in Foundations: the model proposes; the surrounding system authorizes, validates, executes, and owns the consequences ([Foundations](#), [Chapter 14](#)).

2.4 Put Each Kind of Information in the Right Place

Not every relevant fact belongs in a system message. Overloading the instruction prefix consumes the finite context budget described in [Chapter 3](#), makes conflicts harder to diagnose, and increases the blast radius of every edit. A useful division of responsibility is:

- **System or developer instructions:** durable role, objectives, high-value behavioral rules, cross-tool guidance, and response contracts. Use the highest applicable provider role, but do not infer a security guarantee from that placement.
- **Tool descriptions and schemas:** what each tool does, its parameters, and the result contract. The complete invocation and enforcement path belongs to [Chapter 6](#).
- **Retrieved context:** current, large, tenant-scoped, or task-specific evidence. Preserve source and access-control metadata, and treat the returned content as untrusted input; [Chapter 4](#) covers the production data path.
- **Executable policy:** identity, resource permissions, argument constraints, approval requirements, sandbox limits, and egress rules enforced outside the model.

Duplicating the same rule across all four locations creates sources of truth that can drift. Prefer one canonical executable policy for hard controls, then derive concise model-visible guidance and human-facing explanations from the same policy reference where practical.

2.5 Dynamic Assembly Needs Provenance

Most production agents do not use one fixed prompt string. The harness assembles an ordered set of segments: provider or builder instructions, application policy guidance, project instructions, task-scoped skills, retrieved evidence, the user request, and prior observations. Dynamic assembly is safer to debug when the system records an assembly manifest outside the model context.

For each segment, the manifest should identify at least:

- a stable segment ID and content version or hash;
- source and owner;
- scope, such as global, tenant, repository, directory, task, or turn;
- applicable model and instruction role;
- creation or retrieval time when freshness matters; and
- the policy, artifact, or retrieval record from which the segment was derived.

The manifest supports questions that raw prompt text cannot answer reliably: Which version was active? Why was this segment in scope? Which source won a conflict? Was tenant-specific content mixed into another tenant’s call? It also lets traces refer to instruction versions without copying secrets or personal data into every observability record.

Assembly should reject or surface ambiguous conflicts rather than silently depend on accidental string order. A narrower scope may refine a broad convention, but it must not override a higher-trust restriction unless the executable

policy explicitly permits that relationship. Timestamps and request IDs should normally remain in the manifest or a volatile suffix; placing them near the beginning destroys a reusable prefix without adding authority.

2.6 Stable Prefixes and Provider Prompt Caches

Instruction layout can affect cost and latency when a provider offers cross-request prompt caching. This is a **provider prompt cache**, distinct from the per-request KV cache used during one generation. Providers expose different eligibility rules, cache breakpoints, retention periods, data controls, and prices, so those properties must be checked against the current API contract rather than stated as universal facts ([OpenAI — Prompt Caching](#), [Anthropic — Prompt Caching](#)).

Where reuse depends on matching a prefix, place stable, reusable material before volatile content. Base instructions and stable examples may belong near the front; the current task, fresh retrieval, request-specific identifiers, and timestamps belong later. Tool catalogs require an explicit strategy: a stable full catalog, deferred loading or tool search, and a stable meta-tool surface have different cache, context, and authorization tradeoffs, discussed in [Chapters 3 and 6](#).

Cacheability is not correctness. A byte-identical prefix can still contain stale or conflicting policy. Version changes that affect behavior should intentionally invalidate the relevant cached prefix, and cache telemetry should be interpreted using the provider's documented semantics.

2.7 Treat Instructions as Versioned, Evaluated Artifacts

Changing an instruction can change tool selection, refusal behavior, output structure, latency, and cost. Prompt changes therefore need an owner, version control, review, and evaluation against representative tasks. Compare outcomes and failure categories, not just whether the new wording looks clearer. [Chapter 11](#) defines trials and graders; [Chapter 17](#) shows how traces help locate the segment or interaction that contributed to a failure.

The evaluated unit is the full configuration, not the prompt in isolation: instruction version, model and provider settings, tool schemas, retrieval configuration, and executable policy all interact. A prompt tuned for one model may regress on another. Store the configuration identity with each trial and production trace, while applying appropriate access control and redaction to sensitive prompt content.

2.8 Choose the Right Altitude and Verifiable Outputs

Anthropic describes instruction design as choosing the *right altitude*. Instructions that are too low-level become a growing list of brittle special cases; instructions that are too abstract provide little signal for action ([Anthropic — Effective Context Engineering for AI Agents](#)). The useful middle specifies objectives, constraints, decision criteria, and observable completion conditions while leaving room for the model to handle variation.

When a failure appears in a trace, another sentence is only one possible fix. The better intervention may be a clearer tool schema, a deterministic validator, different retrieved evidence, a sandbox rule, or a new eval case. Move stable invariants into executable checks whenever they can be expressed deterministically.

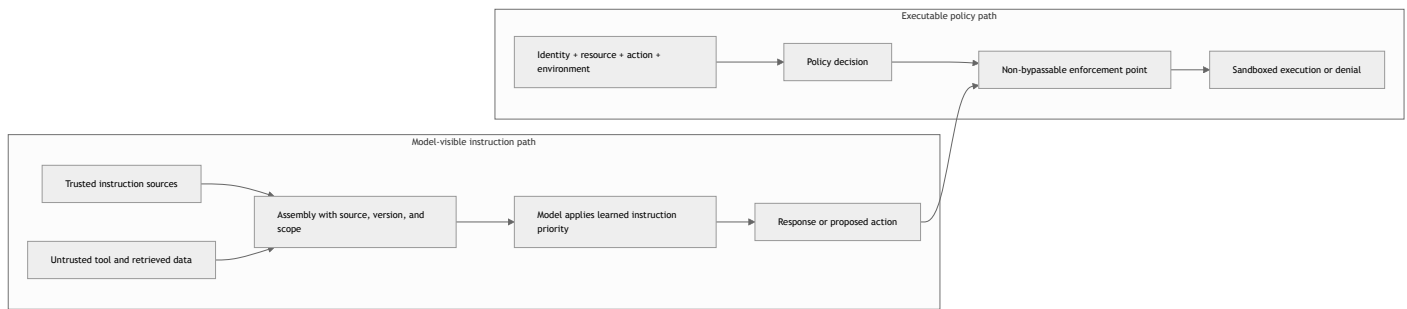
Output contracts should request artifacts that humans and software can inspect: a plan, current status, concise rationale, cited evidence, proposed tool arguments, test results, or a structured uncertainty field. Do not make access to hidden chain-of-thought a correctness or audit requirement. Visible reasoning is generated text and is not proof that the answer is correct or a faithful account of private model computation; verify claims and outcomes independently ([Foundations, Chapter 8](#)).

2.9 Layered and Scoped Instructions

Coding agents often combine builder instructions supplied by the agent product, organization or project instructions such as `AGENTS.md`, directory-scoped conventions, and task-specific skills. These layers form the model-visible instruction set, but their names do not by themselves establish trust. The harness should resolve scope and provenance before assembly, then measure whether the resulting configuration improves outcomes.

Scoped review rules are a useful example. A high-signal rule names a concrete defect, states where it applies, and explains how to recognize it—“flag a new endpoint that bypasses `authorize()`” rather than “follow security best practices.” Directory-level rules can keep guidance close to the code it governs, while linters and tests enforce deterministic invariants ([OpenAI — Custom Code Review Rules for Codex](#)). More instruction text is not automatically better: overlapping rules consume context, create conflicts, and can increase false positives.

Diagram: Behavioral Guidance and Executable Enforcement



The upper path shapes probabilistic behavior. The lower path enforces what may actually happen. Labels and priority help the first path; only the second can provide an execution guarantee.

Key Takeaways

- **Instruction priority is behavioral, not authoritative:** trained role priority and delimiters reduce confusion but do not grant permission or enforce policy.
- **Separate guidance from guarantees:** model-visible policy shapes proposals; executable policy authorizes and constrains dispatch.
- **Record assembly provenance:** version, source, scope, tenant, and policy references make dynamic prompts debuggable and auditable without exposing hidden reasoning.
- **Name the right cache:** cross-request reuse is a provider prompt-cache contract, not the per-request KV cache; matching, retention, and billing are provider-specific.
- **Version and evaluate the full configuration:** prompts interact with models, tools, retrieval, and runtime policy.
- **Ask for verifiable outputs:** plans, rationale, evidence, actions, and results are useful interfaces; hidden chain-of-thought is not a security or audit primitive.

Further Reading

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>
- Zhihan Zhang et al., *IHEval: Evaluating Language Models on Following the Instruction Hierarchy*, 2025. <https://arxiv.org/abs/2502.08745>
- OpenAI, *Prompt Caching*. <https://developers.openai.com/api/docs/guides/prompt-caching>
- Anthropic, *Prompt Caching*. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- OpenAI, *Custom Code Review Rules for Codex*, Jul 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>

Chapter 3: Context as a Finite Resource

This chapter owns per-call context selection and cache lifetimes; later chapters own retrieval pipelines, memory, and durable state.

3.1 Context Is a Budget, Not a Memory Store

Context is the token and multimodal representation visible to one model call. It is finite, assembled for a particular call, and should not be confused with durable execution state or memory. Foundations Chapter 9 explains the model-side limit; this chapter asks what the harness should place inside that limit.

More available context can help, but filling the window is not automatically useful. Anthropic describes context as an "attention budget": additional material can dilute relevant signals, and retrieval accuracy can vary with model, task, length, and information position ([Anthropic — Effective Context Engineering for AI Agents](#)). Treat this *context rot* as a probabilistic engineering risk, not as a fixed turn count or a law that every longer prompt performs worse.

A naive agent loop makes the problem visible:

1. assemble the instructions, tools, transcript, and current task data;
2. ask the model to propose an action;
3. execute the authorized action and append its result;
4. repeat with the enlarged transcript.

Only this append-only baseline grows monotonically. A production harness may trim low-value material, clear bulky tool results, offload artifacts, compact earlier turns, reset the context, retrieve information on demand, or defer tool definitions. Each transformation changes what the next call can see, so it needs explicit correctness criteria and recoverable references; Chapter 5 covers those loss and provenance questions in depth.

3.2 Assemble the Smallest Sufficient Context

The practical objective is the smallest set of high-signal inputs that is sufficient for the next decision, an approach Anthropic describes as finding the smallest set of tokens that maximizes the likelihood of the desired outcome ([Anthropic — Effective Context Engineering for AI Agents](#)). A useful assembly order is:

- stable instructions and behavioral guidance;
- only the tools that the model may need to discover;
- task state represented in a compact, structured form;
- recent actions and observations needed for local continuity;
- retrieved evidence and artifact references relevant to this step.

Structure helps navigation but does not create a security boundary. Markdown headings, XML tags, and clear labels can separate instructions from evidence, while authorization and mandatory policy checks remain responsibilities of the harness and runtime (see Chapters 2 and 7). Tool descriptions should also be distinct enough that the model can choose among them; overlapping or ambiguous interfaces consume context and increase selection errors ([Anthropic — Effective Context Engineering for AI Agents](#)).

Few-shot examples deserve the same discipline. Prefer a small set of representative examples that expose the intended decision boundary. A catalog of every edge case can crowd out the live task without guaranteeing generalization.

3.3 Three Different Objects: Context, KV Cache, and Prompt Cache

Three related concepts have different lifetimes:

Object	Lifetime and owner	What it means for the harness
Context	One model call; assembled by the harness	The complete model-visible input for that call. It is not durable state.
Per-request KV cache	One generation; managed by the inference system	Reuses attention key/value state for tokens already processed during that generation.
Cross-request provider prompt cache	Across eligible requests; exposed by a provider contract	Reuses an exact or otherwise provider-defined prefix. Eligibility, matching, breakpoints, retention, data controls, metrics, latency, and billing are provider-specific.

The third mechanism may internally reuse KV-derived state, but that implementation detail does not make it the same product contract as the per-request KV cache. OpenAI documents automatic caching for eligible exact prompt prefixes and exposes cached-token information in response usage, while Anthropic documents explicit cache breakpoints, prefix matching, and configurable cache lifetimes ([OpenAI — Prompt Caching](#); [Anthropic — Prompt Caching](#)). Code and prose should therefore say *provider prompt-cache hit* when discussing reuse between independent API requests.

Agent loops often make cross-request prompt caching valuable because a large prefix may remain stable while a short action and observation are appended. This is workload-dependent rather than universal. In a July 2025 report about Manus, Yichao “Peak” Ji described an average input-to-output ratio near 100:1 and cited then-current Claude Sonnet prices of \$0.30 per million cached input tokens versus \$3 per million uncached input tokens ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)). Those figures are a named historical case, not current pricing, a cross-provider ratio, or a forecast for another workload.

3.4 Design and Debug the Cacheable Prefix

Cache-aware design starts with the provider's contract. Both OpenAI and Anthropic require matching prefix content for reuse, but their eligibility rules, cache controls, lifetimes, and accounting differ; applications should consult the selected model's current documentation rather than encode a universal threshold or price ([OpenAI — Prompt Caching](#); [Anthropic — Prompt Caching](#)).

Put stable material before volatile material when the API's matching rules reward prefix stability. Keep timestamps, request identifiers, user-specific state, and current observations after the reusable boundary when semantics allow. Deterministic serialization matters too: changing field order or whitespace can change the serialized prefix even when the underlying object appears equivalent.

When a cache hit rate falls, check the following before changing the agent design:

- **Prefix boundary:** Which exact bytes or tokens are inside the reusable prefix?
- **Dynamic fields:** Did a timestamp, nonce, request ID, tenant value, or rotating instruction move into that prefix?
- **Tool definitions:** Did their descriptions, order, schemas, or availability change?
- **Serialization:** Is prompt and tool JSON produced deterministically?
- **Cache controls:** Does this provider require or support an explicit breakpoint, and is it placed correctly?
- **Provider metrics:** Are cached-input usage and latency measured from provider responses rather than inferred from total tokens?
- **Current contract:** Do the model, region, retention mode, and data-control settings remain eligible under the provider's current documentation?

3.5 Tool Catalogs: Three Valid Strategies

There is no universal rule that an agent must keep every tool definition permanently in context. MCP explicitly permits a server to notify clients that its tool list changed, so dynamic catalogs are supported by the protocol even though a particular model or cache design may make them costly ([MCP — Tools Specification](#)). Three patterns are common:

1. **Stable catalog plus action masking.** Keep definitions stable for model continuity and prompt-cache locality, but let a deterministic runtime reject actions that are unavailable in the current state. Manus reports using this pattern, including consistent name prefixes for related actions, in its own provider and workload setup ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)). Its “mask, don't remove” advice is a case study, not a protocol requirement.
2. **Deferred loading or tool search.** Keep a large catalog out of the initial model input and load relevant definitions when discovered. Anthropic's tool-search interface supports `defer_loading` and describes how deferred definitions are expanded when selected while remaining compatible with its prompt-caching design ([Anthropic — Tool Reference](#)).
3. **Stable meta-tool or code surface.** Expose a small, stable search/registry or code-execution interface and resolve a larger capability catalog behind it. This reduces schema volume but adds a discovery step and does not remove the need to validate and authorize the final capability at dispatch.

Choose among these patterns with evaluation data. Relevant variables include catalog size, model tool-selection quality, provider cache behavior, latency, token cost, permission scope, and the risk of a stale tool reference. A dynamic tool-list change must never carry forward authorization merely because an earlier version exposed a tool; Chapter 6 defines the invocation and authorization lifecycle.

3.6 Offload Working Material to Artifacts

Large observations, source documents, intermediate calculations, and generated outputs do not all belong in the live context. A filesystem or artifact store gives them durable, addressable locations. The context can then carry a short summary plus a path, URI, version, or content hash that lets the harness restore the underlying evidence when needed.

This storage is *working material*, not automatic memory. Without meaningful names, indexes, provenance, access checks, and retrieval conventions, an agent may fail to find an artifact or may retrieve one from the wrong task or tenant. LangChain describes the filesystem as a foundational harness primitive because it supports incremental offloading and collaboration, while still requiring the harness to decide what should return to context ([LangChain — The Anatomy of an Agent Harness](#)).

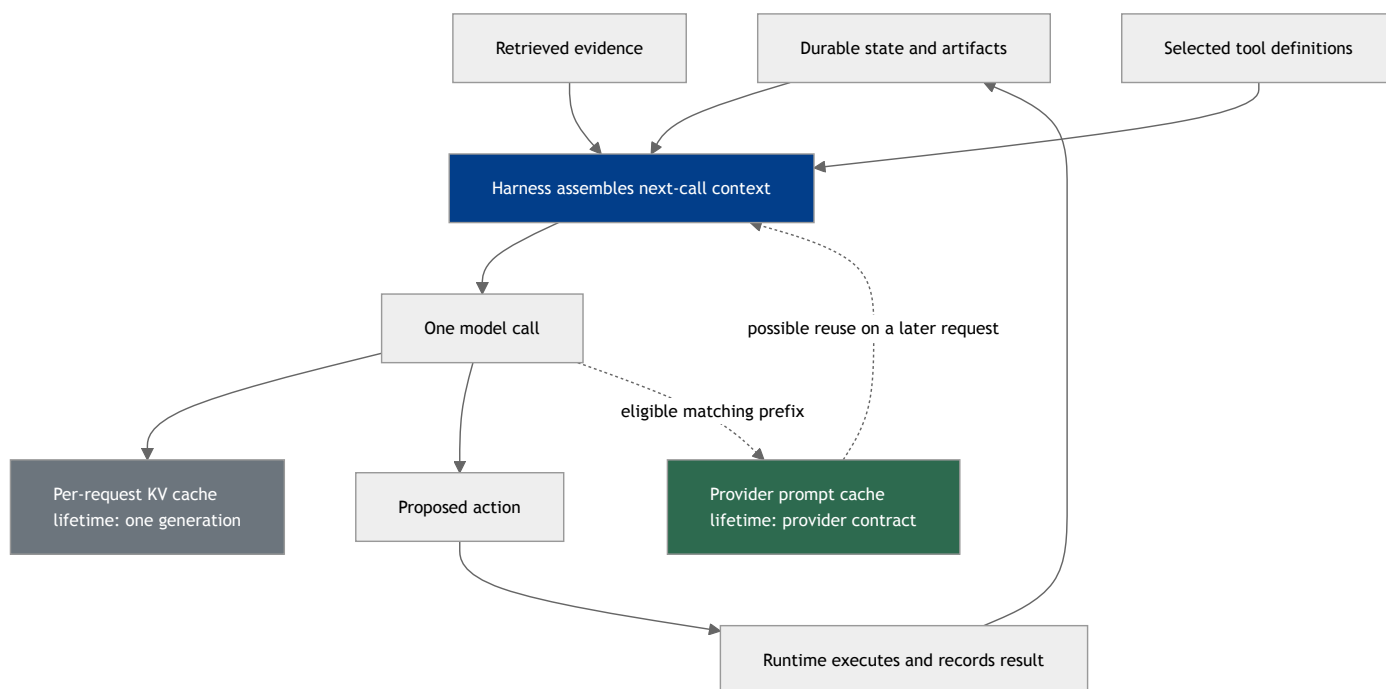
For coding agents, the repository is often the system of record for project facts. OpenAI's Codex harness guidance recommends keeping navigational instructions, design knowledge, and mechanically checked constraints close to the code so a fresh session can discover how the system works and how to verify changes ([OpenAI — Harness Engineering](#)). That principle favors short, local, discoverable documents and standard verification commands over a single ever-growing prompt.

3.7 Just-in-Time Selection, Not the Whole Retrieval Pipeline

Just-in-time retrieval is a context-selection technique: retain lightweight identifiers such as paths, links, and queries, then load the underlying material only when the next decision requires it ([Anthropic — Effective Context Engineering for AI Agents](#)). It complements precomputed retrieval rather than replacing it. A hybrid system might preload a small set of high-value instructions, retrieve indexed evidence for the task, and let the agent inspect artifacts on demand.

This chapter stops at the context boundary: what to load, when to load it, and what reference to retain after offloading. Chapter 4 follows the production data path—ingestion, parsing, chunking, ACL-aware indexing, freshness, hybrid retrieval, reranking, provenance, and retrieval evaluation. Foundations Chapter 11 supplies the embedding and retrieval concepts on which that pipeline depends.

Diagram: Context Selection and Cache Lifetimes



Key Takeaways

- **Context is per-call input, not durable memory:** the harness selects it from state, artifacts, retrieved evidence, and recent interaction.
- **Only a naive append-only transcript grows monotonically:** production loops may trim, offload, compact, reset, retrieve, or defer tools.

- **KV cache and provider prompt cache have different lifetimes:** use the provider term for cross-request reuse and treat its rules and prices as provider-specific.
- **Stable tools are one strategy, not a law:** compare masking, deferred loading/tool search, and a stable meta-tool with workload-specific evaluations.
- **Artifacts make context restorable:** paths and provenance let large material leave the window without becoming irrecoverable.
- **Just-in-time retrieval is context selection:** Chapter 4 owns the production retrieval pipeline.

Further Reading

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- OpenAI, *Prompt Caching*. <https://developers.openai.com/api/docs/guides/prompt-caching>
- Anthropic, *Prompt Caching*. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- Model Context Protocol, *Tools Specification*, Jun 2025. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- Anthropic, *Tool Reference*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-reference>
- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>

Chapter 4: Production Retrieval and Grounding

Foundations Chapter 11 explains retrieval embeddings, lexical and dense search, chunking, reranking, and retrieval metrics. This chapter takes over at the production boundary: how a harness turns changing, permissioned source data into evidence that a model may use safely and that an operator can trace back to its origin.

Retrieval-augmented generation changes a model's input rather than its weights: a system retrieves external information and supplies it for the current generation ([Lewis et al. — Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#)). In production, that apparently simple operation is a data lifecycle, an authorization decision, a context-selection step, and an evaluation problem.

4.1 Retrieval, Grounding, and Outcome Are Different Claims

This book uses three terms deliberately:

- **Retrieval** selects candidate evidence from a governed corpus for a particular query, identity, and task.
- **Grounding** asks whether the generated claims are supported by the evidence actually supplied to the model.
- **Task outcome** asks whether the overall workflow produced the right external result.

A relevant chunk can be retrieved but ignored. An answer can faithfully summarize a retrieved chunk that is stale or wrong. A well-cited answer can still fail the user's task. Conversely, an answer may be correct from parametric knowledge while lacking support in the retrieved corpus. These are separate failure modes and need separate graders.

The harness should therefore treat retrieval output as a structured evidence product, not as an anonymous string:

```
EvidenceItem {  
    source_id, source_version, chunk_id,  
    title, location, content_hash,  
    retrieved_at, index_version,  
    acl_scope, retrieval_scores,  
    text  
}
```

The exact schema is application-specific. The invariant is that text remains attached to source identity, version, location, authorization scope, and the index that produced it. Those fields make later citation checks, deletion, incident review, and reproducible evaluation possible.

4.2 Build an Index from Governed Source Records

An indexing pipeline should begin from a declared system of record rather than from whichever copy a crawler happens to find. A practical source record includes a stable source ID, source version or modification marker, tenant, content type, owner, access policy or ACL reference, and deletion status. The ingest job records when and how it observed that state.

The production path is:

1. **Ingest:** enumerate or receive source changes, preserving source identity and permissions.
2. **Parse:** extract text and structure while retaining page, heading, table, code-symbol, or timestamp locations.
3. **Chunk:** create retrievable units without detaching claims from necessary qualifiers.
4. **Annotate:** attach provenance, tenant, ACL, language, timestamps, document type, and other filterable metadata.
5. **Deduplicate:** identify exact copies by content hash and near-duplicates by an evaluated rule; retain every source relationship even if only one body is indexed.
6. **Represent:** create the lexical fields and retrieval embeddings required by the chosen retrievers.
7. **Publish:** write immutable records to a named index version and make that version eligible for serving only after validation.

Chunk size, boundary, overlap, and document context affect retrieval behavior; they are tuning variables, not neutral preprocessing details. Anthropic's Contextual Retrieval report, for example, prepends document-specific context before creating both embeddings and a BM25 index, but that is one evaluated preprocessing design rather than a universal requirement ([Anthropic — Contextual Retrieval](#)).

Record the versions of the parser, chunker, metadata schema, embedding model and mode, lexical analyzer, and contextualization prompt or model. Without that manifest, “the index changed” cannot be decomposed into a corpus change, a preprocessing change, or a retrieval-model change.

Deduplication must not erase provenance. If the same policy paragraph appears in three controlled sources, the index may avoid sending all three copies to the model, but it still needs to know which source versions contain the text and which access rules apply. A content hash proves equality of bytes under a chosen normalization; it does not prove that two source records have the same authority or lifecycle.

4.3 Execute an ACL-Aware Query Path

At query time, the harness assembles more than a search string. It binds the request to a tenant and principal, derives mandatory filters, formulates one or more retrieval queries, and carries the current index and policy versions through the result.

A robust query path has these stages:

1. **Formulate the query:** use the user's request plus the minimum workflow state needed for retrieval; preserve the original query alongside any rewrite.
2. **Apply scope:** select the tenant, corpus, time range, document type, and authorization filters before evidence is exposed.
3. **Generate candidates:** run dense and/or sparse retrieval according to the workload.
4. **Fuse:** combine ranks or normalized scores; raw scores from different retrievers are not automatically comparable.
5. **Rerank:** apply a more expensive relevance model to a bounded candidate set when evaluation justifies the extra work.
6. **Deduplicate and diversify:** avoid spending result slots and context on equivalent passages when the task needs broader evidence.
7. **Assemble context:** select evidence within a token budget, preserve source labels, and keep quoted evidence distinct from instructions.
8. **Return evidence metadata:** make source and version identifiers available to the model-facing prompt and to downstream citation validation.

Dense and lexical retrieval capture different signals, and reranking can only reorder candidates that first-stage retrieval supplied. Anthropic's published implementation combines embeddings, BM25, rank fusion, deduplication, and reranking in this order; its measured results belong to its evaluated corpora, models, candidate counts, and metric, not to every RAG system ([Anthropic — Contextual Retrieval](#)).

Query rewriting is also fallible. Log the original query, each derived query, the filters, candidate IDs, stage scores, and selected evidence. A rewrite that silently drops an identifier or negation can make the search look healthy while making the result irrelevant.

4.4 Freshness, Deletion, and Index Migration

An index is a derived view, not the source of truth. Its lifecycle should make four states observable for each source record: current, pending update, tombstoned, or failed. “The ingestion job ran” is weaker than “all changes through source watermark W are represented in serving index V.”

Define freshness service levels by source class. A product catalog may tolerate minutes; revoked access or a deleted confidential document may require exclusion from retrieval before the slower text and vector cleanup completes. The safe pattern is to make a tombstone or deny marker effective on the serving path first, then perform physical deletion and compaction asynchronously.

Re-index when a parser, chunker, embedding model, lexical analyzer, or schema change makes old and new records incompatible. Build and validate a new immutable index version, compare it with the current version on a fixed eval set, and then switch the serving pointer. Elasticsearch index aliases are one vendor-specific example of changing which index an application uses and supporting reindexing without downtime; the architectural lesson is versioned publication, not dependence on that API ([Elastic — Aliases](#)).

Every migration needs explicit handling for:

- **Backfill:** can every live source record be reproduced in the new schema?
- **Dual writes or change capture:** how are edits arriving during the backfill applied to both versions?
- **Validation:** do document counts, ACL coverage, sampled content hashes, and retrieval evals pass?

- **Cutover and rollback:** which serving version handled each request, and can traffic return to the prior version?
- **Retirement:** when may the old index and its sensitive copies be deleted?

A source deletion, ACL revocation, or legal hold is not merely another relevance update. Track it through ingestion, index publication, caches, replicas, backups, and citation surfaces with an auditable completion state.

4.5 Enforce Permission at Retrieval

Permission checks belong on the retrieval path, not only in the interface that displays the final answer. Bind the authenticated principal, tenant, groups or roles, resource scope, purpose where applicable, and policy version to the search request. Only authorized candidates should be eligible for context assembly.

Microsoft's documented security-filter pattern illustrates the mechanism: principal identifiers are stored in filterable metadata and supplied in the query so nonmatching documents are excluded from results. Microsoft also warns that this pattern is string-based filtering rather than authentication by itself, so the application must authenticate the caller and construct the filter correctly ([Microsoft — Security filters for trimming results in Azure AI Search](#)).

Treat the following as correctness invariants:

- every chunk inherits the current permission semantics of its source;
- ACL changes propagate on a measured, high-priority path;
- all retrieval modes—including dense, sparse, hybrid, reranking, cache hits, and direct document lookup—apply equivalent scope;
- a reranker never receives candidates the calling service is not allowed to disclose;
- evidence caches vary by tenant and authorization scope as well as query and index version;
- citations resolve through an authorization check at view time instead of becoming permanent capability URLs.

Post-filtering an already assembled top- k list is not equivalent to permission-aware retrieval: it can leave too few authorized candidates and may expose restricted text to intermediate services, logs, or models. If a search backend cannot enforce the policy directly, place an unavoidable policy enforcement point before any untrusted consumer sees candidate content and retrieve enough authorized candidates under that design.

Tenant isolation is a system property, not just a metadata field. Depending on risk, it may require separate indexes, encryption keys, service identities, network boundaries, or retention policies. Whichever design is chosen, cross-tenant negative tests belong in every retrieval release suite.

4.6 Retrieved Content Is Untrusted Content

Retrieval relevance does not grant instruction authority. A web page, uploaded PDF, ticket, code comment, or internal document can contain text that attempts to redirect the model or cause tool use. NIST defines prompt injection as an attack that exploits the concatenation of untrusted input with a higher-trust prompt, which is exactly the boundary a retrieval system creates ([NIST AI 100-2e2025 — Adversarial Machine Learning Taxonomy](#)).

The harness should:

- label retrieved spans as evidence, never as system or developer instructions;
- preserve source and trust metadata through query rewriting, reranking, and context assembly;
- delimit evidence for model navigation without claiming that delimiters are a security boundary;
- minimize content to what the current question needs;
- require normal authorization and mandatory approval gates for any action suggested by retrieved text;
- keep secrets and high-impact tools outside the model's authority even when a passage requests them;
- record which evidence preceded an action so an incident can be reconstructed.

Instruction priority is a behavioral defense discussed in Chapter 2. Sandboxes, credential brokers, allowlists, policy enforcement points, and approval gates in Chapter 7 provide the executable boundary. Retrieval filtering can reduce exposure to known malicious material, but a classifier or blocklist is not a proof that the remaining text is safe.

4.7 Preserve Citations and Provenance

A citation has at least two independent correctness conditions:

1. **Attribution:** the citation resolves to the source version and location actually supplied.
2. **Support:** that passage supports the claim attached to it.

Research on citation-enabled generation evaluates citation correctness and completeness separately from general answer quality, reinforcing that adding source markers is not enough ([Gao et al. — Enabling Large Language Models to Generate Text with Citations](#)).

Give the model stable evidence IDs rather than asking it to reproduce arbitrary URLs. After generation, parse claim-to-evidence links, verify that every ID came from the authorized evidence set, and render a human-facing citation from the stored source record. Where stakes justify it, use entailment checks or human review to test support; do not relabel a retrieved passage as “verified” merely because it ranked highly.

Provenance should survive answer generation. A stored output can point to `source_id + source_version + location + content_hash`, while the interface separately resolves the latest authorized view. This lets an operator distinguish “what the model saw then” from “what the source says now.”

4.8 Evaluate the Pipeline, Including No-Answer Cases

Tune retrieval with end-to-end trials rather than one aggregate relevance number. An evaluation harness should pin the corpus snapshot, index version, identity and ACL set, query transformations, model versions, and grader versions. Anthropic's agent-evaluation guidance similarly treats the environment, trial, transcript, outcome, and graders as distinct components that must be defined and stabilized ([Anthropic — Demystifying Evals for AI Agents](#)).

Use a layered scorecard:

Layer	Questions and example measurements
Corpus and index	Are expected sources present, current, deduplicated, and permission-complete? What is deletion and ACL-propagation lag?
Candidate retrieval	Recall@ k , Precision@ k , MRR, NDCG, and misses by query cohort; Foundations Chapter 11 defines these metrics.
Selection	Does fusion or reranking improve useful evidence within the context budget? How often are results duplicated or mutually conflicting?
Abstention	When the corpus lacks support or all candidates fall outside the authorized scope, does the system return a calibrated no-evidence state rather than fabricate support?
Grounding and citation	Which claims are supported, unsupported, contradicted, or missing citations? Do citations resolve to the exact source version used?
Task outcome	Did the answer or agent workflow solve the task, and did it avoid disallowed actions or data disclosure?
Operations	Ingestion lag, query latency by stage, failure rate, tokens, reranker calls, and cost per successful outcome.

Include answerable, unanswerable, stale-source, conflicting-source, deleted-source, revoked-ACL, cross-tenant, exact-identifier, paraphrase, and adversarial-document cases. A no-answer decision is part of the task definition, not an exception to omit from the dataset.

Tune candidate count, final top- k , fusion weights, reranker cutoff, and no-evidence threshold together. Larger candidate sets may improve recall but increase search and reranking work; more final passages consume model context and can add distraction. Anthropic reports this latency/cost/quality tradeoff for its reranking setup and explicitly recommends workload-specific experiments ([Anthropic — Contextual Retrieval](#)). Raw similarity or reranker scores should not be treated as portable probabilities; calibrate thresholds for the exact model, corpus, index, and query cohorts.

Report both stage latency and end-to-end latency. At minimum, separate query formulation, embedding, sparse and dense search, fusion, reranking, context assembly, and generation. Optimize cost per successful, policy-compliant outcome—not merely cost per query or Recall@ k .

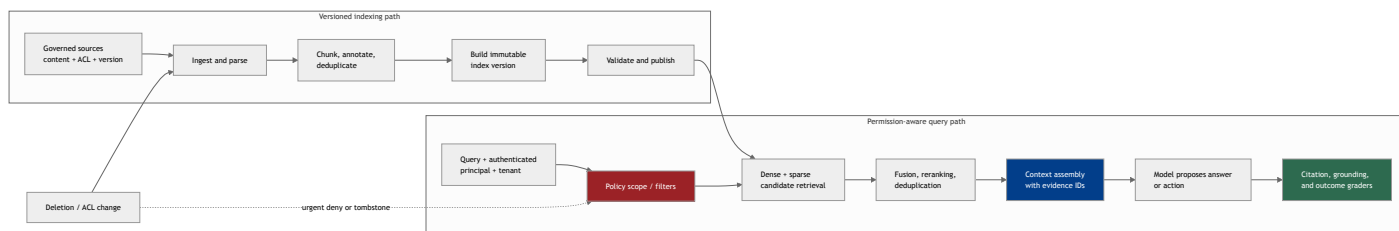
4.9 Case Study: Contextual Retrieval, Not a Universal Baseline

Anthropic's 2024 Contextual Retrieval report is a useful named example of a combined pipeline. It added chunk-specific context before embedding and BM25 indexing, fused lexical and dense candidates, deduplicated them, and tested a reranking stage. Its published improvements were measured with particular corpora, embedding configurations, candidate counts, top-20 recall-based failure rate, and a specific reranker ([Anthropic — Contextual Retrieval](#)).

The transferable lesson is the experimental method: define the corpus and metric, vary one pipeline component, measure retrieval and downstream behavior, and include cost and latency. The reported percentage improvements

and preferred top- k are not defaults for another corpus. A production team should reproduce the comparison on its own documents, permissions, questions, and outcome graders before adopting the design.

Diagram: The Governed Retrieval Path



Key Takeaways

- **Retrieval is a governed data pipeline:** ingestion, parsing, chunking, metadata, ACLs, index versions, and deletion are part of answer correctness.
- **Permission applies before evidence exposure:** bind identity and tenant to every retrieval mode, cache, reranker, and citation resolver.
- **Retrieved text is untrusted:** relevance does not give a passage instruction authority or permission to cause side effects.
- **Evidence needs durable provenance:** preserve source version, location, content hash, index version, and authorization scope.
- **Retrieval, grounding, citation, and outcome require different graders:** success at one layer does not prove success at the next.
- **Tune the whole system:** top- k , thresholds, reranking, abstention, latency, and cost are workload-specific decisions.
- **Named results remain named cases:** Anthropic's Contextual Retrieval findings motivate an experiment; they are not universal performance guarantees.

Further Reading

- Patrick Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. <https://arxiv.org/abs/2005.11401>
- Anthropic, *Introducing Contextual Retrieval*, Sep 2024. <https://www.anthropic.com/engineering/contextual-retrieval>
- Microsoft, *Security filters for trimming results in Azure AI Search*. <https://learn.microsoft.com/en-us/azure/search/search-security-trimming-for-azure-search>
- Elastic, *Aliases*. <https://www.elastic.co/guide/en/elasticsearch/reference/current/aliases.html>
- NIST, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, NIST AI 100-2e2025. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-2e2025.pdf>
- Tianyu Gao et al., *Enabling Large Language Models to Generate Text with Citations*, 2023. <https://arxiv.org/abs/2305.14627>
- Anthropic, *Demystifying Evals for AI Agents*. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>

Chapter 5: Compaction, Memory, and Context Handoffs

A long-horizon task can outlive any one model call and eventually any one context window. Continuity therefore cannot mean “keep appending the transcript.” The harness must transform what the next call sees while preserving enough evidence to recover what was omitted. Foundations Chapter 9 establishes that context is finite, temporary input rather than persistent state; this chapter develops the harness mechanisms that follow from that boundary ([LLM Foundations, Chapter 9](#)).

The central design question is not simply how to make context shorter. It is:

What may be lost now, what must remain authoritative elsewhere, and how can a later call recover the evidence?

5.1 Five Objects That Must Not Collapse into One

Context engineering becomes unsafe when every form of continuity is called “memory.” This book uses five distinct objects:

Object	Meaning	Source of authority
Context	The token and multimodal representation visible to one model call	The harness assembles it for that call; it expires as model-visible input when the call ends
Memory	Information stored so that a future call can retrieve or receive it	A memory store with explicit scope, provenance, update, access, and forgetting rules
Execution state	Structured workflow facts such as the current step, pending action, budget, and approval status	The application or runtime state machine
Artifact	An addressable work product such as a file, diff, report, dataset, build, or recorded tool output	The artifact store or underlying system of record, identified by a path, URI, version, or content hash
Event history	An ordered record used to recover durable workflow execution	The runtime's durable history; a replay claim requires recorded model and tool results to be reused rather than silently recomputed

The model-side distinction between temporary context and persistent state comes from Foundations Chapters 9 and 14 ([LLM Foundations, Chapter 9](#); [LLM Foundations, Chapter 14](#)). Temporal provides one concrete durable-execution design in which persisted workflow-history events are used to reconstruct workflow state after failure ([Temporal — History Service](#)). That implementation is an example, not a requirement that every harness use Temporal.

A compaction summary may become context for the next call, an artifact for inspection, and a candidate memory for later retrieval. It is still not automatically the authoritative execution state or the event history. If the summary says an approval was granted but the runtime state says it is pending, the runtime state wins. Chapter 10 owns execution state, checkpoints, and event histories; this chapter stays with context transformations, memory, and handoff artifacts.

5.2 Transformations Have Different Loss Models

The harness has several ways to keep a task within its context budget. They are not interchangeable:

Technique	Transformation	What can be lost or go wrong	Recovery path
Tool-result clearing	Replace a bulky observation with a compact conclusion and pointer	Raw fields, ordering, warnings, or an error detail may disappear	Reload the recorded result or artifact
Recitation	Restate a goal, plan, or constraint near the end of the current context	Nothing is intentionally removed, but stale or incorrect material can be amplified	Compare with authoritative task state and source evidence
Compaction	Summarize selected earlier context into a smaller representation	Omitted rationale, minority evidence, exact wording, provenance, or uncertainty	Follow citations and artifact pointers back to primary material
Context reset plus handoff	Start a new context from a deliberately constructed handoff package	Any fact absent from the package is no longer model-visible	Read authoritative state, artifacts, event history, or memory on demand
Retrieval	Select external material and inject it into a later call	Relevant evidence can be missed; stale, unauthorized, or misleading material can be selected	Re-query, inspect provenance, and evaluate the retrieval pipeline

Compaction and clearing reduce material already in context. Reset changes the continuity boundary. Retrieval brings selected external material back. Recitation changes salience without creating persistence. Chapter 4 covers retrieval quality and permission-aware production pipelines; Chapter 3 covers the per-call budget and provider prompt cache behavior.

These choices also have different cache consequences. Rewriting an early prefix may prevent a cross-request **provider prompt cache** match, but matching, retention, and billing are provider-specific contracts rather than correctness guarantees ([OpenAI — Prompt Caching](#); [Anthropic — Prompt Caching](#)). Never retain stale or unsafe content merely to preserve a cache hit.

5.3 Compaction Is a Lossy, Testable Contract

Compaction summarizes older interaction as the context approaches a chosen budget, then uses the summary as part of a later context. Anthropic recommends tuning compaction prompts on realistic, complex trajectories: first improve recall so continuation-critical information survives, then improve precision by removing material that no longer helps ([Anthropic — Effective Context Engineering for AI Agents](#)). A trigger based on token count or context utilization is an implementation policy, not a universal threshold; triggering too late risks exhausting the window, while triggering too early creates avoidable information loss.

A production compaction should have a schema. At minimum, preserve:

- **decisions** already made, including rationale when it constrains later work;
- **open tasks** and the next expected action, without marking incomplete work as done;
- **constraints**, including user requirements and references to applicable policy checks;
- **artifact pointers**, preferably with a version, commit, or content hash;
- **provenance**, linking important claims to their source observations or documents;
- **unresolved uncertainty**, including conflicting hypotheses and evidence still needed.

Recent actionable failures may also belong in the next context, but they need a lifecycle. Retain the latest error when its exact message, failed arguments, or stack trace can change the next attempt. If the same failure repeats, replace duplicate output with a diagnosis, retry count, and pointer to the raw log. Manus describes retaining useful failures so the model can adapt, while the 12-Factor Agents formulation recommends compacting errors into the next context ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#); [HumanLayer — 12-Factor Agents](#)). Neither practice justifies keeping unlimited logs in the live window.

Compaction should be evaluated as a transformation, not judged by how polished its prose sounds. Useful tests resume representative tasks from the compacted package and measure whether the agent preserves required constraints, identifies the next step, locates cited artifacts, retains calibrated uncertainty, and avoids repeating resolved or exhausted work. Compare against an uncompressed baseline where feasible, and include adversarial cases in which an important caveat appears only once or conflicts with the majority of the trace.

5.4 Memory Is a Governed Information Product

Structured note-taking turns selected information into a reusable product. Anthropic describes agents maintaining progress notes, maps, and strategies across thousands of game steps, then reloading those notes after context reset ([Anthropic — Effective Context Engineering for AI Agents](#)). The important mechanism is not the filename or prose format. It is the write–retrieve lifecycle around the note.

Each memory item should answer:

- **Scope:** which task, user, tenant, repository, or agent may use it?
- **Provenance:** which observation, source, identity, and tool produced it?
- **Type:** is it an observation, a user preference, a derived conclusion, or an instruction candidate?
- **Freshness:** when was it created, when must it be revalidated, and what can supersede it?
- **Authority:** who authorized the write, and may the future reader access the underlying evidence?
- **Forgetting:** how is it expired, deleted, rolled back, or corrected?

Memory is not automatically true because it is persistent. Retrieval can select the wrong item, consolidation can merge incompatible facts, and a once-correct item can become stale. MemGPT explores virtual context management by moving information between a bounded model context and external storage ([Packer et al. — MemGPT: Towards LLMs as Operating Systems](#)). Mem0 explores extracting, consolidating, and retrieving memories rather than replaying a complete multi-session history ([Chhikara et al. — Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory](#)). These are useful architectural case studies, not evidence that every short task needs a memory subsystem.

Persistent memory is also a trust boundary. Untrusted text copied from a webpage into a shared note can influence later runs long after the original context is gone. Treat observations as data, never promote retrieved content into policy merely because a model summarized it, and re-check authorization when reading as well as writing. Anthropic describes this cross-run risk as persistent memory poisoning and emphasizes containment around agent-accessible state ([Anthropic — How We Contain Claude](#)).

5.5 Recitation and Clearing Serve Local Continuity

Recitation deliberately rewrites a short task representation—often a checklist—near the end of the current context. Manus reports using a repeatedly updated `todo.md` to keep a long task's goals salient during action-heavy trajectories ([Manus — Context Engineering for AI Agents: Lessons from Building Manus](#)). This is an attention-management technique, not durable state and not a correctness mechanism. The checklist should be regenerated from or reconciled with authoritative task state; otherwise recitation can keep an obsolete plan salient.

The motivation is consistent with the empirical “lost in the middle” result: in the evaluated retrieval and question-answering settings, model performance varied with the position of relevant information in long inputs ([Liu et al. — Lost in the Middle](#)). That result does not prove that every model or task has the same positional curve, so recitation should be validated on the target workload.

Tool-result clearing is the narrowest lossy operation. Clear a result only after preserving what the next decision needs: the derived conclusion, any material warning or unresolved error, the action taken because of it, and a restorable pointer to the original output. Clearing is especially risky before a side effect has been confirmed, while a result is disputed, or when exact fields may be needed for audit or debugging. The raw output belongs in an artifact store or appropriate record, not indefinitely in the live context.

5.6 Handoffs Are Typed Interfaces, Not Trusted Summaries

A context handoff occurs whenever work crosses a context boundary: after reset, between sessions, from a sub-agent to a parent, or from one model configuration to another. Treat it like an interface with a schema rather than an informal paragraph.

A robust handoff package contains:

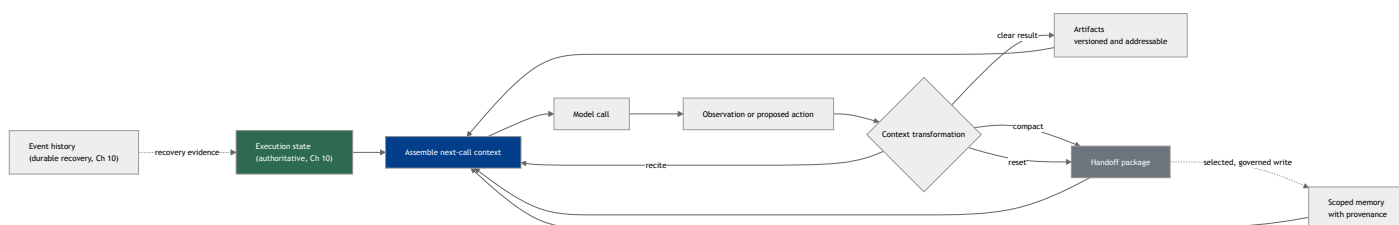
1. **Task identity and bounded scope** — what was requested, what was deliberately excluded, and the completion criterion.
2. **Worker identity and authority scope** — which agent/model/tooling performed the work and which resources it could access.
3. **Claims and decisions** — the concise result, clearly separated from observations and proposals.

4. **Evidence and provenance** — inline citations plus addressable artifacts, versions, queries, commands, or tool-call identifiers.
5. **External effects** — actions attempted, confirmed outcomes, failures, and any pending approval.
6. **Uncertainty and open questions** — confidence limits, conflicts, missing evidence, and assumptions.
7. **Verification status** — tests or checks actually run, with their results and pointers to complete output.
8. **Next action** — what the receiver should do, including any revalidation required before a privileged action.

Sub-agents are valuable context firewalls because they can perform a tool-heavy investigation in an independent window and return a compact result to the parent (Anthropic — Effective Context Engineering for AI Agents; HumanLayer — Skill Issue: Harness Engineering for Coding Agents). The firewall removes intermediate noise; it does not make the result more trustworthy.

In particular, a high-privilege parent must not treat a low-privilege worker's summary as authorization or verified fact. The parent should inspect the cited evidence, re-check current policy at action dispatch, and independently confirm high-impact outcomes. Identity and scope must travel with the result so the receiver knows what the worker could and could not observe. A handoff artifact can inform execution state, but only the runtime or application may commit that state transition; Chapters 6, 7, and 19 develop dispatch, enforcement, and fleet identity.

5.7 A Context-Continuity Loop



The dashed memory edge is deliberate: not every summary should become durable memory. Promotion requires an explicit write policy, provenance, scope, and lifecycle. Likewise, the event history supports recovery but is not simply poured into every model call.

Key Takeaways

- **Context, memory, execution state, artifacts, and event history are different objects:** a summary can reference authoritative state but does not replace it.
- **Every context transformation has a loss model:** clearing, recitation, compaction, reset, and retrieval fail in different ways and need different recovery paths.
- **Compaction needs a schema and an evaluation:** preserve decisions, open tasks, constraints, artifact pointers, provenance, and unresolved uncertainty.
- **Useful errors have a lifecycle:** retain recent actionable detail, compact repeated noise, and keep raw evidence recoverable.
- **Memory is governed, scoped, and revisable:** persistence does not make a model-generated note true or authorized.
- **Recitation changes salience, not authority:** reconcile repeated plans with the actual task state.
- **A context firewall is not a trust firewall:** sub-agent results need evidence, identity and authority scope, uncertainty, verification status, and addressable artifacts.
- **A high-privilege parent must revalidate before acting:** a worker's summary is neither authorization nor confirmed outcome.
- **Use provider prompt cache for cross-request reuse:** never call it the per-request KV cache or sacrifice correctness merely to preserve a hit.

Further Reading

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- OpenAI, *Prompt Caching*. <https://developers.openai.com/api/docs/guides/prompt-caching>
- Anthropic, *Prompt Caching*. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>

- Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.08560>
- Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
- Nelson F. Liu et al., *Lost in the Middle: How Language Models Use Long Contexts*, TACL, 2024. <https://arxiv.org/abs/2307.03172>
- Anthropic Safeguards Research Team, *How We Contain Claude*, Anthropic, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>
- Temporal, *History Service*. <https://github.com/temporalio/temporal/blob/main/docs/architecture/history-service.md>

Chapter 6: Tools and the Invocation Lifecycle

Foundations Chapter 12 stops at the model boundary: a tool call is structured output proposed by the model, and a tool result returns as potentially untrusted context. This chapter follows that proposal through the harness. The central rule is simple: **a well-formed tool call is neither permission nor proof of success**.

6.1 From Model Proposal to External Effect

For client-executed tools, the model emits a structured request, application code performs the operation, and the result is sent back to the model. Anthropic describes this explicitly as a contract in which the model never executes the operation itself; OpenAI likewise distinguishes a model-generated tool call from the tool-call output produced by the application ([Anthropic — How Tool Use Works](#); [OpenAI — Function Calling](#)). A provider may host some tools and hide part of this round trip, but the responsibility boundary remains: model output proposes; an execution system decides and acts.

A production invocation should pass through an explicit lifecycle:

Stage	Primary owner	Required record	Retry meaning
1. Receive and correlate	Harness adapter	provider response ID, provider call ID, harness invocation ID	No execution yet
2. Complete streamed arguments	Harness adapter	completion event and assembled bytes	Reconnect or abandon; do not dispatch fragments
3. Parse	Parser	parsed value or syntax error	Ask for a corrected call; do not execute
4. Validate schema	Validator	schema version and field errors	Correct arguments; do not execute
5. Validate semantics	Domain adapter	resolved resources, preconditions, bounds	Refresh state or ask for correction
6. Authorize	Dispatcher / policy enforcement point	actor, capability, resource, policy version, decision	Re-evaluate on changed identity, resource, tool, or policy
7. Obtain mandatory approval	Runtime / product	approver, exact action preview, expiry, decision	Never infer approval from model text
8. Establish idempotency and deduplication	Dispatcher	idempotency key, prior-attempt lookup	Resume or return the known result instead of duplicating an effect
9. Execute	Tool adapter or remote service	attempt ID, request, start time	Governed by side-effect semantics
10. Enforce timeout and cancellation	Runtime	deadline, cancellation state	A timeout creates an unknown outcome until checked
11. Classify completion	Tool adapter	success, retryable failure, terminal failure, partial success, or unknown	Retry only under an explicit policy
12. Normalize the result	Adapter	stable result/error envelope and artifact references	No model-specific ad hoc shape
13. Return an observation	Harness	result linked to the original call ID	Observation is untrusted context, not execution authority
14. Confirm the external outcome	Verifier / environment	postcondition evidence and final status	Reconcile before any consequential retry

Not every read-only lookup needs a human approval or a separate postcondition query. The lifecycle is a checklist of distinct decisions, not a demand for fourteen network round trips. Stages may collapse inside one trusted component, but their evidence and failure semantics should remain distinguishable.

6.2 Correlation, Streaming, and Parallel Calls

Every invocation needs a stable identity. Preserve the provider's call ID and add a harness-level invocation ID when work can cross queues, retries, providers, or process boundaries. Results must correlate to the specific call, never merely to a tool name or list position. OpenAI tool-call outputs reference the originating `call_id`; Anthropic tool

results similarly use `tool_use_id` to match a particular `tool_use` block ([OpenAI — Function Calling](#); [Anthropic — Handle Tool Calls](#)).

Streaming adds a completion boundary. Argument deltas are an in-progress serialization, not a sequence of executable commands. OpenAI's streaming contract emits argument-delta events followed by a completed event containing the assembled arguments; the harness should wait for the provider's completion signal, assemble by call identity, and only then parse and validate ([OpenAI — Function Calling, Streaming](#)). Dispatching a syntactically plausible prefix such as `{"recipient":"alice"}` risks acting on truncated or subsequently revised intent.

Parallel calls need independent state machines. If three calls are proposed together and one succeeds, one fails validation, and one times out, record three outcomes. Do not collapse them into a single turn-level Boolean. Return each completed observation under its own call ID and keep unresolved calls explicit; Anthropic's API, for example, requires each client `tool_use` block to receive its corresponding `tool_result` in the next message ([Anthropic — Handle Tool Calls](#)).

A useful internal envelope is:

```
invocation_id, provider_call_id, tool_name, tool_version
actor, tenant, capability, policy_version, approval_id
arguments_hash, idempotency_key, attempt_id, deadline
status, normalized_result, artifacts, postcondition_evidence
```

[Chapter 10](#) defines the durable `session → run → step → action → attempt` hierarchy. A dispatcher may alias `invocation_id` to `action_id` only when one invocation represents exactly one intended effect; a composite invocation must preserve its contained action and attempt identities rather than hiding them behind one opaque ID.

This is execution state and lineage metadata. It is not necessary to put every field back into model context.

6.3 Validity, Authorization, and Outcome Are Different Questions

Treating “validation” as one Boolean hides the most important boundaries:

1. **Syntactic and schema validity:** Can the arguments be parsed, and do their fields and types match the declared interface? Provider strict-schema features can improve this layer, but their guarantees are provider- and API-specific ([OpenAI — Function Calling, Strict Mode](#)).
2. **Semantic validity:** Do identifiers exist? Is the amount positive? Is the target state compatible with the operation? Is a referenced revision still current? These checks require domain state and cannot be encoded completely by JSON Schema.
3. **Authorization:** May this actor perform this action on this resource, in this tenant, for this purpose, now? A tool appearing in model context is not a capability grant.
4. **Approval:** Does policy require a person to approve this exact action and payload? The runtime, not the model, must insert a mandatory gate. The MCP specification recommends that applications let people see exposed tools and deny sensitive invocations ([MCP — Tools Specification](#)).
5. **Outcome confirmation:** Did the intended external state actually result? A successful transport response, a tool's prose, and a verified postcondition are different evidence strengths.

Consider `transfer_funds({from, to, amount})`. Valid JSON and numeric `amount` establish only the first layer. Semantic checks must resolve both accounts and business limits; authorization must bind the acting identity to `from`; policy may require approval for the exact amount and recipient; execution needs a transaction identity; outcome confirmation checks the resulting transaction or balances. Skipping any one layer changes the safety and correctness claim.

6.4 Idempotency, Deduplication, Timeouts, and Retries

A retry policy begins with the action's effect semantics, not with the exception class. RFC 9110 defines an idempotent request as one whose intended server effect is the same after one or several identical requests, and warns against automatically retrying non-idempotent requests unless the client knows the semantics are idempotent or knows the first request was not applied ([RFC 9110 §9.2.2](#)). Tool adapters need the same discipline even when they do not use HTTP.

Action class	Example	Default after timeout	Required protection
Pure/read-only	Search, fetch status	Retry within budget	Deadline, rate limit, request correlation
Naturally idempotent	Set desired label to approved	Retry if preconditions still hold	Resource version or conditional write
Made idempotent by contract	Create payment with an idempotency key	Query by key, then resume or retry	Durable dedup record and stable key
Non-idempotent consequential action	Send message, submit order without a key	Do not retry blindly	Outcome lookup, reconciliation, or human escalation

An **idempotency key** names one intended effect across repeated delivery attempts. A **deduplication record** stores what happened under that key. A call ID identifies a proposal; an attempt ID identifies one dispatch. Using a fresh idempotency key on every retry defeats deduplication, while reusing one key for two different intended effects creates a collision.

A timeout means the caller stopped waiting; it does not prove the callee stopped or rolled back. Move such an invocation to `unknown`, query an operation-status or target-state endpoint when available, and only then decide whether to retry. Cancellation is similarly a request to stop unless the tool contract guarantees rollback. For multi-effect tools, return which sub-operations succeeded and expose compensating actions instead of reporting an undifferentiated failure.

Retry logic must not bypass authorization or approval. Reauthorize when identity, resource state, tool version, arguments, policy version, approval scope, or approval expiry changes. Count attempts against one invocation budget and emit a terminal observation when that budget is exhausted so the model cannot create an accidental infinite retry loop.

6.5 Normalize Results, Then Confirm Outcomes

Provider protocols encode tool results differently. A harness should normalize them into a small internal vocabulary while retaining the raw response as an artifact when needed:

```
status: succeeded | retryable_error | terminal_error | partial | unknown
summary: compact model-facing observation
data: typed structured result, when available
artifacts: references to large or reviewable outputs
error: stable category, safe message, retry guidance
effect: external operation ID and postcondition evidence
```

MCP distinguishes protocol errors from tool-execution errors and supports structured results with an output schema; its specification requires servers to conform to a declared result schema and recommends that clients validate results before passing them to the model ([MCP — Tools Specification](#)). Preserve that distinction. “Unknown tool” is not the same failure as “payment provider rejected the charge,” and neither is the same as “charge may have succeeded but its response was lost.”

Results are observations, not instructions. A webpage, email, repository file, peer-agent response, or third-party API can contain adversarial text; Anthropic's tool-use documentation therefore tells clients to treat externally influenced tool results as untrusted content ([Anthropic — Handle Tool Calls](#)). Keep provenance and trust labels, constrain result size, and present only the material needed for the next decision. For consequential actions, confirmation should prefer environment state—transaction status, file hash, deployed version, sent-message ID—over the model's own statement that the task is complete.

6.6 Designing the Agent–Computer Interface

Anthropic uses *agent–computer interface* (ACI) by analogy with HCI: tool schemas, descriptions, responses, and error affordances deserve deliberate design. Its early guidance recommends formats that resemble training data, leave the model room to plan before committing to rigid syntax, and avoid unnecessary formatting burdens; in one SWE-bench agent, changing file tools from relative to absolute paths removed most path errors observed after leaving the repository root ([Anthropic — Building Effective Agents](#)).

Do not wrap every API endpoint mechanically. A model that must call `list_contacts`, scan a large response, and then call `send_message` has a worse interface than one offered a scoped `search_contacts` or `message_contact` operation. Consolidate steps that are almost always chained, but do not hide separately authorized or separately reviewable effects inside one giant tool. `get_customer_context` may safely combine reads; a `research_and_wire_money` tool would erase necessary policy boundaries.

Descriptions should make implicit domain knowledge explicit: when to use the tool, when not to use it, required formats, units, side effects, and meaningful parameter names. Anthropic reports that namespacing and description changes materially affect model tool-use performance and recommends evaluation-driven iteration rather than assuming the first schema is adequate ([Anthropic — Writing Effective Tools for Agents](#)). Schema design improves proposal quality; it does not replace the lifecycle checks above.

6.7 Three Tool-Catalog Strategies

Chapter 3 introduced three valid catalog patterns. They solve different context and continuity problems:

1. **Stable catalog plus action masking.** Keep tool definitions stable for model continuity and provider prompt-cache locality, while a deterministic dispatcher blocks actions that are unavailable in the current state. Manus reports using name prefixes and masking in its own system; this is a provider/workload case study, not a protocol rule ([Manus — Context Engineering for AI Agents](#)).
2. **Deferred loading or tool search.** Keep rarely used definitions out of initial context and load them when discovered. Anthropic's tool-search interface supports deferred tools and documents how selected definitions are expanded into the conversation ([Anthropic — Tool Reference](#)).
3. **Stable meta-tool or code surface.** Expose a small registry, search tool, or sandboxed code interface; resolve the larger capability set behind it. This reduces schema volume and model round trips, but the dispatcher must still validate and authorize each final capability.

Choose with workload-specific evals. Measure tool-selection accuracy, context cost, latency, provider prompt-cache behavior, stale-reference failures, and permission exposure. “Mask, don't remove” is not universal, and dynamic loading is not automatically safer merely because fewer schemas are visible.

6.8 MCP Exposes Capabilities; the Harness Governs Calls

The Model Context Protocol is a client–server protocol in which servers **expose** tools and clients list and invoke them. A server may also expose resources and prompts, but the protocol does not imply that it owns, controls, or can inspect every downstream system behind a tool ([MCP — Tools Specification](#); [MCP — Server Overview](#)).

MCP standardizes discovery and transport; it does not eliminate lifecycle responsibility. The specification requires servers to validate inputs, implement access controls, rate-limit calls, and sanitize outputs. It recommends that clients confirm sensitive operations, show inputs before dispatch, validate results, enforce timeouts, and log usage ([MCP — Tools Specification](#)). A production client may add stricter policy, mandatory approvals, tenant isolation, deduplication, and postcondition verification.

Deployment topology matters too. A private capability need not be exposed through a new inbound public endpoint. OpenAI's secure MCP tunnel documents an outbound-only client inside the private network that connects to a configured destination while retaining authentication and streaming ([OpenAI — Connect Private MCP Servers](#)). Treat this as one deployment pattern, not a property guaranteed by MCP itself.

6.9 Dynamic Tool Lists Require Fresh Decisions

MCP's `listChanged` capability lets a server notify a client that its available-tool list has changed ([MCP — Tools Specification](#)). Therefore a cached schema or an earlier appearance in model context is not current authority.

On a list-change notification, the client should:

1. fetch the new list under the current authenticated identity;
2. diff tool identity, schema, annotations, server identity, and version;
3. invalidate stale model-visible definitions and cached routing decisions;
4. re-run capability and policy checks before the next dispatch;
5. require fresh approval if the action, arguments, resource, or risk changed;
6. record which catalog and policy versions governed the call.

The same rule applies to deferred loading and meta-tools. Discovery answers “what is advertised now?” Authorization answers “what may this actor do now?” They are separate operations.

6.10 Namespaces and Context-Efficient Responses

Consistent prefixes such as `jira_issues_search` or `asana_projects_create` reduce collisions and let a harness group related capabilities. Anthropic's tool-design evaluations found that naming and prefix-versus-suffix choices can affect performance, with the best choice depending on the workload ([Anthropic — Writing Effective Tools for Agents](#)). A namespace is a usability and routing aid, not proof that two tools share an authorization domain.

Tool responses should favor relevant, typed information over maximal payloads. Offer pagination, filters, ranges, concise/detailed modes, and explicit truncation markers. Use meaningful labels for model-facing references while retaining canonical IDs for subsequent calls. Anthropic reports that replacing opaque UUIDs with more natural identifiers improved accuracy in its tool evaluations and recommends token-efficient response controls ([Anthropic — Writing Effective Tools for Agents](#)).

Errors should be actionable but safe: identify the stable category, which field or precondition failed, whether a retry is allowed, and where full diagnostics are stored. Avoid flooding context with thousands of successful log lines or exposing secrets through raw tracebacks. Large raw outputs belong in artifacts; the normalized observation should include a recoverable reference.

6.11 Code Execution as a Meta-Tool

Instead of placing hundreds of direct tool definitions and every intermediate result in model context, a harness can expose a sandboxed code surface backed by typed wrappers. Anthropic's "Code Execution with MCP" describes progressive discovery through generated TypeScript files and reports a Google Drive-to-Salesforce example in which token use fell from 150,000 to 2,000; that number is a named example, not a universal savings rate ([Anthropic — Code Execution with MCP](#)).

This pattern is useful for bounded dataflow: filtering, joining, sorting, deduplicating, aggregating, and validating before returning a compact result. OpenAI's programmatic tool-calling documentation describes the same broad benefit—tools can be called from generated code so intermediate results need not all pass through the model—but still requires the application to execute client-side tools and return their outputs ([OpenAI — Programmatic Tool Calling](#)).

Code execution does not create ambient authority. The sandbox, typed proxy, and dispatcher must constrain the callable tools, credentials, network, filesystem, budgets, and side effects. Record the generated program, the capability set and policy version it ran under, each consequential sub-call, and final artifacts. Do not hide separately approvable actions inside a program merely to reduce model round trips.

6.12 Evaluate Tools and the Lifecycle Together

Tool evals should test more than whether the model selected the right name. Anthropic recommends iterating through prototype, realistic evaluation, programmatic runs, and transcript analysis, using observed failures to refine tools ([Anthropic — Writing Effective Tools for Agents](#)). For this chapter's lifecycle, include slices for:

- malformed, incomplete, and schema-invalid arguments;
- valid schemas with nonexistent resources or stale versions;
- cross-tenant and over-scoped authorization attempts;
- required approvals that the model does not request;
- duplicate delivery, lost responses, timeout-after-effect, and cancellation races;
- partial success in parallel or multi-effect tools;
- untrusted result content and oversized outputs;
- dynamic catalog changes between proposal and dispatch;
- postcondition mismatch despite a nominally successful response.

Capture proposal, validation decisions, policy version, approval, attempts, normalized result, and outcome evidence. Chapter 11 develops evaluation terminology and graders; Chapter 17 develops production tracing. The important point here is that a clean model transcript can still conceal a duplicate effect or a failed authorization check unless the eval inspects environment state.

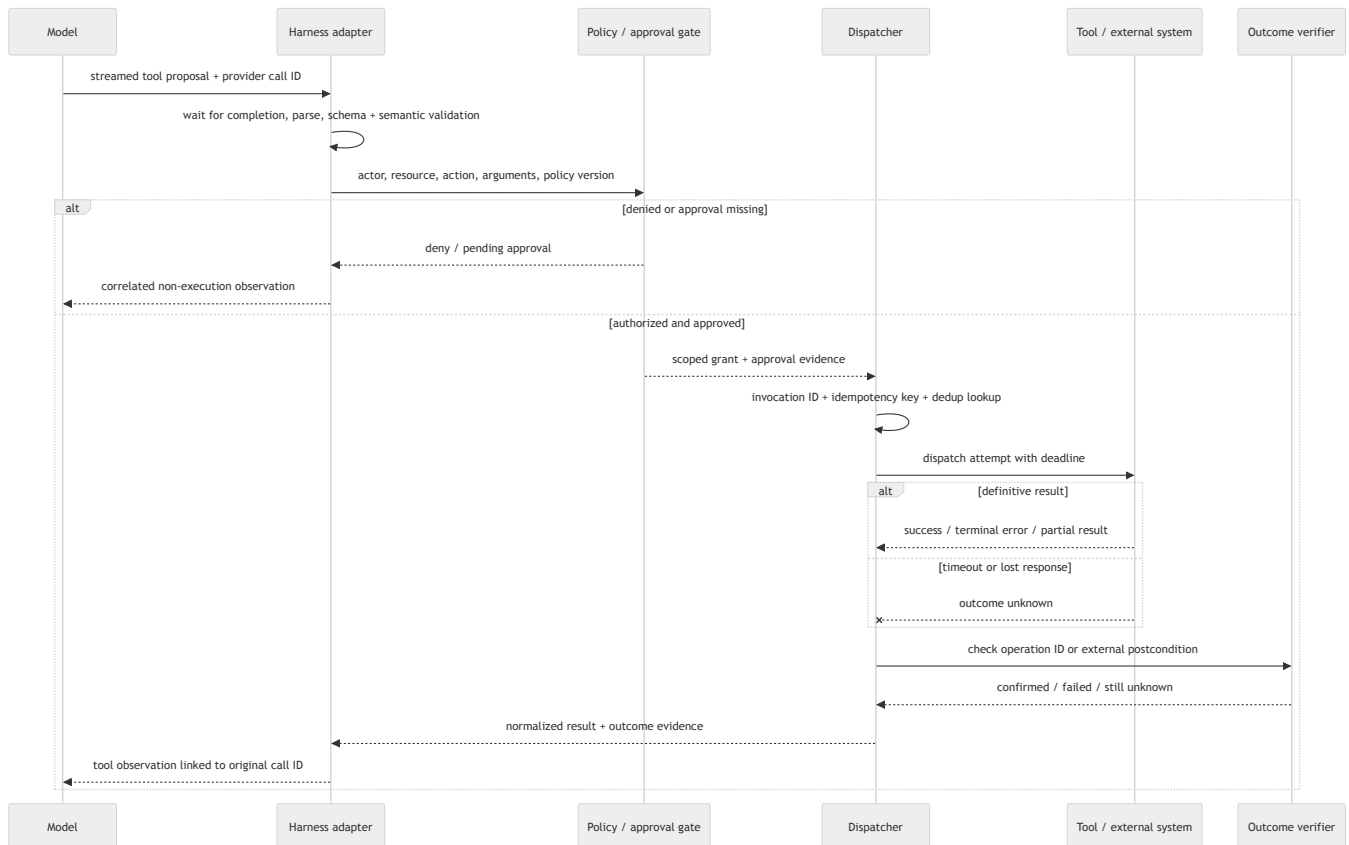
6.13 MCP and A2A Cross Different Boundaries

MCP connects a client to capabilities exposed by a server. A2A connects an agentic client to another agentic service through messages and tasks. The current A2A specification defines Agent Cards for discovery, task and message operations, streaming, authentication/authorization fields, and multiple protocol bindings; it does not make the peer's internal model, memory, tools, or sandbox inspectable to the caller ([A2A Protocol Specification](#)).

That difference changes the contract. An MCP tool call should have a precise operation schema and a result tied to one call. An A2A delegation may be long-running, exchange messages, return artifacts, and require task-level status or cancellation. In both cases, the local harness must establish identity, delegated authority, data scope, deadlines, provenance, and outcome evidence. Treat peer-agent output as untrusted external content, not as a higher-authority instruction.

Use native multi-agent delegation when tasks are bounded, have clear ownership, and produce mergeable artifacts. Shared mutable state or strict sequential dependencies usually need a single owner plus deterministic coordination. A2A standardizes an interoperability boundary; it does not remove distributed-systems or trust problems.

Diagram: One Tool Call From Proposal to Confirmed Outcome



Key Takeaways

- **A tool call is a proposal:** schema validity is not authorization, approval, execution, or proof of outcome.
- **Call identity must survive the whole lifecycle:** correlate streaming fragments, retries, attempts, results, and confirmations without relying on tool name or position.
- **Never dispatch partial streamed arguments:** wait for the provider's completion boundary, then parse and validate.
- **Timeout means unknown outcome:** reconcile state before retrying a consequential or non-idempotent action.
- **Results need normalization and provenance:** return compact observations while retaining structured data, artifacts, errors, and postcondition evidence.
- **Tool catalogs admit multiple strategies:** stable catalog plus masking, deferred loading/tool search, and a stable meta-tool all have valid workloads.
- **Dynamic discovery does not carry authority:** an MCP `listChanged` event or deferred tool requires fresh capability and policy checks.
- **MCP exposes tools; it does not guarantee ownership of their resources:** clients and servers still need explicit validation, access control, approval, timeouts, and logging.
- **A2A is a peer-agent boundary, not a tool-schema substitute:** preserve delegated authority, task status, provenance, and distrust of returned content.
- **Evaluate external state, not only the transcript:** lifecycle bugs often look successful in model-visible text.

Further Reading

- Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- Anthropic, *Handle Tool Calls*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls>
- OpenAI, *Function Calling*. <https://developers.openai.com/api/docs/guides/function-calling>
- Model Context Protocol, *Tools Specification*, Jun 2025. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- A2A Project, *A2A Protocol Specification*. <https://a2a-protocol.org/latest/specification/>
- RFC Editor, *RFC 9110: HTTP Semantics*, Jun 2022. <https://www.rfc-editor.org/rfc/rfc9110.html>
- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/code-execution-with-mcp>
- OpenAI, *Programmatic Tool Calling*. <https://developers.openai.com/api/docs/guides/tools-programmatic-tool-calling>

Chapter 7: Sandboxing, Guardrails, and Runtime Enforcement

7.1 The Agent Security Threat Model

Most of this chapter covers *mitigations*: sandboxes, hooks, and approval gates. Before discussing them, we need to be explicit about the risks they address. An agent that can read untrusted content and act on external systems has a distinctive threat profile ([Anthropic — Beyond Permission Prompts](#)):

- **Prompt injection** (see Foundations ch 8, "Prompt Injection as Context Confusion") — the model cannot reliably distinguish data from instructions. Untrusted content that the agent reads—a web page, issue comment, source file, or tool result—can therefore steer it as if that content were a command.
- **Data exfiltration** — a compromised or steered agent with network access can send secrets, such as SSH keys, API tokens, or proprietary source code, to an attacker-controlled destination.
- **Destructive action** — a steered agent with filesystem or shell access can delete or corrupt files, or commit and push harmful code.
- **Tool and supply-chain risk** — a malicious or compromised MCP server, package, or dependency can introduce hostile tools or instructions that the agent may then trust.

The combination practitioners worry about most is sometimes called the *lethal trifecta*: access to private data, exposure to untrusted content, and the ability to communicate externally ([Simon Willison — The lethal trifecta for AI agents](#)). Each capability can be manageable on its own. When one agent has all three, however, a single injected instruction can read a secret and send it to an attacker. Most controls in this chapter work by breaking one leg of the trifecta. Network isolation removes external communication, filesystem isolation removes access to private data, and approval gates place a human in the path of consequential actions.

The key premise for the rest of the chapter is that the model is not a trusted component. It is capable, but it can be steered. The harness is what stands between a hostile instruction and a real-world consequence.

Anthropic's later containment work sharpens this threat model along two dimensions ([Anthropic — How We Contain Claude](#)). Risk can originate from a **misusing user**, from **model misbehavior**, or from an **external attacker** steering the model through content. Defenses, in turn, can be placed in the **model**, the **execution environment**, or at the **external-content boundary**. This matrix matters because no single defensive layer covers every source of risk. Alignment cannot guarantee that a model will ignore an injection, while a sandbox cannot determine whether a permitted email is semantically harmful. Production containment therefore requires defense in depth across all three layers.

7.2 The Permission Fatigue Problem

Coding agents that operate without oversight are dangerous, but agents that request permission for every action are impractical. Anthropic describes the resulting problem as approval fatigue: "Constantly clicking 'approve' slows down development cycles and can lead to 'approval fatigue,' where users might not pay close attention to what they're approving, and in turn making development less safe" ([Anthropic — Beyond Permission Prompts: Making Claude Code More Secure and Autonomous](#)). The solution is structural: establish boundaries within which the agent can act freely, and request permission only when an action crosses one of them.

In Anthropic's internal use, sandboxing safely reduced permission prompts by 84% ([Anthropic — Beyond Permission Prompts](#)).

7.3 Sandbox as Cage, Reset Button, and License

The OpenReview survey presents the sandbox as more than a security mechanism. In an agent system, it serves three purposes at once ([OpenReview — Agent Harness Engineering: A Survey](#)):

- **Security**: it bounds the blast radius of unpredictable model-generated actions and prompt-injection-driven behavior.
- **Reproducibility**: it gives evals, training trajectories, and long-running sessions a resettable baseline. A container or microVM can be destroyed and rebuilt; a developer workstation cannot.
- **Liveness**: it defines a region where the agent is allowed to act without asking a human on every file write, package install, or network call.

The third purpose is distinctive to agent systems. A sandbox is not only a cage; it is also a license to act. By moving permission decisions from individual actions into session-level configuration, it makes long-horizon autonomy

practical without producing constant approval prompts.

Containment should scale with the work. Three recurring patterns form a useful ladder ([Anthropic — How We Contain Claude](#)):

- An **ephemeral container** is disposable, cheap, and appropriate for bounded tasks where the agent needs broad freedom inside a small blast radius.
- A **human-in-the-loop sandbox** supports interactive work: safe operations proceed automatically, while boundary crossings suspend for approval.
- A **sealed virtual machine** isolates higher-risk workloads behind a stronger kernel and network boundary, at greater startup and operational cost.

The useful question is not simply "Is it sandboxed?" Instead, ask which resources remain reachable, which state survives a reset, and what authority can cross the boundary. Resettable compute does not neutralize a credential mounted inside the sandbox, a poisoned memory written outside it, or an egress path capable of transmitting private data.

7.4 Filesystem and Network Isolation Must Be Paired

Claude Code's sandbox enforces filesystem and network boundaries together, and Anthropic argues that both are necessary. Filesystem isolation prevents a prompt-injected agent from modifying sensitive files. Network isolation prevents it from leaking data or downloading malware. Without the network boundary, a compromised agent could exfiltrate SSH keys. Without the filesystem boundary, it could escape the sandbox and gain network access ([Anthropic — Beyond Permission Prompts](#)).

The implementation builds on OS-level primitives—Linux bubblewrap and macOS seatbelt—and applies to subprocesses as well as direct Claude Code interactions. Network traffic passes through a Unix domain socket to a proxy, which enforces domain restrictions and asks the user to confirm newly requested domains. The runtime is open source ([Anthropic — Beyond Permission Prompts](#)).

Claude Code on the web extends the design to a cloud sandbox in which sensitive credentials, including git credentials and signing keys, never reside inside the sandbox with the agent. A custom proxy handles git interactions and attaches scoped credentials only after confirming that an operation is permitted—for example, that a push targets the configured branch ([Anthropic — How We Contain Claude](#)).

An egress allowlist is a **capability grant**, not merely a list of harmless destinations. Access to a package registry permits the agent to download executable code. Access to a source host may permit it to publish content. Access to a general web endpoint can complete the exfiltration leg of the lethal trifecta. Policies should therefore bind network access to a destination, protocol, operation, identity, and task—not to a domain alone—and log the rule that authorized each connection.

Containment must also begin **before trust is established**. Opening a repository can trigger configuration loading, dependency discovery, language servers, hooks, or local listeners before the user sees a trust dialog. Treat project-open and configuration-load paths as hostile. Parse without executing where possible, disable automatic hooks and listeners, and withhold credentials and egress until the workspace has been explicitly trusted ([Anthropic — How We Contain Claude](#)).

7.5 Identity, Policy Decisions, Enforcement, and Evidence

Sandbox boundaries are necessary, but they are not sufficient. Runtime enforcement asks who the agent represents, which resource and action are in scope, which policy version applies, and what evidence remains after execution. NIST separates the component that computes an access decision—the policy decision point (PDP)—from the policy enforcement point (PEP) that applies the decision to a request for a protected resource ([NIST — Zero Trust Architecture Glossary](#)). The same split is useful for agent actions even when the implementation does not use those exact product names.

Three design moves matter in production:

- **Identity and delegated authorization:** the agent should act through scoped, short-lived authority, not a user's full ambient credentials. A credential broker or proxy should attach a secret only after the operation has been authorized and only for its intended audience.
- **Policy decision:** a PDP can evaluate identity, tenant, purpose, tool, normalized arguments, resource, session state, environment, and risk. Static allow/deny lists are inspectable but blunt; contextual policy is useful only if its inputs and version are recorded.

- **Policy enforcement:** a PEP on the dispatch path applies allow, deny, redact, constrain, stronger-sandbox, or require-approval outcomes. A rule in a prompt is not a PEP, and an enforcement hook that the agent can bypass cannot support a hard guarantee.
- **Evidence:** record the proposed action, identity, decision and policy version, approval when required, execution result, and verified outcome. A trace can carry some of this evidence, but only records with the required completeness, integrity, retention, identity, and access controls should be described as audit records ([NIST — SP 800-53 Rev. 5, Audit and Accountability controls](#)).

This is also where supply-chain attacks become harness concerns. MCP tool poisoning, tool squatting, rug-pull updates, hallucinated packages, and retrieval-source poisoning all cross the boundary between tool interfaces and governance. A safe harness needs provenance and integrity checks for tools, packages, datasets, and retrieval sources; a prompt that merely says "be careful" is not enough.

7.6 Advisory Hooks and Blocking Enforcement

A sandbox is one kind of programmatic guardrail. Hooks and middleware can add finer-grained behavior. Claude Code supports user-defined commands or scripts that run automatically at lifecycle events, such as agent start, after a tool call, or on stop ([HumanLayer — Skill Issue: Harness Engineering for Coding Agents](#)). LangChain's middleware follows a similar structure. Some hooks are deterministic scripts; others only inject advice back into the model.

Automatic execution alone does not make every hook an enforcement point. An **advisory hook** can notify, add context, or ask the model to reconsider; the model may still ignore it. A **blocking hook** can support a hard rule only if every relevant action passes through it, denial prevents dispatch, failures are handled safely, and the agent cannot reach the protected resource through another path. The distinction is about placement and bypass resistance, not whether the hook is written in code.

Common uses include notifications, automated approvals or denials, integrations, and verification. A hook might play a sound when an agent finishes, deny a database migration and ask the user to run it, post a Slack message, open a pull request, or run type checking and a build before the agent stops. HumanLayer's example runs Biome and TypeScript in parallel whenever Claude attempts to stop. It exits silently on success; on failure, it returns only the errors with exit code 2, causing the harness to re-engage the agent.

LangChain reports that this kind of middleware helped move deepagents-cli from the Top 30 to the Top 5 on Terminal-Bench 2.0. Its `PreCompletionChecklistMiddleware` intercepts the agent before exit and prompts it to verify its work against the task specification. `LocalContextMiddleware` runs at startup to map the working directory and discover available tools. `LoopDetectionMiddleware` counts edits to each file and, after N edits to the same file, asks the agent to reconsider its approach. This can break "doom loops" in which the agent repeatedly tries small variations of a failing strategy ([LangChain — Improving Deep Agents with Harness Engineering](#)).

The survey's governance taxonomy places these hooks within a broader pipeline. Pre-invocation checks can deny unsafe tool calls. Post-invocation hooks can mark or redact untrusted outputs before they enter context. Stop hooks can require verification, and escalation hooks can route ambiguous cases to humans. The more consequential an action is, the less its safety should depend on the model remembering an instruction ([OpenReview — Agent Harness Engineering: A Survey](#)).

7.7 Mandatory Approval Is a Runtime Gate

An agent may call a tool such as `request_human_approval`, which is a useful way to suspend and resume a workflow. It is not sufficient enforcement for high-risk actions because a compromised or mistaken model may simply omit the call. For protected operations, the dispatcher or PEP must classify the normalized action and require an approval record before execution. The MCP tools specification likewise assigns clients a responsibility to present confirmation for tool operations and let a human deny them; that confirmation happens around invocation, not because the model voluntarily requested it ([MCP — Tools Specification](#)).

The approval should bind to the exact action a reviewer saw: agent and delegating identity, tenant, resource, normalized arguments, policy version, risk explanation, expiry, and any execution constraints. If a material argument, destination, credential scope, or policy version changes after approval, the gate must evaluate the new action again. This closes a time-of-check/time-of-use gap in which a harmless proposal is approved but a different action is dispatched.

Retries and fallbacks pass through the same gate. A previous approval does not authorize an altered request, and a previous denial must not be converted into a model instruction that can be argued away. If policy changes while a run

is waiting, resume from a fresh decision rather than from stale authority. Chapter 14 develops the human review surface; Chapter 19 places policy administration and distributed PEPs in the fleet architecture.

7.8 Feedforward and Feedback: A Cybernetic View

Thoughtworks' Birgitta Böckeler offers a higher-level taxonomy for these controls ([Thoughtworks — Harness Engineering](#)). Outer-harness controls operate in two directions:

- **Guides (feedforward)** anticipate the agent's behavior and steer it before it acts. Examples include instructions in AGENTS.md, skills, reference documentation, and language-server hints. Their goal is to increase the probability of a good result on the first attempt.
- **Sensors (feedback)** observe the agent's work after it acts and help it self-correct. Tests, linters, type checkers, and AI code review all serve this role.

A harness with only feedforward guides keeps issuing rules but never determines whether they work. A harness with only feedback sensors repeatedly catches mistakes without helping prevent them. Both are necessary.

Each direction can be divided along a second axis:

- **Computational** controls — linters, type checkers, structural tests — are deterministic, run in milliseconds to seconds, and produce reliable results.
- **Inferential** controls — semantic analysis, AI code review, LLM-as-judge — handle nuance but are slower, more expensive, and non-deterministic.

The two axes are independent. Coding conventions in AGENTS.md are inferential feedforward; ArchUnit tests that check module boundaries on commit are computational feedback. A `/code-review` skill provides inferential feedback, while a pre-bootstrap script that creates the project structure is computational feedforward. A well-engineered harness combines all four types.

7.9 Three Regulation Categories

Böckeler also distinguishes harnesses by the qualities they regulate ([Thoughtworks — Harness Engineering](#)):

- **Maintainability harness** — regulates internal code quality, duplication, complexity, coverage, and style. This is the easiest category because it can draw on a long history of existing tools.
- **Architecture fitness harness** — regulates performance, observability, and debuggability. It captures the application's cross-cutting "fitness functions."
- **Behavior harness** — asks whether the application behaves as intended. This remains the least-solved category. Most teams currently use functional specifications as feedforward and AI-generated test suites as feedback, sometimes supplemented by mutation testing. Böckeler is candid that trusting AI-generated tests "is not good enough yet."

These categories make harness coverage easier to assess. A harness that is strong on maintainability but weak on behavior can create a false sense of safety.

7.10 Timing: Keep Quality Left

Continuous integration teaches that the earlier a problem is found, the cheaper it is to fix. The same principle applies to harness design. Fast computational sensors, such as linters and quick tests, should run before commit. More expensive computational and inferential sensors, such as mutation testing and broader code review, can run after integration in the pipeline. Continuous-drift sensors—including dead-code detection, dependency scanners, and log-anomaly judges—operate outside the change lifecycle altogether ([Thoughtworks — Harness Engineering](#)).

As Böckeler notes, the OpenAI Codex team's harness follows the same pattern: custom linters and structural tests enforce a layered architecture, while recurring "garbage collection" passes scan for drift and ask agents to suggest fixes.

7.11 Harnessability, Agentic Readiness, and Ambient Affordances

Not every codebase is equally easy to harness. A strongly typed language provides type-checking sensors by default. Clear module boundaries make architectural constraints enforceable. Opinionated frameworks such as Spring hide details that the agent would otherwise need to manage ([Thoughtworks — Harness Engineering](#)).

The term *ambient affordances*, which Böckeler credits to Ned Letcher, captures these properties of the environment that make it legible, navigable, and manageable for agents ([Thoughtworks — Harness Engineering](#)). Greenfield teams

can design these affordances from the start. Legacy teams face the opposite situation: a harness is often most necessary where it is hardest to build.

Looking ahead, Böckeler proposes *harness templates*: bundles of guides and sensors for specific service topologies, such as a JVM CRUD service, a Go event processor, or a Node dashboard. These could accompany existing service templates. She invokes Ashby's Law of Requisite Variety to formalize the argument: a regulator must have at least as much variety as the system it governs. Choosing a constrained topology reduces that variety and makes a comprehensive harness more achievable ([Thoughtworks — Harness Engineering](#)).

The broader production concept is **agentic readiness**: can an autonomous caller safely understand, invoke, observe, retry, and, when necessary, undo an operation? A service may be easy for a human developer to use yet hostile to an agent if its APIs conceal side effects, return ambiguous errors, or lack stable operation identifiers. Readiness improves when:

- mutations accept idempotency keys and expose operation status;
- APIs distinguish read, propose, commit, and compensate rather than hiding them behind one opaque call;
- machine identity and delegated authorization are first-class;
- errors state what failed, whether a retry is safe, and what evidence would prove recovery;
- state changes are observable, attributable, and reversible through compensating actions where literal undo is impossible.

These features act as ambient affordances for autonomous software. They reduce the amount of probabilistic reasoning required from the model and give the Chapter 19 control plane stable surfaces for policy, lifecycle management, and audit evidence.

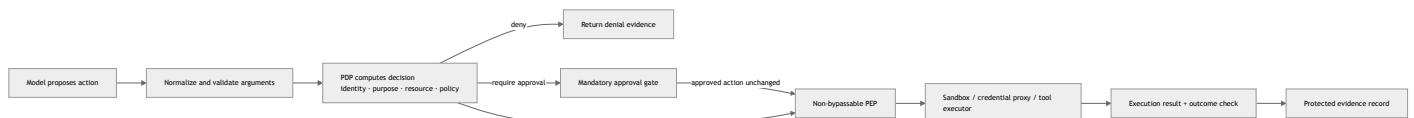
7.12 Operational Safety: Circuit Breakers, Kill Switches, Budgets, and Canaries

Sandboxes, policy, and enforcement points (§7.3–7.7) constrain what an agent *may* do. A second family of controls limits the consequences when the agent or its tools *misbehave at runtime*. These controls come almost unchanged from distributed-systems reliability and security operations. They belong in the surrounding system because an agent that has been steered or caught in a loop cannot be trusted to apply them to itself.

- **Circuit breakers.** Wrap a flaky or expensive dependency—a tool, downstream service, or sub-agent—so that the breaker trips after a threshold of failures. Subsequent calls fail fast instead of hanging or producing a retry storm ([Fowler — CircuitBreaker](#)). For agents, this limits the blast radius of an erroring tool or an agent that repeatedly retries the same failing action. It complements Chapter 5's context policy: keep one actionable failure visible while preventing it from recurring without limit.
- **Kill switches.** Provide a human- or policy-triggered mechanism that can halt one agent or an entire fleet immediately, independently of the agent's own control flow. A prompt-injected agent may be actively working against its instructions, so the switch must live in the harness—as a supervisor process, revocable credential, or sandbox teardown—not in a prompt that says "stop if asked."
- **Action budgets, iteration caps, and cost governors.** Set hard limits on tool calls, tokens, wall-clock time, or spending. When a limit is reached, the loop stops and escalates rather than continuing unchecked. This is the operational form of the loop stop rules in Chapter 13 and the per-task budgets in Chapter 18: an unbounded loop creates both an unbounded bill and an unbounded blast radius.
- **Canary tokens.** Place fake secrets—an unused API key, decoy file, or tripwire URL—where a prompt-injected agent might try to read or exfiltrate them. A callback from a canary is a high-signal warning that the agent has been steered toward data it should not touch ([Thinkst — Canarytokens](#)). Unlike a sandbox, a canary does not *prevent* the exfiltration leg of the lethal trifecta (§7.1); it *detects* it. That makes it a useful final line of defense when prevention is imperfect.

The principle is consistent throughout the chapter: the more consequential a failure is, the less its prevention should depend on the model choosing to avoid it. Prevention through sandboxes and policy works together with detection through canaries and the drift alerts discussed in Chapter 18. Neither is sufficient alone.

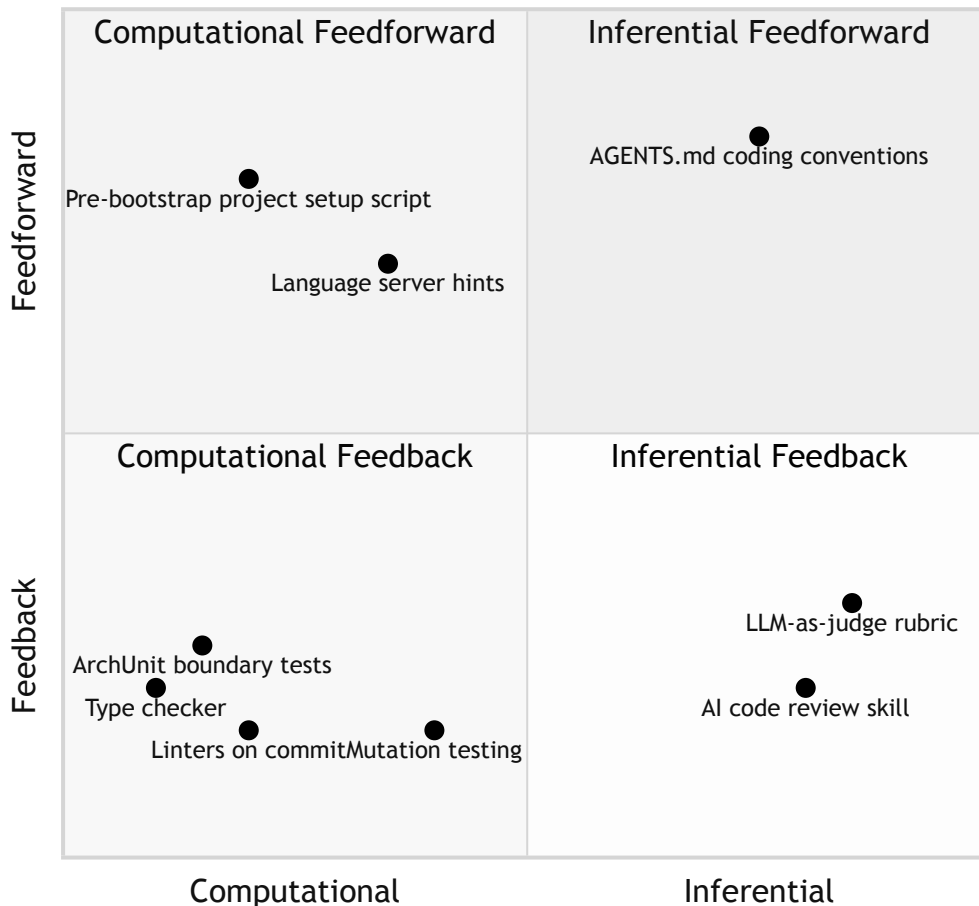
Diagram: Proposal to Enforced Effect



The model can propose and explain an action. Only the external decision and enforcement path can authorize credentials, cross a sandbox boundary, or create a protected side effect.

Diagram: Feedforward/Feedback × Computational/Inferential Quadrant

Outer-Harness Control Types



Key Takeaways

- **The threat model comes first:** prompt injection, data exfiltration, destructive action, and supply-chain risk all become especially dangerous when private data, untrusted content, and external communication form the *lethal trifecta*.
- **Anthropic reported an 84% reduction in permission prompts in its internal sandbox rollout:** this is a dated case study, not a universal reduction factor.
- **Sandboxing has three jobs:** security, reproducibility, and liveness.
- **Filesystem and network isolation must be paired:** each addresses a different attack vector, and either alone is insufficient.
- **Containment is a matrix, not a single sandbox:** defend against misuse, model misbehavior, and external attack at the model, environment, and content boundaries; choose ephemeral containers, interactive sandboxes, or sealed VMs according to risk.
- **Egress is authority:** an allowed destination grants a real capability, so bind network access to the operation, identity, and task—and grant no ambient trust merely because a project has been opened.

- **Decisions and enforcement are different:** a PDP computes policy; a non-bypassable PEP applies it on the action path.
- **Approval is mandatory when policy requires it:** a model-requested approval tool is useful orchestration, but it cannot be the only high-risk gate.
- **Hooks differ in strength:** advisory hooks can steer or notify; only blocking, non-bypassable hooks can enforce a hard rule.
- **Trace is not automatically audit:** preserve identity, policy decision, approval, execution, and outcome under explicit integrity and retention requirements.
- **Feedforward and feedback are both required:** guides without sensors have no learning loop; sensors without guides react but don't prevent.
- **Three categories of harness coverage:** maintainability (well-tooled), architecture fitness (achievable), and behavior (the unsolved problem).
- **Ambient affordances matter:** strongly-typed languages and opinionated frameworks make harnessing easier from day one.
- **Agentic readiness is an API property:** idempotency, explicit operation status, machine identity, retry semantics, observable state, and compensating actions make systems safer for autonomous callers.
- **Runtime safety also requires operational controls:** circuit breakers, kill switches, action and cost budgets, and canary tokens limit misbehavior when it occurs. They belong in the harness because a steered agent cannot be trusted to stop itself.

Further Reading

- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>
- Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
- Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
- Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
- *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
- Martin Fowler, *CircuitBreaker*, martinfowler.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
- Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>
- Anthropic Safeguards Research Team, *How We Contain Claude*, Anthropic, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>
- NIST, *Zero Trust Architecture Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>
- NIST, *SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- Model Context Protocol, *Tools Specification*, Jun 18, 2025. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>

Chapter 8: Model Selection, Routing, and Reasoning Budgets

Agent = Model + Harness. Model choice is part of the harness configuration, but “send it to another model” is not one operation. A router, a quality cascade, a reliability fallback, a hedged request, and a human escalation solve different problems and have different correctness conditions.

This chapter treats a route as safe only when the target configuration can honor the call's input, output, tool, state, policy, and data-handling contracts. The companion *LLM Foundations* volume explains model behavior; this chapter owns operational selection and switching.

8.1 Select a Model-Plus-Harness Configuration

The unit of selection is normally a model call or workflow step, not an entire application. A loop can use a capable model for planning and a smaller model for bounded classification or extraction, provided evaluation shows that the smaller configuration preserves the required outcome. Anthropic likewise recommends using the simplest workflow that succeeds and adding model-driven complexity only when it produces measurable value ([Anthropic — Building Effective Agents](#)).

The selected object is more than a model ID:

```
RouteTarget {
  provider, model_snapshot, endpoint,
  modalities, context_and_output_limits,
  tool_protocol, tool_schemas,
  structured_output_contract,
  reasoning_configuration,
  policy_profile, data_region,
  timeout, price_schedule
}
```

OpenAI's model catalog, for example, publishes capabilities such as context window, maximum output, modalities, tools, function calling, and structured-output support per model rather than as properties of a universal API model ([OpenAI — Models](#)). A deployment must snapshot the capabilities it actually validated; a mutable provider alias can change without the application changing its own code.

Per-call selection has one important exception: a stateful assistant turn or tool-use continuation may bind several API calls to one provider-specific protocol. Switching midway is unsafe unless the harness has an explicit translation or restart strategy. Section 8.5 returns to this reasoning-state boundary.

8.2 Five Different Routing Mechanisms

Do not use *routing* as a catch-all for every model switch.

Mechanism	Decision point	Purpose	Required signal
Router	Before an attempt	Match a request or step to a suitable target	Features available before execution and an evaluated routing policy
Quality cascade	After a cheaper attempt	Escalate when the first result does not meet a quality bar	A verifier that can accept or reject that result
Reliability fallback	After an infrastructure or availability failure	Continue service on a compatible target	Failure classification plus a prevalidated fallback target
Hedging	Concurrently or after a short delay	Reduce tail latency by racing equivalent attempts	An operation safe to duplicate, a winner rule, and loser cancellation
Human escalation	Before or after an automated attempt	Transfer ambiguity, authority, or high-impact judgment to a person	A stop condition, review package, and durable wait/resume path

RouteLLM is a research example of a learned pre-execution router: it was trained from preference data to choose between stronger and weaker models and evaluated the resulting cost-quality tradeoff on its benchmark suite ([Ong et al. — RouteLLM](#)). Its measured savings are properties of that router, model pair, prices, and evaluation set—not a guarantee that difficulty routing will pay off for another workload.

FrugalGPT is a research example of quality cascades: it evaluates an answer from one service and conditionally escalates to another, optimizing under a budget on the studied datasets ([Chen et al. — FrugalGPT](#)). A cascade without a trustworthy acceptance signal is merely a slower route; schema validity alone proves neither semantic correctness nor task success.

Hedging is not fallback. The *Tail at Scale* describes issuing redundant requests to reduce tail latency in large online services, accepting extra work as the tradeoff ([Dean and Barroso — The Tail at Scale](#)). In an agent system, hedge model generation only when duplicate attempts are policy-compatible and cannot independently cause external side effects. The harness chooses one result and prevents losing proposals from reaching tool dispatch.

Human escalation is not a “stronger model tier.” Use it when the missing ingredient is authorization, responsibility, domain judgment, or clarification rather than more inference compute. The workflow must preserve state while it waits and show the reviewer the relevant evidence, proposed action, uncertainty, and alternatives.

8.3 Require Compatibility Before Fallback

A fallback target is eligible only if it passes a compatibility gate for the current call. Test the exact model snapshot, endpoint, and harness adapter; provider families and “OpenAI-compatible” transports do not establish semantic equivalence.

Dimension	Compatibility question	Failure if ignored
Modalities	Can the target accept every input type and produce the required output type at the required fidelity?	Images, audio, files, or tool results are dropped or transformed incorrectly.
Context and output limits	Do serialized input, reserved reasoning/output budget, tool schemas, and expected result fit?	Truncation, rejected requests, or missing final output.
Tool protocol and schema	Does the target support the same call/result protocol, parallelism, tool-choice modes, schema subset, and call-ID continuation?	Invalid calls, lost correlations, or different dispatch behavior.
Structured output	Does the target enforce the required schema and refusal/error representation?	Downstream code accepts prose or a different failure shape.
Reasoning and continuation state	Can provider-specific reasoning items, signatures, response IDs, and unfinished tool turns be continued or intentionally restarted?	Rejected requests or a continuation that silently loses prior work.
Safety and policy	Is the target approved for this tenant, risk class, content, tool set, and action scope, and does the harness normalize refusal handling?	A fallback bypasses a model allowlist or changes policy behavior.
Data region and retention	Are endpoint, model, tools, storage, processing region, retention mode, and subprocessors permitted?	The route violates residency or data-handling requirements.

Capability differences are concrete API facts, not theoretical edge cases: model catalogs expose different modality, tool, structured-output, and context support ([OpenAI — Models](#)). Data-region support can also vary by endpoint, model, tool, storage, and processing mode; OpenAI's data-control documentation, for example, distinguishes regional storage from regional processing and lists service-level eligibility ([OpenAI — Data Controls](#)). Treat current provider documentation as an input to the compatibility registry, not as a substitute for local contract tests.

Latency, price, capacity, and rate limits are selection constraints rather than semantic compatibility. A target may be compatible but too slow or expensive for the request's service tier. Conversely, a fast target is not eligible if it cannot honor the output or policy contract.

Precompute the compatibility graph for known targets, then re-check request-specific values at dispatch: actual token count, attached modalities, selected tools, tenant policy, data region, and current provider status. Fail closed to another eligible route or human escalation; do not silently strip an unsupported input to make a target fit.

8.4 Fallback and Retry Must Respect Side Effects

A model call proposes output; Chapter 6's dispatcher decides whether an external action executes. This boundary determines whether retry is cheap or dangerous.

Classify an attempt before retrying:

1. **No action proposed:** retry or fallback may repeat inference cost, but not an external effect.

2. **Action proposed but not dispatched:** discard the old proposal, route again, and authorize the new proposal normally.
3. **Dispatch outcome known:** record the outcome; retry only under the tool's documented semantics.
4. **Dispatch outcome unknown:** query the target system by action or idempotency key before deciding whether to repeat, compensate, or escalate.

HTTP Semantics states that a client should not automatically retry a non-idempotent request unless it knows the request semantics are idempotent or can determine that the original request was not applied ([RFC 9110 §9.2.2](#)). Agent retries need the same discipline even when the tool transport is not HTTP.

For a side-effecting action, use a stable action ID or idempotency key that survives model fallback, gateway retry, worker restart, and timeout. Persist the proposed operation before dispatch and store the external resource ID or outcome beside it. If the target system offers no idempotency or status lookup, an ambiguous timeout is a human-review or compensation case—not permission to call the action again.

Retries never inherit authorization merely because the first attempt was allowed. Re-evaluate policy when identity, resource state, action arguments, policy version, route target, or approval validity changes. A fallback also must not reinterpret a refusal, deny, or mandatory approval as an infrastructure error.

8.5 Reasoning Budgets Are Provider-Specific Controls

Reasoning effort is a useful routing axis, but there is no portable cross-provider unit called “one reasoning token” or “high effort.” Each provider defines its own parameters, token accounting, supported models, continuation objects, and interactions with tools and caching.

OpenAI currently documents model-specific `reasoning.effort` levels and provider-managed persisted reasoning through `reasoning.context` and response continuation; its guidance says to compare settings on representative workloads rather than assume the maximum level is best ([OpenAI — Model Guidance](#)). Anthropic documents manual and adaptive thinking modes, `budget_tokens` or effort controls depending on model, and rules for preserving thinking blocks during tool-use continuations ([Anthropic — Extended Thinking](#)). These are different API contracts, not two spellings of one universal control.

The harness should expose a provider-neutral objective such as:

```
ReasoningPolicy {
  quality_tier,
  max_end_to_end_latency,
  max_call_cost,
  allow_persisted_reasoning,
  data_handling_profile
}
```

The provider adapter maps that objective to supported parameters and records the resolved settings and usage. Do not translate `high` to `high` by name alone. Calibrate each provider/model configuration on the same task slice.

Reasoning state is also part of compatibility. Anthropic requires complete, unmodified thinking blocks in certain tool-use continuations, while OpenAI documents continuation through provider-specific response items and IDs ([Anthropic — Extended Thinking](#); [OpenAI — Model Guidance](#)). A harness cannot serialize one provider's opaque state into another provider's protocol and call that a continuation. It must either stay on the original target, restart from an explicit context summary and durable tool outcomes, or use a translation that has its own evals.

Visible or summarized reasoning is not a verifier. Providers may omit or summarize internal reasoning, and the returned representation belongs to their API contract ([Anthropic — Extended Thinking](#)). Judge the answer, evidence, tool outcomes, and environment state instead.

8.6 Apply Policy Before Cost Optimization

Routing is a constrained optimization problem. First determine the eligible set; only then optimize among it.

Eligibility can depend on:

- tenant and user entitlements;
- approved provider, model snapshot, and endpoint;
- data classification, storage and processing region, and retention mode;

- required safety or domain policy profile;
- allowed modalities, hosted tools, and third-party subprocessors;
- action risk, approval status, and current incident controls.

A cheap route outside the eligible set is not a saving. Provider data-residency documentation demonstrates why eligibility must include the complete request path: regional storage does not necessarily imply regional processing, and third-party tools can have separate policies ([OpenAI — Data Controls](#)).

Record the policy version and eligibility result with every routing decision. If a provider error leaves no eligible automated target, use an explicit fail-closed result or human escalation rather than falling through to an unapproved default.

8.7 Evaluate the Routing System, Not Just Each Model

A routing policy can make every candidate look good in isolation and still fail by choosing the wrong candidate. Evaluate the complete model-plus-harness configurations, router decisions, escalation signals, and failure handling. Anthropic's evaluation guidance separates tasks, trials, graders, transcripts, outcomes, the agent harness, and the evaluation harness; that separation is useful for attributing routing failures to the decision rather than only to the selected model ([Anthropic — Demystifying Evals for AI Agents](#)).

Use representative slices, not one blended average:

Slice	What to measure
Quality	Task success, required evidence, verifier false accept/reject, easy/hard and domain cohorts
Cost	Input, output, reasoning, cache, tool, hedge, and escalation cost per successful outcome
Latency	Queue, router, provider, tool, verifier, and end-to-end p50/p95/p99; timeouts by stage
Failure class	Rate limit, timeout, transport error, context overflow, schema failure, refusal, tool mismatch, ambiguous side effect
Policy	Ineligible-route rate, region/retention/model-allowlist violations, approval and tenant-isolation tests
Routing behavior	Route distribution, misroutes, cascade escalation rate, fallback success, hedge win/cancel rate, human escalation rate

For router evals, label both costly false escalation and more serious false economy: cases sent to a cheaper target that fail the task or policy. For cascades, grade the first answer independently so a weak verifier cannot make the cascade appear efficient. For fallback tests, inject each failure class and assert which transitions are allowed. For hedging, count duplicate tokens and verify that losing outputs cannot dispatch actions.

Pin model snapshots, adapters, prompts, tools, policy versions, prices, and regional endpoints in the trial manifest. Report current price as dated configuration data, not as a timeless property of a model family. A route is ready only when quality, cost, latency, failure, and policy slices all meet their gates.

8.8 An AI Gateway Is a Placement Option, Not an Architectural Role

An AI gateway can centralize provider adapters, credentials, quotas, routing, retries, telemetry, and sometimes policy checks. It can be a useful implementation location, but the product label does not make it a control plane or a policy enforcement point.

Kubernetes uses *control plane* for components that manage the overall state of a cluster, distinct from node components that run workloads ([Kubernetes — Components](#)). In this book, a fleet control plane similarly manages desired configuration, identity, policy administration and decisions, versions, and lifecycle across many agents. A gateway on the synchronous model-call path is ordinarily a data-path component even if a separate control service configures it.

NIST defines a policy enforcement point (PEP) by function: it enforces access-policy decisions when a subject requests a protected resource ([NIST — Zero Trust Glossary](#)). A gateway is a PEP only for the decisions it enforces on an unavoidable path. Logging a policy result, adding a guardrail callback, or hosting routing logic does not by itself make the gateway the PEP for tool execution, file access, or downstream side effects.

Centralization also creates correlated risk. Put explicit timeouts and circuit breakers around the gateway, preserve end-to-end call and action IDs, and ensure gateway retries use the same idempotency and outcome rules as application retries. Keep route policy in versioned configuration and test it independently of the gateway product.

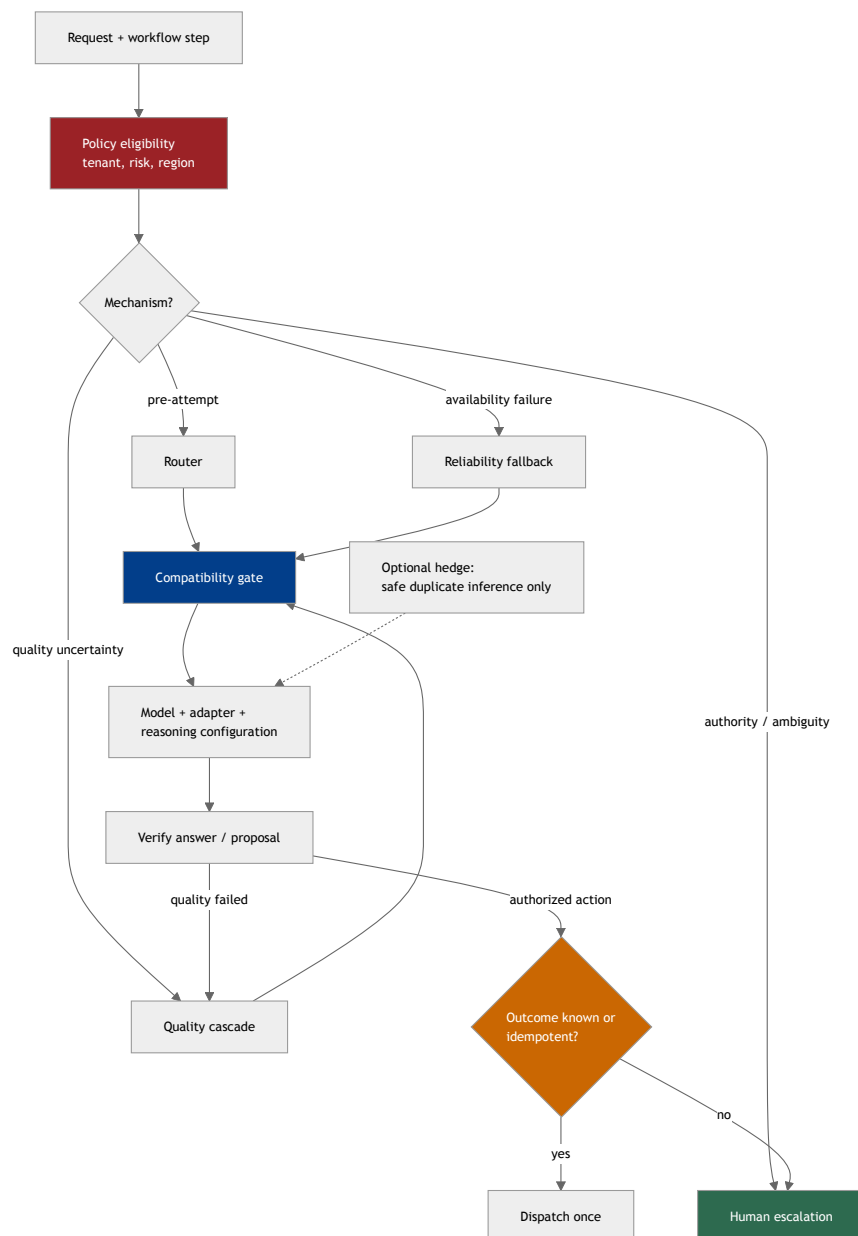
8.9 Treat Every Route Change as a Release

A model upgrade, provider swap, reasoning-setting change, new cascade verifier, region change, or gateway policy edit changes the model-plus-harness configuration. Release it through the same eval and rollback discipline as code.

Use shadow traffic only when data policy permits it and its duplicate request cannot cause actions. Canary an eligible configuration on bounded traffic, compare the slices in Section 8.7, and retain the route target and decision reason in the trace. Aliases are convenient for provider updates, but production reproducibility requires recording the resolved model snapshot when the provider exposes one.

Rollback is also subject to the compatibility gate. An older target may no longer support a new tool schema, output contract, or policy requirement. “Last known good” means last known good for the current harness contract, not merely a previously deployed model ID.

Diagram: A Policy- and Compatibility-Gated Route



Key Takeaways

- **Select a configuration, not a model name:** endpoint, modalities, limits, tools, output contract, reasoning state, policy, and region travel together.
- **Name the mechanism:** router, quality cascade, reliability fallback, hedging, and human escalation have different triggers and proofs.

- **Compatibility precedes fallback:** never strip inputs, tools, state, or policy requirements merely to make a backup target accept the call.
- **Retry does not mean repeat the side effect:** use stable action IDs, idempotency, and outcome verification before another dispatch.
- **Reasoning controls are provider-specific:** map a neutral quality/latency/cost objective through a tested adapter; opaque continuation state is not generally portable.
- **Policy defines the eligible set:** optimize cost and latency only among targets permitted for the tenant, risk, data path, and action.
- **Evaluate routing as a system:** slice quality, cost, latency, failure class, policy, and routing behavior.
- **A gateway is not automatically a control plane or PEP:** classify it by the management and enforcement functions it actually performs.

Further Reading

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
- Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
- Jeffrey Dean and Luiz André Barroso, *The Tail at Scale*, 2013. <https://research.google/pubs/the-tail-at-scale/>
- OpenAI, *Models*. <https://developers.openai.com/api/docs/models>
- OpenAI, *Model Guidance*. <https://developers.openai.com/api/docs/guides/latest-model>
- Anthropic, *Extended Thinking*. <https://platform.claude.com/docs/en/build-with-claude/extended-thinking>
- IETF, *RFC 9110: HTTP Semantics*, §9.2.2. <https://www.rfc-editor.org/rfc/rfc9110.html#section-9.2.2>
- OpenAI, *Data Controls in the OpenAI Platform*. <https://developers.openai.com/api/docs/guides/your-data#data-residency-controls>
- Anthropic, *Demystifying Evals for AI Agents*. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Kubernetes, *Kubernetes Components*. <https://kubernetes.io/docs/concepts/overview/components/>
- NIST, *Zero Trust Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>

Chapter 9: Agentic Workflow Patterns

Workflow patterns describe how a system arranges model calls, tools, checks, and state transitions. They do not change the responsibility boundary established in Chapter 6: a model can propose content, a route, a decomposition, or a next action, while the harness or runtime validates that proposal, executes authorized effects, records state, and returns observations. For client-executed tools, Anthropic's API contract likewise has the model emit a tool-use request and application code execute it ([Anthropic — How Tool Use Works](#)).

This chapter compares control-flow shapes. Chapter 10 owns durable execution state and event histories, Chapter 11 owns system evaluation, and Chapter 13 develops verifier hierarchies. A workflow diagram alone supplies none of those capabilities.

9.1 Three Control Regimes

Anthropic distinguishes **workflows**, whose model and tool calls follow predefined code paths, from **agents**, in which the model dynamically directs process and tool use; it recommends starting with the simplest architecture that meets the need ([Anthropic — Building Effective Agents](#)). For implementation, a three-way distinction is more precise:

Regime	Who selects the next step?	Who executes and owns state?	Best fit
Deterministic workflow	Code selects a predefined transition; a model may fill in one node's output	The workflow controller/runtime	Stable processes whose branches and checks are known in advance
Model-directed loop	The model proposes the next action from the current observation	The harness validates and dispatches; the runtime owns budgets, status, and stop rules	Open-ended tasks whose next useful step cannot be enumerated reliably beforehand
Hybrid workflow	Code fixes the outer graph while a model chooses inside bounded nodes or loops	The outer controller remains authoritative; bounded inner loops have explicit budgets and handoff contracts	Production tasks that need both predictable boundaries and local flexibility

An *augmented LLM*—Anthropic's term for a model connected to retrieval, tools, and memory—can appear in any of these regimes ([Anthropic — Building Effective Agents](#)). Adding a model to one node does not turn a deterministic workflow into an autonomous agent. Conversely, letting a model select an action does not grant it credentials or make its transcript the source of execution state.

The word **routing** also appears at two levels. Workflow routing chooses a processing branch; model routing chooses which model or reasoning budget should handle a call. Chapter 8 covers the second decision. A workflow may use both, but the decisions should have separate records and evaluations.

9.2 Every Pattern Needs an Operational Contract

A box-and-arrow diagram is incomplete until each pattern specifies six fields:

- Applicability:** what task structure makes the pattern worth its added calls and coordination?
- State owner:** which external component records the current node, attempts, artifacts, budgets, and pending work?
- Stop condition:** which success, failure, abstention, escalation, cancellation, or exhaustion event ends it?
- Failure propagation:** how are invalid outputs, tool failures, timeouts, partial effects, and downstream contamination represented?
- Evaluation unit:** is quality scored at a call, route, branch, iteration, full run, final artifact, or external outcome?
- Effect boundary:** where are model proposals validated, authorized, executed, and confirmed?

Anthropic's evaluation terminology separates a task from repeated trials, the transcript from the final outcome, and the agent harness from the evaluation harness ([Anthropic — Demystifying Evals for AI Agents](#)). That distinction matters here: a pattern can produce an attractive transcript while routing to the wrong branch, duplicating a side effect, omitting a subtask, or accepting a bad final artifact. Pattern evaluations must inspect the relevant external state and outcome, not only model text.

The effect boundary is constant across all patterns. Chapter 6 defines the full proposal-to-outcome lifecycle. The descriptions below focus on the other five fields and assume that every consequential tool call still passes through that lifecycle.

9.3 Compositional Workflow Patterns

Anthropic presents prompt chaining, routing, parallelization, orchestrator-workers, and evaluator-optimizer as common compositional patterns, with different tradeoffs in latency, cost, and predictability ([Anthropic — Building Effective Agents](#)). They can be deterministic, model-directed, or hybrid depending on where selection occurs.

9.3.1 Prompt Chaining

Prompt chaining feeds one stage's validated output into a later stage. The chain is usually deterministic even when every stage contains a model call: code, not the model, chooses the next node.

- **Use when:** the task decomposes into a stable sequence and intermediate interfaces can be stated clearly.
- **State owner:** the workflow controller records the current stage, validated intermediate values, artifacts, and attempts.
- **Stop when:** the last stage passes its acceptance gate, or a stage reaches a terminal failure or escalation path.
- **Typical failure:** an early omission or malformed intermediate value contaminates later stages; latency also accumulates serially.
- **Eval unit:** grade each stage contract and the end-to-end outcome. A high final score should not hide a stage that leaks data or creates an incorrect effect.

Do not pass arbitrary prose between stages when a typed intermediate representation is possible. A gate can reject or repair an invalid proposal before it becomes the next stage's assumed fact.

9.3.2 Routing

Routing assigns an input to a specialized branch. A deterministic rule, a conventional classifier, or a model can propose the route; the harness applies the routing policy and performs the dispatch.

- **Use when:** inputs form operationally meaningful categories with distinct handlers, and the boundary can be evaluated.
- **State owner:** the router/controller records the candidate label, confidence or abstention signal, policy version, selected branch, and any fallback.
- **Stop when:** one permitted branch accepts the item, or the router abstains, escalates, or rejects it under an explicit rule.
- **Typical failure:** an ambiguous input is confidently misrouted and fails silently inside a plausible but wrong branch.
- **Eval unit:** evaluate the routing decision on labeled slices and the downstream task outcome conditional on that route. Include abstentions and fallback cost.

Keep workflow routing separate from Chapter 8's model routing. “Send to billing” and “use model B” are different decisions even if one component computes both.

9.3.3 Parallelization

Parallelization launches multiple branches before aggregation. **Sectioning** assigns independent subtasks; **voting** asks multiple workers to address the same decision ([Anthropic — Building Effective Agents](#)). A model may propose the split or votes, but an external coordinator creates branch records, enforces limits, and aggregates completed results.

- **Use when:** branches are genuinely independent enough to run concurrently, or multiple samples have measured value for a decision.
- **State owner:** the coordinator owns the branch set, call IDs, deadlines, partial results, cancellation status, and aggregation rule.
- **Stop when:** all required branches finish, a specified quorum is reached, or a deadline/failure policy selects a partial-result or escalation path.
- **Typical failure:** partial completion is mistaken for total success; workers duplicate work or share the same mistaken premise; aggregation drops minority evidence.
- **Eval unit:** score branches and the aggregate outcome, including coverage, agreement calibration, wall-clock latency, and total resource use.

Agreement is not proof. Voting needs diversity and calibration measurements; sectioning needs an explicit coverage check so that the split does not omit necessary work.

9.3.4 Orchestrator–Workers

In orchestrator–workers, a model dynamically proposes a decomposition, workers perform bounded subtasks, and a synthesis step combines their results. Anthropic uses this pattern in its research system, where a lead agent delegates searches to parallel subagents and then synthesizes their findings ([Anthropic — How We Built Our Multi-Agent Research System](#)). That is a documented system and workload, not evidence that the pattern improves every task.

- **Use when:** the necessary subtask shape depends on the input, the subtasks have clear ownership, and their results can be merged through explicit artifacts or evidence.
- **State owner:** the coordinator/runtime owns the task graph, worker identities and scopes, budgets, artifacts, completion status, and synthesis attempt.
- **Stop when:** required subtasks are accounted for and the synthesized outcome passes its task-level gate, or fan-out, time, cost, or no-progress limits are reached.
- **Typical failure:** uncontrolled fan-out, duplicate or missing coverage, conflicting writes, lost provenance, or a synthesis that overstates worker findings.
- **Eval unit:** grade the full task outcome plus decomposition coverage, worker handoff quality, synthesis faithfulness, latency, and cost.

Chapter 5's handoff contract applies: a worker result needs evidence, artifact pointers, identity and authority scope, uncertainty, and verification status. A higher-privilege orchestrator must not turn an unverified worker summary into a privileged effect.

9.3.5 Evaluator–Optimizer

Evaluator–optimizer alternates candidate generation with feedback and revision. Anthropic recommends it when evaluation criteria are clear and iterative refinement has demonstrable value ([Anthropic — Building Effective Agents](#)). A model-generated critique is an observation for the controller; it does not itself execute the next iteration or declare the external task complete.

- **Use when:** a candidate can be improved iteratively, the acceptance criteria are repeatable, and the improvement justifies extra calls.
- **State owner:** the loop controller owns candidate versions, feedback, scores, iteration count, budget, and accepted artifact.
- **Stop when:** the acceptance rule passes, progress stalls, the iteration/budget cap is reached, or the evaluator abstains and escalates.
- **Typical failure:** evaluator and generator share the same blind spot; scores oscillate; the candidate optimizes the rubric without improving the real outcome; the loop never converges.
- **Eval unit:** inspect each iteration and the final accepted outcome; measure improvement over the initial candidate, false accepts/rejects, resource use, and regressions outside the optimized criterion.

This pattern only establishes where an evaluation checkpoint sits. Chapter 11 explains tasks, trials, graders, and outcome measurement; Chapter 13 explains when deterministic checks, self-checks, or independent evaluators provide the right level of assurance. This chapter does not impose a universal “maker must never be checker” rule.

9.4 Model-Directed Loops and Bounded Hybrid Agents

A ReAct-style loop interleaves model reasoning, an action proposal, an externally produced observation, and another model decision ([Yao et al. — ReAct](#)). In production, the harness must make that abstract loop operational:

- **Use when:** the next useful action depends materially on observations that cannot be predicted in a fixed graph.
- **State owner:** the runtime owns run/step/action identities, context references, budgets, approvals, tool outcomes, and current status. The transcript is a model-facing view, not the authoritative state.
- **Stop when:** an externally checked success condition holds, no work remains, the task is blocked, approval or human input is required, or an iteration/time/cost/no-progress/cancellation limit fires.
- **Typical failure:** drift, repeated ineffective actions, retries after unknown outcomes, premature self-declared completion, or context growth that erases earlier constraints.

- **Eval unit:** score the complete run and external outcome, then diagnose at action and step level. Include side effects, recovery behavior, latency, and resource use.

A **hybrid** puts this loop inside a deterministic envelope. Code may fix ingest → analyze → review → publish, while a bounded agent chooses search and analysis actions inside the analyze node. The outer workflow owns entry conditions, allowed capabilities, budgets, handoff artifacts, and exit gates. This structure localizes model uncertainty without pretending that a static DAG can enumerate every useful inner step.

HumanLayer's 12-Factor Agents and LangChain's deploybot description both advocate focused agentic components inside more deterministic control structures ([HumanLayer — 12-Factor Agents](#); [LangChain — The Anatomy of an Agent Harness](#)). Treat their suggested shapes as practitioner case studies, not universal turn-count limits. Choose the loop boundary with workload-specific evaluations.

For a hybrid pattern:

- **State owner:** the outer workflow remains authoritative; the inner loop receives a scoped task state and returns a typed handoff.
- **Stop condition:** both the inner loop's budget/exit contract and the outer node's acceptance gate must be satisfied.
- **Failure propagation:** inner failure becomes an explicit node result—retryable, terminal, partial, unknown, or escalated—rather than an invented success.
- **Eval unit:** test inner-loop capability on its bounded task and the full workflow outcome, including handoff correctness.

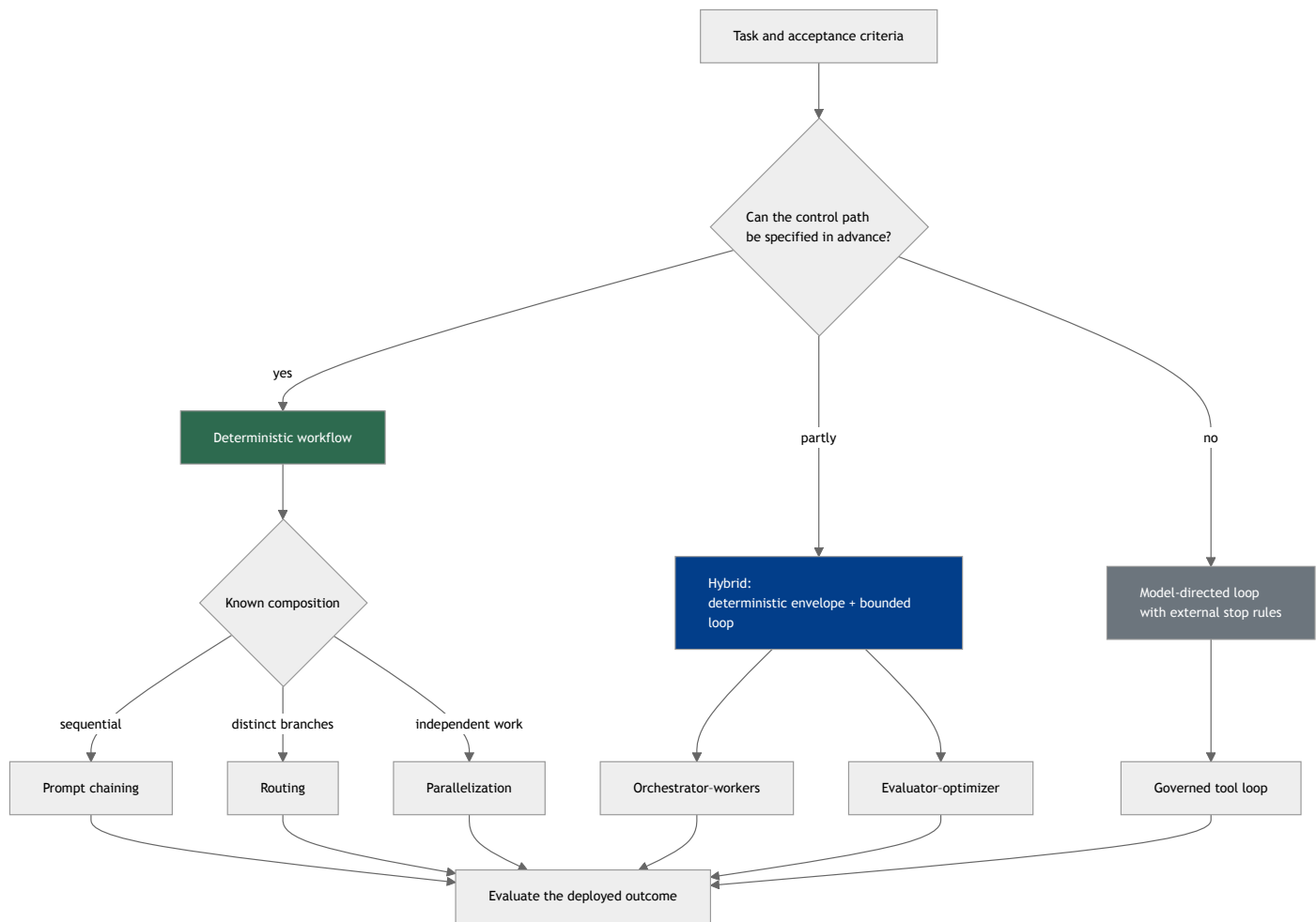
9.5 Research Patterns Are Design Lineage, Not Production Guarantees

Several influential papers explore search, reflection, and planning structures. They are useful design vocabulary, but results from a paper's tasks, models, tools, and budgets do not establish equal production maturity or a universal improvement. If implemented, their search frontier, reflections, candidate set, tool results, and termination status still belong to the external harness or runtime.

Pattern	Mechanism studied	Production interpretation
Reflexion	Stores natural-language reflection after feedback and reuses it on later attempts (Shinn et al. — Reflexion)	Treat reflection as fallible memory with provenance and expiry, not as verified diagnosis
Self-Refine	Uses model-generated feedback to revise the model's earlier output iteratively (Madaan et al. — Self-Refine)	The controller still needs an external cap and acceptance rule; self-satisfaction is not outcome confirmation
CRITIC	Uses external tools during critique and correction (Gou et al. — CRITIC)	Tool evidence can strengthen a check, but every call still needs Chapter 6's validation and authorization lifecycle
Tree of Thoughts (ToT)	Searches over candidate “thought” branches with lookahead and backtracking (Yao et al. — Tree of Thoughts)	The harness owns the frontier, branching budget, value records, and termination; text branches are not durable execution state
Language Agent Tree Search (LATS)	Applies Monte Carlo tree search to language-agent trajectories using value estimates and reflection (Zhou et al. — LATS)	Tree growth, environment effects, and rollout budgets need explicit isolation and accounting
ReWOO	Separates an upfront Planner, tool-executing Workers, and a Solver that combines observations (Xu et al. — ReWOO)	A prewritten plan can reduce repeated planning in the studied setup, but workers must surface plan-invalidating observations rather than execute stale steps blindly

These mechanisms can be nested inside the compositional patterns above. For example, ToT can search inside one deterministic node, Reflexion can update scoped memory after a failed trial, and ReWOO can define a hybrid plan–execute–synthesize workflow. Evaluate the composition actually deployed rather than attributing quality to the pattern name.

9.6 Selecting and Composing Patterns



Patterns compose along separate axes. A router can select a chain; a chain can contain a parallel stage; an orchestrator can route workers to different models using Chapter 8's policy; an evaluator-optimizer can operate inside one bounded node. At every boundary, preserve a stable task identity, typed state transition, explicit budget, failure status, and outcome evidence.

Prefer the least dynamic regime that satisfies the task. More model-directed control can be useful when observations change the path, but it also increases the state space that stop rules, recovery, and evaluation must cover. Frameworks may package these patterns; they do not remove the need to inspect the actual prompts, calls, state transitions, and effects. Anthropic similarly cautions that framework abstractions can obscure underlying prompts and responses during early development ([Anthropic — Building Effective Agents](#)).

Key Takeaways

- **Models propose; external systems execute and remember:** workflow patterns never transfer authority or state ownership into the model.
- **Distinguish deterministic, model-directed, and hybrid control:** a workflow containing an LLM is not automatically an autonomous agent.
- **Give every pattern an operational contract:** applicability, state owner, stop condition, failure propagation, eval unit, and effect boundary must be explicit.
- **Evaluate the right unit:** calls, routes, branches, iterations, runs, artifacts, and external outcomes answer different questions.
- **Use workflow routing and model routing as separate decisions:** Chapter 8 owns model and reasoning-budget selection.
- **Parallelism and delegation need coverage accounting:** agreement is not proof, and a worker summary is not authorization.
- **Evaluator-optimizer does not define verifier strength:** grader design belongs to Chapter 11 and verifier independence to Chapter 13.

- **Research pattern names are not maturity claims:** Reflexion, Self-Refine, CRITIC, ToT, LATS, and ReWOO must be evaluated in the exact deployed configuration.
- **Hybrid patterns are often the practical middle:** deterministic outer boundaries can contain locally flexible model-directed work.

Further Reading

- Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
- Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- Anthropic, *Demystifying Evals for AI Agents*, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
- Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023. <https://arxiv.org/abs/2210.03629>
- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, NeurIPS 2023. <https://arxiv.org/abs/2303.11366>
- Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, NeurIPS 2023. <https://arxiv.org/abs/2303.17651>
- Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, 2023. <https://arxiv.org/abs/2305.11738>
- Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, NeurIPS 2023. <https://arxiv.org/abs/2305.10601>
- Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, 2023. <https://arxiv.org/abs/2310.04406>
- Binfeng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, 2023. <https://arxiv.org/abs/2305.18323>

Chapter 10: State, Event Histories, and Production Factors

An agent cannot recover from a crash, wait safely for approval, or explain a duplicate side effect if its only state is the next prompt. This chapter defines the durable data model beneath later discussions of evaluation, long-running tasks, tracing, AgentOps, and fleets. HumanLayer's “12 Factor Agents” remains useful as a named practitioner manifesto, but it is not a formal standard or a complete architecture ([HumanLayer — 12-Factor Agents](#)).

10.1 Six Objects That Must Not Collapse Into One

The word *history* is often used for several different products. This book uses the following definitions:

Object	Canonical meaning	Primary use	What it is not
Execution state	Structured current workflow state: status, current step, pending action, budgets, retry count, approval state, and relevant business references	Decide what may happen next	The model's context or an unstructured transcript
Event history	An ordered, durable record of accepted workflow events sufficient for recovery under a stated replay contract	Reconstruct state and resume	Automatically a compliance audit log
Checkpoint	A recoverable state snapshot plus the history position and references required to continue	Bound recovery time and create a resume point	The complete causal history
Artifact	An addressable work product such as a file, diff, report, dataset, build, or large raw result	Preserve outputs and evidence outside live context	Execution state merely because the model can read it
Trace	Observability data that represents an execution path with spans, timestamps, attributes, events, links, and status	Debug latency, causality, and failures	A durable recovery source or audit record by definition
Audit record	A protected record retained for accountability under defined requirements for content, identity, timestamps, integrity, access, and retention	Investigation, governance, and compliance	Every log line or sampled trace

Temporal defines Event History as an append-only log used to recover a workflow after crashes or failures ([Temporal — Events and Event History](#)). OpenTelemetry defines traces in terms of spans and permits sampling, so traces may intentionally omit executions or details ([OpenTelemetry — Tracing API](#); [OpenTelemetry — Tracing SDK](#)). NIST SP 800-53's AU control family separately addresses audit-record content, generation, protection, review, and retention ([NIST SP 800-53 Rev. 5](#)). These sources establish why one product can feed another without becoming its synonym.

A production trace may link to an event ID, an event may reference an artifact, and selected events may be copied into a protected audit store. That lineage is useful. It still does not justify the equation `trace = history = audit`. Context and memory are different again: context is the representation visible to one model call, while memory is information selected for possible use in future calls. Neither should be the authoritative execution state.

10.2 Identity Hierarchy: Session, Run, Step, Action, Attempt, and Call

Durability requires identifiers that survive process and model-context boundaries. This book uses a local hierarchy rather than assuming one vendor's naming scheme:

Identifier	Identifies	Lifetime and relation
<code>session_id</code>	A durable user or work stream, including its conversations and artifacts	May contain several runs, resumptions, or branches
<code>run_id</code>	One logical workflow execution under a pinned objective and configuration lineage	Belongs to one session and survives worker replacement
<code>step_id</code>	One logical state transition or orchestration position	Belongs to one run; may be revisited only under explicit loop semantics
<code>action_id</code>	One intended external effect	Belongs to one step and remains stable across delivery retries
<code>attempt_id</code>	One execution or delivery try for a step or action	A retry dimension beneath the same logical step/action
<code>call_id</code>	One model-call or tool-call protocol exchange and its correlated output	Belongs to a step or attempt; never substitutes for an action's idempotency identity

The containment path is usually `session → run → step → action → attempt`; model and tool `call_id`s attach at the step or attempt that issued them. Branches receive new run or branch identities and record their parent history position. A worker ID, sandbox ID, model context ID, and trace ID are associations, not replacements for these durable IDs.

Some dispatchers use `invocation_id` for their internal tool-dispatch record. Publish its mapping instead of adding another ambiguous synonym: it may alias `action_id` when one invocation represents exactly one intended effect, while a composite invocation can contain several actions and attempts.

This separation answers practical questions. A crashed worker can be replaced without changing `run_id`. A timed-out payment retry gets a new `attempt_id` but retains its `action_id` and idempotency key. A model retry gets a new `call_id`; it must not silently redefine an already-approved action. Temporal itself distinguishes a durable Workflow ID from individual Run IDs in a workflow-execution chain, illustrating why logical work and one execution instance need separate identities ([Temporal — Workflow Execution](#)).

10.3 Reducers, Event Sourcing, Checkpoints, and Workflow Engines

Four related techniques solve different problems:

1. **Reducer.** A deterministic function such as `state_next = reduce(state_previous, event)` computes a projection. It can run over a full history, a suffix after a checkpoint, or events copied from another store. A reducer is code, not persistence.
2. **Event sourcing.** Accepted domain changes are stored as an append-only event stream that is authoritative for the sourced state. Microsoft describes the pattern as persisting a series of actions and rebuilding current state by replaying them; it also warns that the pattern adds complexity and should be adopted selectively rather than for every CRUD subsystem ([Azure Architecture Center — Event Sourcing](#)).
3. **Snapshot or checkpoint.** A materialized state image at a known history position reduces rehydration cost. Azure's event-sourcing guidance treats snapshots as an optimization that avoids replaying the entire stream ([Azure Architecture Center — Event Sourcing](#)). In this book, a *checkpoint* additionally includes the references and resume contract needed to continue execution.
4. **Workflow engine.** Runtime infrastructure schedules work, persists timers and waits, dispatches tasks, manages retry policy and ownership, and resumes progress after failure. Temporal, for example, records commands and events so a Workflow Execution can recover from its latest durable state ([Temporal — Workflow Execution](#); [Temporal — History Service Architecture](#)).

These choices are composable, not mandatory as a bundle. A system can use a reducer over an ordinary append-only workflow log without event-sourcing its customer database. It can take checkpoints without discarding the prior event history. A workflow engine can orchestrate activities whose domain data remains in conventional transactional stores. Conversely, an event-sourced business aggregate does not by itself provide timers, worker leases, approval waits, or activity retries.

Keep an explicit source-of-truth map. For example, the event history can own workflow position and approval transitions; a payment system can own transaction outcome; an artifact store can own generated files; a current-state projection can serve queries. “Unify state” should mean shared identity and deliberate derivation, not forcing every byte into one prompt, table, or event stream.

10.4 Durable Execution and Honest Replay

Durable execution means that progress survives process, worker, network, or infrastructure failure and can resume from recorded state. Temporal's documented replay mechanism runs workflow code against an existing Event History, checks newly generated commands against that history, and resumes after the last recorded event ([Temporal — Workflow Execution](#)). That is a concrete implementation, not the only possible engine.

Agent workflows add a critical boundary: model calls and external tool calls are non-deterministic activities. Temporal records completed Activity results in Event History, and its Side Effect primitive returns a recorded result instead of re-executing the function during replay ([Temporal — Activities](#); [Temporal — Events and Event History](#)). Apply the same rule to an agent:

- record the complete model response or structured proposal under its `call_id`;
- record each tool result, error, external operation ID, and outcome status;
- on replay, return those recorded outputs to the reducer or workflow code;
- do **not** call the model again or rerun a side-effecting tool and label the new trajectory a replay;

- if a new answer is desired, create an explicit retry, fork, or new run with new identities and lineage.

Replaying an event history reconstructs an earlier execution. Re-running from the same user input samples a new execution. The second operation is valuable for evals and recovery experiments, but it is not deterministic replay.

Workflow-code changes also need version discipline. A reducer or workflow version that emits a different command sequence for the same old history can make replay incompatible. Store code, model, prompt, tool-schema, policy, and reducer versions with the run; use migration, version gates, or a new run rather than silently reinterpret old events. Checkpoints must state the history position and reducer/schema version that produced them.

10.5 Events Need a Contract

An event should describe an accepted fact, not an uncommitted intention disguised in past tense. A practical envelope includes:

```
event_id, event_type, schema_version, occurred_at, recorded_at
session_id, run_id, step_id, action_id, attempt_id, call_id
actor, tenant, source, causation_id, correlation_id
payload_or_artifact_ref, policy_version, previous_sequence
```

Not every event has every ID. `approval_received` needs the approver and action; `worker_replaced` needs the run and worker identities; `model_call_completed` needs the call and step. The envelope should make omitted fields intentional rather than ambiguous.

Append with an expected sequence or version so two workers cannot both commit incompatible next events. Give inbound commands and externally delivered events stable deduplication identities. Microsoft notes that ordering and per-entity event identifiers are central event-sourcing concerns, and that event stores are not interchangeable with message brokers ([Azure Architecture Center — Event Sourcing](#)). A queue can deliver work; the accepted workflow history decides what became part of the run.

Large payloads should normally be stored as immutable or versioned artifacts and referenced by content hash or durable URI. Keep enough metadata to authorize access and verify the referenced version. Redaction must preserve the fact that an event occurred and the applicable lineage; deleting bytes from a model-facing context is not the same operation as applying retention or erasure policy to durable stores.

10.6 Recovery Is a State Machine, Not “Retry Everything”

The runtime should define a recovery transition for each interruption:

Interruption	Durable record before recovery	Safe next transition
Process or machine crash	Last committed event, current checkpoint, ownership lease	Rehydrate with the recorded history; issue only work not already accepted
Worker replacement or deploy	Run ID, worker/config versions, in-flight step and lease	Acquire ownership, replay recorded outputs, continue under an explicit compatible version
Tool timeout or lost response	Action ID, attempt ID, idempotency key, external operation reference, status <code>unknown</code>	Reconcile external state before retrying a consequential action
Duplicate delivery	Command/event dedup key and accepted sequence	Return the recorded disposition; do not append or execute twice
Approval wait	Exact pending action, policy decision, required approver scope, expiry	Release compute; resume only from an authenticated approval/deny event
Cancellation request	Requester, scope, reason, time, current action state	Stop scheduling new work, propagate cancellation, then record actual terminal or partial outcome

Temporal's workflow status model distinguishes a cancellation request from a successfully handled `Cancelled` terminal state, and its Activity history separately records requested and accepted cancellation ([Temporal — Workflow Execution](#); [Temporal — Events and Event History](#)). Preserve that distinction. Cancellation is not rollback, and a timeout is not evidence that no side effect occurred.

Approval waits should be durable events, not a sleeping web process. HumanLayer's manifesto correctly emphasizes launch/pause/resume APIs and structured human-input events ([HumanLayer — 12-Factor Agents](#)). For high-risk

actions, however, [Chapter 7](#)'s non-bypassable policy gate and [Chapter 14](#)'s approval semantics apply: the model may request consultation, but the runtime decides whether approval is mandatory and verifies the approver.

Every recovery path needs a budget and a terminal state. Repeated transient failures can become `failed` or `needs_human`; incompatible workflow code can become `migration_required`; an unresolved external effect can remain `unknown` rather than being falsely labeled `failed`. The event history should preserve how that decision was reached.

10.7 The Twelve Factors: Principle or Implementation Choice?

HumanLayer presents the following twelve factors as guidance for production LLM software ([HumanLayer — 12-Factor Agents](#)). Read them as a manifesto to evaluate, not as a standards checklist:

HumanLayer factor	Durable principle worth keeping	Context-dependent implementation choice
1. Natural Language to Tool Calls	Separate model proposals from deterministic execution	Not every task needs a tool; wire format need not be JSON
2. Own Your Prompts	Version and evaluate model-visible configuration	Teams may use provider/framework templates if versions and rendered inputs remain inspectable
3. Own Your Context Window	Make context assembly explicit	XML and a single-message event rendering are formatting choices
4. Tools Are Just Structured Outputs	A tool call is output, not an effect	Provider-native, MCP, or custom serialization can represent it
5. Unify Execution and Business State	Use shared IDs and defined sources of truth	Deriving all execution state from one event stream is optional; external systems may own business facts
6. Launch/Pause/Resume With Simple APIs	Long-running work needs explicit lifecycle operations	Endpoint shape and workflow engine are implementation choices
7. Contact Humans With Tool Calls	Human requests and responses should be structured, durable events	Model-requested consultation cannot replace a mandatory runtime approval gate
8. Own Your Control Flow	The harness must own stopping, waiting, retry, and dispatch	A custom loop, graph runtime, and managed workflow engine are alternatives
9. Compact Errors Into Context	Preserve actionable error information for the next decision	Full diagnostics belong in history/artifacts; exact compaction and escalation thresholds require evals
10. Small, Focused Agents	Bound scope and evaluate reliability near the model's capability frontier	"3–10, maybe 20 steps" is the manifesto's heuristic, not a universal limit
11. Trigger From Anywhere	Normalize external triggers into authenticated, idempotent events	Slack, email, SMS, webhook, and cron support are product choices
12. Make the Agent a Stateless Reducer	Reconstruct current execution state from durable inputs where practical	A pure reducer does not require event-sourcing every domain system

This reading preserves the manifesto's strongest insight—agent systems are mostly software around probabilistic calls—without turning its examples into universal facts. In particular, “stateless reducer” describes worker behavior: a replaceable worker can compute from durable inputs. The system as a whole is intentionally stateful.

10.8 From One Program to AgentOps and Fleets

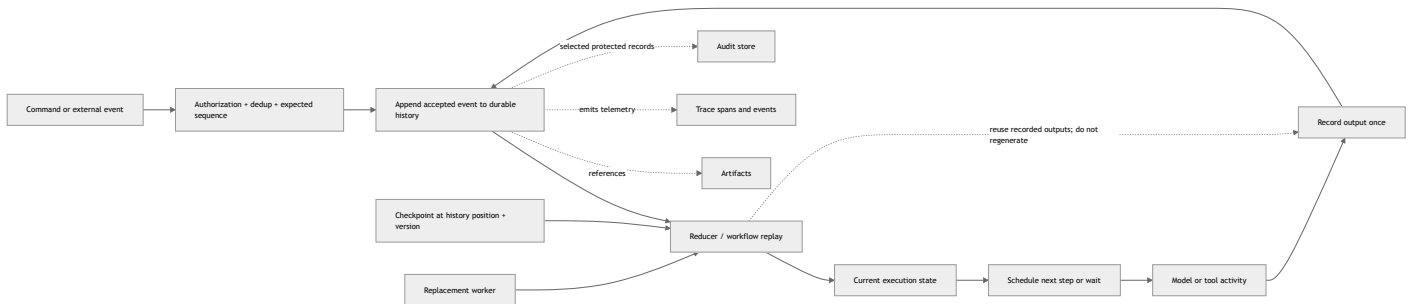
A local prototype may keep execution state and artifacts in a repository. A shared service additionally needs tenant boundaries, schema migration, retention, quotas, worker ownership, configuration rollout, and reconciliation across many active runs. That is a shift in operational scope, not evidence that one event log should become a universal control plane.

[Chapter 17](#) treats traces as observability inputs for iteration. [Chapter 18](#) applies this chapter's identities and data boundaries to cost attribution, privacy, retention, deployment, and incident response. [Chapter 19](#) adds fleet identity, policy administration, distributed policy enforcement, lineage, and protected audit records. The handoff is deliberate:

- this chapter owns execution state, recovery history, checkpoints, and replay semantics;
- AgentOps owns operating and changing those systems safely at scale;
- the fleet/control-plane chapter owns cross-run identity, governance, and enforcement topology.

The boundary also prevents a common shortcut: an event history can supply selected evidence to AgentOps or an audit pipeline, but it is not automatically complete, access-controlled, immutable enough, or retained long enough for every audit requirement. Those properties must be designed and verified explicitly.

Diagram: Durable State, Replay, and Evidence Products



Key Takeaways

- **Execution state, event history, checkpoint, artifact, trace, and audit record are different products:** link them with lineage instead of collapsing their guarantees.
- **Use durable identities:** session, run, step, action, attempt, and call IDs answer different recovery and deduplication questions.
- **Reducer, event sourcing, checkpoint, and workflow engine are not synonyms:** each is independently adoptable.
- **Replay must reuse recorded model and tool outputs:** calling them again creates a new attempt or branch, not a replay of the old execution.
- **Recovery is explicit state transition:** crash, worker replacement, timeout, duplicate delivery, approval wait, and cancellation require different evidence and next actions.
- **A timeout or cancellation request is not a confirmed external outcome:** reconcile consequential effects.
- **HumanLayer's twelve factors are a named manifesto:** preserve general ownership and lifecycle principles while labeling XML formats, step counts, channels, and full event sourcing as choices.
- **A stateless worker runs over a stateful system:** durable state lives outside replaceable compute.
- **This chapter feeds AgentOps and fleet governance:** it does not redefine traces as histories or histories as audit records.

Further Reading

- Temporal, *Workflow Execution*. <https://docs.temporal.io/workflow-execution>
- Temporal, *Events and Event History*. <https://docs.temporal.io/workflow-execution/event>
- Temporal, *Activities*. <https://docs.temporal.io/activities>
- Temporal, *History Service Architecture*. <https://github.com/temporalio/temporal/blob/main/docs/architecture/history-service.md>
- Microsoft Azure Architecture Center, *Event Sourcing Pattern*. <https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing>
- OpenTelemetry, *Tracing API*. <https://opentelemetry.io/docs/specs/otel/trace/api/>
- NIST, *SP 800-53 Rev. 5: Security and Privacy Controls*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>

Chapter 11: Evaluation

Foundations Chapter 13 establishes the model-behavior layer: representative tasks, repeated trials, grader types, pass@k, pass^k, and the limits of metrics as proxies. It explicitly hands tool execution, permissions, side effects, agent trajectories, environment-state grading, operational reliability, and release gates to this volume ([LLM Foundations — Chapter 13](#)).

This chapter evaluates the system that users actually encounter: a model plus an agent harness, runtime, tools, policies, and environment. Its output is not just a score. It is evidence for a bounded claim such as “configuration C is ready for task distribution D under budget B and policy P.”

11.1 The Evaluation Contract

An evaluation begins by stating the claim before choosing tasks or metrics. OpenAI's guidance for trustworthy evaluations distinguishes capability elicitation, controlled comparison, and safeguard-performance claims because each requires a different harness and different supporting evidence ([OpenAI — Trustworthy Third-Party Evaluations](#)).

A release-oriented contract should name:

- the task distribution and excluded cases;
- the model-plus-harness configuration under test;
- the environment, tools, permissions, and data available;
- token, time, attempt, cost, and side-effect budgets;
- success, failure, abstention, and policy-violation criteria;
- the graders and human adjudication policy;
- the uncertainty and per-slice reporting required for a decision.

A headline score without this contract is easy to overgeneralize. It describes observed behavior under one setup, not an intrinsic property of a model or agent.

11.2 Canonical Agent-Eval Vocabulary

Anthropic defines the main components of an agent evaluation and explicitly separates the evaluation harness from the agent harness ([Anthropic — Demystifying Evals for AI Agents](#)). This book uses the following normalized vocabulary:

Term	Definition in this book
Task	One test case with declared inputs, initial environment, available capabilities, budgets, and success criteria.
Trial	One attempt by one tested configuration at one task from a restored baseline.
Grader	Versioned logic that judges one stated property from declared evidence and returns a verdict or score.
Eval transcript / trajectory	The complete model-, tool-, and observation-level record retained for grading one trial.
Outcome	The final relevant state produced by the trial, including environment and external-effect state rather than only response text.
Evaluation harness	Test infrastructure that provisions baselines, invokes the agent harness, records evidence, runs graders, aggregates trials, and produces reports.
Agent harness	The model-facing system under test that assembles context, drives the loop, validates proposals, and coordinates tools and runtime.
Evaluation suite	A versioned collection of tasks intended to support a stated capability, regression, reliability, or policy claim.

Anthropic sometimes uses *transcript*, *trace*, and *trajectory* as synonyms for a complete eval run. This book reserves **trace** for observability data made of spans, attributes, events, links, timestamps, and status; OpenTelemetry also permits sampling at the tracing layer ([OpenTelemetry — Trace API](#)). An eval trajectory may be derived from trace instrumentation only if the evaluation harness guarantees that every grader-required event and artifact is complete. A sampled production trace is not automatically an eval transcript.

11.3 Grade Four Evidence Layers Separately

Agent success is multi-layered. Do not collapse every grader into one notion of “correct.”

Evidence layer	What it can establish	Typical graders	What it cannot establish alone
Process / trajectory	Required or forbidden actions, approval use, tool arguments, turn count, budget, recovery behavior	Event assertions, policy checks, trajectory rubric	That the final artifact or environment is correct
Artifact	Properties of a produced file, diff, report, dataset, build, or citation set	Unit tests, static analysis, schema/content checks, expert rubric	That it was deployed or caused the intended external effect
Environment state	Database rows, filesystem state, UI/backend state, messages sent, permissions changed, cleanup completed	State queries, integration tests, independent API checks	That the user experienced or valued the result
User / business outcome	Resolution, acceptance, satisfaction, conversion, loss avoided, or another product-level effect	User confirmation, business event, controlled study, delayed outcome join	Fine-grained cause without additional evidence

Anthropic's flight-booking example makes the environment distinction concrete: an agent saying a flight was booked is transcript content; a reservation row in the environment is the outcome ([Anthropic — Demystifying Evals for AI Agents](#)). The same rule applies to code: a plausible final message is not a passing build, and a passing unit test is not necessarily a working end-to-end user flow.

Process grading is appropriate when the path itself is part of the contract—for example, mandatory approval, prohibited data access, or a cost ceiling. Do not require one exact tool sequence merely because the author expected it; Anthropic reports that rigid path grading can reject valid solutions that reach the intended outcome by another route ([Anthropic — Demystifying Evals for AI Agents](#)).

Preserve links across all four layers: `task_id` → `trial_id` → trajectory events → artifact versions → environment snapshot → user/business event. That lineage lets a reviewer distinguish a process violation from an artifact defect, an environment failure, or an outcome that changed after the trial.

11.4 Run Multiple Trials from Isolated, Recoverable Baselines

One successful trial demonstrates possibility, not reliability. Agent outputs and tool paths can vary, so the evaluation harness should run multiple trials per task when the product claim depends on repeatability ([Anthropic — Demystifying Evals for AI Agents](#)).

Every trial starts from a declared baseline:

- immutable environment image or reproducible fixture version;
- clean filesystem, database, queues, caches, browser profile, and external-resource namespace;
- task-specific credentials and tenant identity;
- controlled clock, network stubs, or recorded external responses where required;
- known resource quotas and no artifacts from earlier trials;
- teardown verification and a recovery path when setup or cleanup fails.

Anthropic documents both failure correlation and artificial score inflation from shared state, including an internal example where an agent inspected git history left by earlier trials ([Anthropic — Demystifying Evals for AI Agents](#)). Isolation therefore protects measurement validity, not merely test hygiene.

Distinguish three results:

1. **Task verdict:** the tested system passed, failed, or received partial credit.
2. **Infrastructure-invalid trial:** setup, dependency, grader, or cleanup failed, so the attempt does not measure the agent claim.
3. **System reliability failure:** an in-scope timeout, tool error, recovery failure, or resource exhaustion occurred under the declared production-like contract.

Do not discard category 3 as “eval noise.” Do not count category 2 as an agent failure without reporting it separately. Retry an invalid trial only under a recorded policy; otherwise repeated retries can hide an unstable evaluation harness.

11.5 Freeze the Tested Configuration and Report Slices

An eval result belongs to the complete tested configuration. Pin or record:

```
EvalConfiguration {  
  model_snapshot, provider_endpoint, reasoning_settings,  
  agent_harness_version, prompt_and_context_policy,  
  tool_catalog_and_schemas, runtime_and_environment_image,  
  retrieval_and_index_versions, policy_and_approval_rules,  
  retry_timeout_and_budget_rules,  
  task_suite, graders, adjudication_policy  
}
```

OpenAI's evaluation playbook likewise asks reports to disclose the system, harness, tool access, elicitation method, budgets, safeguards, and validity checks because these choices can materially change the claim a score supports ([OpenAI — Trustworthy Third-Party Evaluations](#)).

Report per-slice results before a blended aggregate. At minimum include:

- **task and quality:** domain, difficulty, language, input shape, ordinary vs edge vs historical failure;
- **routing:** route target, reasoning tier, cascade/fallback/hedge/human escalation, as defined in Chapter 8;
- **cost:** input, output, reasoning, cache, retrieval, tool, grader, and human-review cost;
- **latency:** router, model, tool, grader, end-to-end p50/p95/p99, timeout and queue time;
- **failure class:** model, transport, schema, tool, environment, recovery, grader, policy, ambiguous side effect;
- **policy:** tenant, identity, data region, model allowlist, approval, restricted-content and cross-tenant tests.

An aggregate can improve while a high-risk slice regresses. Release gates should therefore operate on critical slices and hard policy checks, not only on a weighted average.

11.6 Use pass@k , pass^k , and Uncertainty Correctly

Let p be a per-trial success probability. If k trials are independent and identically distributed, then:

- **pass@k** is the probability of at least one success: $1-(1-p)^k$;
- **pass^k** is the probability that all attempts succeed: p^k .

pass@k fits a product that genuinely generates several candidates and can identify a successful one. pass^k expresses repeated consistency. Anthropic emphasizes that the product contract determines which metric is appropriate ([Anthropic — Demystifying Evals for AI Agents](#)). Do not report pass@k for a one-attempt user experience merely because it produces a larger number.

When n sampled candidates contain c correct candidates, the HumanEval/Codex paper uses the estimator $1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$ for pass@k when $n \geq k$ ([Chen et al. — Evaluating Large Language Models Trained on Code](#)). That estimator answers a sampling question; it does not make correlated trials independent or prove that a selector can find the passing candidate.

Always report counts and uncertainty with the summary: number of tasks, trials per task, successes, invalid trials, and the interval method. NIST describes Wilson and related confidence intervals for a binomial proportion, which are preferable to presenting the observed fraction as exact truth ([NIST/SEMATECH — Confidence Intervals for a Proportion](#)). A binomial interval assumes the observations used in it behave like the stated binomial sampling model.

Agent trials are often clustered by task or environment. When attempts share a task, outage, cache, account, or fixture defect, treating every attempt as independent overstates precision. Use task-level paired comparisons, task-level bootstrap or a hierarchical model when appropriate, and disclose the sampling unit. Chapter 16 examines a measured case in which infrastructure configuration materially shifted agentic coding results ([Anthropic — Infrastructure Noise](#)).

Uncertainty is not only a confidence interval. Also report sensitivity to task-set version, grader version, invalid-trial policy, and infrastructure configuration. A statistically narrow estimate of a misconfigured eval is still invalid evidence.

11.7 Calibrate Graders as Measurement Instruments

Use deterministic graders for properties they can establish, model graders for rubric-based semantic judgment, and humans for expert or consequential ambiguity. Anthropic documents these three grader families and recommends calibrating model-based graders against human judgment ([Anthropic — Demystifying Evals for AI Agents](#)).

Every grader has a contract:

```
GraderResult {  
  grader_version, evidence_ids,  
  verdict: pass | fail | partial | abstain,  
  score, reason_code, explanation,  
  confidence_or_supporting_checks  
}
```

Build an adjudicated calibration set containing clear positives, clear negatives, boundary cases, valid alternative solutions, insufficient-evidence cases, and adversarial attempts to exploit the grader. For each grader and critical slice, measure:

- **false positive:** grader passes an adjudicated failure;
- **false negative:** grader fails an adjudicated success;
- **abstention rate:** grader correctly or excessively declines to decide;
- **coverage:** fraction of intended cases for which the grader can render a supported verdict;
- **agreement by slice:** where model and human judgments diverge, not only aggregate agreement;
- **cost and latency:** whether the grader is usable in a live loop, CI gate, or offline audit.

Here, *positive* means the grader's `pass` label. If a safety classifier instead defines “violation” as positive, the names reverse; publish the labeled confusion matrix rather than relying on the words alone.

The cost of errors is asymmetric. A false positive in a destructive-action safety check may matter more than many false negatives; a false negative in a creative-writing rubric may mostly waste review time. Set thresholds and escalation rules from consequence, not from one universal target accuracy.

Give semantic graders an explicit **abstain** or **insufficient evidence** option. Anthropic reports that model graders require careful calibration and recommends an “Unknown” path when evidence is insufficient ([Anthropic — Demystifying Evals for AI Agents](#)). Its motivated-mislabeling research also found that consequences disclosed to a model evaluator can alter labels and that abstention and tighter rubrics reduced, but did not eliminate, the observed effect ([Anthropic — Agentic Misalignment in Summer 2026](#)). Blind judges to candidate identity and deployment consequence when those facts are irrelevant to the rubric.

Define conflict resolution before running the release eval:

1. deterministic evidence wins only for the narrow property that check establishes;
2. a mandatory policy failure cannot be averaged away by quality graders;
3. grader disagreement or abstention enters a review queue with all evidence;
4. a human adjudicator applies a versioned rubric and records a reason code;
5. adjudicated cases feed the calibration set, but changing the rubric creates a new grader version.

Human judgment is not automatically consistent ground truth. Use multiple reviewers or an escalation reviewer for high-impact disputes, measure disagreement, and improve the rubric when experts interpret it differently.

11.8 Measure the Verifiers Used in Agent Loops

Chapter 13 will choose among schema checks, deterministic tests, environment checks, same-agent critique, independent model graders, and human review. This chapter supplies the evidence for that choice.

For every proposed verifier, measure:

- which failure classes it detects and misses;
- false-positive, false-negative, abstention, and disagreement rates;
- latency and cost at the point where it would run;
- whether its failures correlate with the generator's failures;
- susceptibility to leaked answers, reward hacking, or path overfitting;
- the consequence of accepting a bad result or rejecting a good one.

A verifier used inside the live loop and a grader used in the evaluation harness may share code, but they have different roles. The live verifier affects the trajectory and can give the agent repair information; the holdout grader measures the resulting system. If the agent can inspect the hidden tests, reference answer, or release grader, the evaluation may measure grader exploitation rather than task success.

Do not select an independent model judge merely because “the maker must not be the checker.” Independence can reduce one correlated failure, but the measured error profile, consequence, latency, and cost determine whether it is useful. Chapter 13 consumes this evidence to build a verifier hierarchy rather than a universal checker rule.

11.9 Turn Eval Results into Release Evidence

A release packet should be reconstructible from immutable artifacts:

- claim and decision owner;
- tested configuration and comparison baseline;
- task-suite manifest and slice coverage;
- environment baseline, isolation and reset evidence;
- trial counts, invalid-trial accounting, pass/fail/partial results;
- pass@k or pass^k where product semantics justify them, with uncertainty;
- grader calibration results, conflicts, abstentions, and human adjudications;
- policy and safety gate results;
- cost, latency, reliability, and routing slices;
- representative trajectories, artifacts, environment diffs, and known limitations;
- release decision, exception owner, expiry, rollback trigger, and post-release monitors.

OpenAI recommends that evaluation reports state the claim, tested system, harness and tools, budgets, elicitation choices, and checks for hazards such as broken tasks, reward hacking, contamination, refusals, and sandbagging ([OpenAI — Trustworthy Third-Party Evaluations](#)). The same discipline makes an internal release decision reviewable rather than turning a dashboard number into an unexplained approval.

Release gates should combine hard constraints and statistical evidence. A confirmed cross-tenant disclosure, unauthorized action, or broken mandatory approval is a gate failure even if aggregate task success improves. A small quality regression may require an uncertainty-aware decision rather than an automatic block. Record which rule produced the decision.

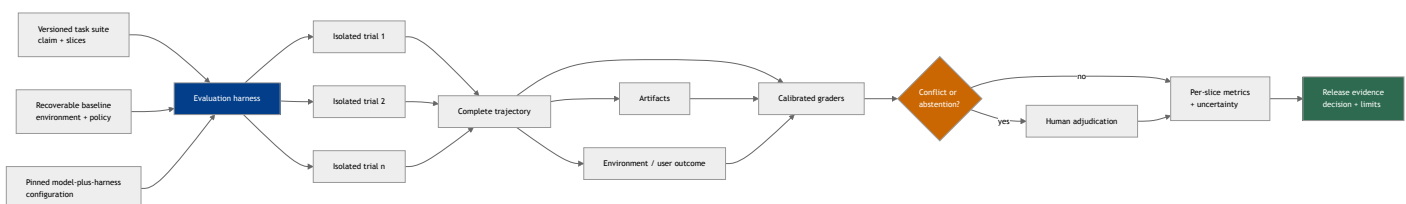
11.10 Maintain the Suite and Close the Production Loop

Read trajectories and grader disagreements on every important evaluation run. Anthropic reports that transcript review revealed ambiguous tasks, overly rigid graders, shared-state shortcuts, and other defects invisible in aggregate scores ([Anthropic — Demystifying Evals for AI Agents](#)). Failures should look fair: the task was solvable, the environment worked, and the verdict follows from declared evidence.

Evaluation suites are versioned products. Add escaped production failures, retain important regressions, rotate held-out cases, retire unsound tasks, and preserve history rather than editing old scores in place. If a task or grader changes, do not compare the new score with the old one as though only the agent changed.

Offline eval does not replace production monitoring, user feedback, controlled experiments, or incident review. Production supplies new task distributions and delayed user/business outcomes; the evaluation harness turns suitable cases into isolated, repeatable tests. Chapter 17 uses traces to find failure patterns, but this chapter's release evidence remains based on complete trial trajectories, artifacts, outcomes, and declared graders.

Diagram: From Isolated Trials to Release Evidence



Key Takeaways

- **Foundations ends at model-behavior eval; Harness owns system evidence:** trajectories, tools, permissions, side effects, environment outcomes, reliability, and release gates live here.

- **Use precise eval objects:** task, trial, grader, eval trajectory, outcome, evaluation harness, and agent harness are distinct.
- **A trace is not automatically a trajectory:** observability data may be sampled; grader evidence must be complete.
- **Grade evidence layers separately:** process, artifact, environment state, and user/business outcome answer different questions.
- **Repeat isolated trials:** recoverable baselines and invalid-trial accounting are prerequisites for reliability claims.
- **Report the tested configuration and slices:** quality, routing, cost, latency, failure, and policy results must accompany aggregates.
- **Use pass@k and pass^k according to product semantics:** state independence assumptions, counts, intervals, and the sampling unit.
- **Calibrate graders:** measure false positives, false negatives, abstention, coverage, conflict, and human adjudication.
- **Measure verifiers before placing them in loops:** Chapter 13 uses these error, consequence, latency, and cost profiles.
- **Ship release evidence, not a naked score:** preserve claims, versions, artifacts, uncertainty, exceptions, rollback rules, and post-release monitors.

Further Reading

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- OpenAI, *A Shared Playbook for Trustworthy Third-Party Evaluations*, May 2026. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>
- OpenTelemetry, *Tracing API*. <https://opentelemetry.io/docs/specs/otel/trace/api/>
- Mark Chen et al., *Evaluating Large Language Models Trained on Code*, 2021. <https://arxiv.org/abs/2107.03374>
- NIST/SEMATECH, *Confidence Intervals for a Proportion*. <https://itl.nist.gov/div898/handbook/prc/section2/prc241.htm>
- Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
- Anthropic Safeguards Research Team, *Agentic Misalignment in Summer 2026*, 2026. <https://alignment.anthropic.com/2026/agentic-misalignment-summer-2026/>

Chapter 12: Long-Running Agents and Multi-Context Tasks

Long-running agent engineering is **horizon extension**: preserving enough verified intent, progress, and operational context for useful work to continue beyond the useful life of one model context or worker session. It is not the practice of keeping one transcript open indefinitely.

This chapter assumes the durable runtime from [Chapter 10](#). Chapter 10 owns event history, replay, checkpoints, retries, action identity, approval-state persistence, and crash recovery; [Chapter 14](#) owns the human approval contract. This chapter asks a different question: **what semantic work state must survive each boundary so that the next context can continue the task correctly?**

12.1 Define the Horizon Before Extending It

“Long” can refer to several different quantities:

- **Wall-clock duration**: how long a run remains open.
- **Model-context span**: how much visible input one model call or context can use.
- **Session span**: how long one worker or sandbox is retained.
- **Task horizon**: how difficult a coherent end-to-end task the complete model-plus-harness system can finish at a stated reliability level.

These quantities are not interchangeable. A six-hour process may spend most of its time waiting, and a reset may replace the model context without ending the run or replacing the sandbox. Conversely, one short call may complete a task that took a human expert hours.

METR's **task-completion time horizon** is a useful but narrower capability measure. As of METR's May 8, 2026 Time Horizon 1.1 update, the 50%-time horizon is the human-expert task duration at which a fitted model predicts 50% agent success. METR fits a logistic curve from agent success to human task duration; for most tasks, the human duration is the geometric mean of successful attempts by contracted professionals given similar instructions and affordances. The current suite contains more than one hundred self-contained, automatically graded tasks drawn from RE-Bench, HCAST, and shorter software tasks, primarily in software engineering, machine learning, and cybersecurity. METR warns that current estimates above 16 hours are unreliable ([METR — Task-Completion Time Horizons](#)).

This metric is **not** the amount of time an agent can run autonomously, nor a claim that it can do every task below that duration. The task distribution is comparatively clean and low-context, while longer real projects often require collaboration, tacit knowledge, and interaction with humans. A January 22, 2026 methodological note continued to characterize the 2019–2025 long-run trend as roughly one doubling every 6–7 months, but also warned that the task distribution is not sharply defined and that extrapolation to month- or year-scale work is fragile ([METR — Clarifying Limitations of Time Horizon](#)).

The practical lesson is methodological: record the date, task distribution, harness configuration, success threshold, and fitting method whenever citing a horizon number. Do not claim that a new handoff mechanism “extends the METR horizon” unless the complete system is re-evaluated on an appropriate task distribution.

12.2 Separate Semantic Continuity from Durable Execution

A long-running system needs two complementary layers:

Layer	Question it answers	Owned here?
Durable execution	Which action happened, what result was recorded, and where may execution safely resume without duplicating effects?	No — Chapter 10
Horizon extension	What has been achieved, what evidence supports it, what remains risky, and what should the next context do?	Yes

The distinction prevents two common category errors:

1. A progress document is a **projection**, not a complete causal event history. It may be stale, incomplete, or overwritten.
2. An event log is not automatically a good handoff. A successor should not have to reread thousands of low-level events to discover the current objective and next action.

The durable history makes resumption safe. The handoff makes resumption intelligible. A production system usually needs both.

12.3 The Initializer Establishes a Resumable World

An **initializer** is a first-run role, not necessarily a separate model. It turns the request and environment into a work system that later contexts can inspect and operate. In Anthropic's long-running coding experiment, an initializer created a startup script, progress file, initial commit, and comprehensive feature list; later coding sessions made incremental progress and left artifacts for the next context ([Anthropic — Effective Harnesses for Long-Running Agents](#)).

A general initializer should establish:

- **Target outcome and non-goals:** what observable state would satisfy the request, and what is outside scope.
- **Acceptance map:** milestones, dependencies, and verification methods.
- **Reproducible entry path:** commands or procedures to inspect, run, and test the current environment.
- **Baseline evidence:** the initial state of relevant tests, services, data, or external resources.
- **Progress state:** a versioned projection of milestone status, decisions, risks, and next work.
- **Artifact namespace:** stable locations or IDs for code, reports, datasets, screenshots, logs, and other work products.
- **Policy and permission envelope:** applicable restrictions, granted capabilities, approval state, and where authoritative policy is defined.

For software work, repository-local, versioned artifacts are especially effective because a successor can discover and verify them. OpenAI reports using a short repository map, structured documentation, versioned execution plans, progress and decision logs, linters, and documentation-gardening jobs rather than a single large instruction file ([OpenAI — Harness Engineering](#)). This is an implementation pattern, not a requirement that every domain use Git: the invariant is that the authoritative artifacts are addressable, versioned where necessary, and accessible to the next worker.

The initializer should also fail explicitly. If it cannot establish a runnable baseline, resolve the target outcome, or obtain required permissions, it should record a blocker rather than manufacture a confident plan on top of unknown state.

12.4 Milestones Turn a Large Goal into Bounded Work

A milestone is a **verifiable intermediate outcome**, not merely an activity label such as “work on the backend.” A useful milestone record contains:

Field	Purpose
milestone_id	Stable identity across contexts
outcome	Observable state that should exist
dependencies	Required prior state and artifacts
verification	Checks that can falsify completion
evidence	Results and artifact pointers from executed checks
status	not_started , in_progress , blocked , or verified
risks	Known uncertainty, debt, or incomplete coverage

Anthropic's experiment used a JSON feature list and asked each coding session to work on one feature at a time. The JSON choice, the more-than-200-feature example, and the one-feature slice were observations from that particular coding setup, not universal constants ([Anthropic — Effective Harnesses for Long-Running Agents](#)). Choose the slice size empirically: small enough to finish, verify, and hand off cleanly; large enough to produce meaningful progress.

Milestone state should be monotonic only when the evidence supports it. A previously verified item can return to `in_progress` or `blocked` if dependencies change or a regression invalidates its evidence. A checked box is a navigation aid, not proof.

12.5 Make Every Handoff an Artifact Contract

A **handoff** is the semantic package that lets a successor continue without guessing. Whether the successor is the same model after a reset, a different model, a human, or another worker, every handoff must include:

1. **Completed work** — specific changes and milestone transitions, not “made progress.”
2. **Verified evidence** — checks actually run, results, timestamps where relevant, and the scope those checks cover.

3. **Open risks** — failures, uncertainty, untested paths, stale assumptions, and blockers.
4. **Next action** — one bounded, executable step plus its expected result.
5. **Artifact pointers** — stable paths or IDs with version, commit, or content identity where drift matters.
6. **Permission context** — active grants, denials, pending approvals, expiry or scope, and any operation that must be re-authorized.

One possible schema is:

```

handoff_version: 1
objective: "observable target state"
completed_work:
  - milestone_id: M-03
    change: "what changed"
verified_evidence:
  - check: "command, query, or inspection"
    result: "pass/fail plus relevant measurement"
    artifact: "artifact://run/.../evidence/..."
open_risks:
  - "known uncertainty or blocker"
next_action:
  step: "one bounded action"
  expected_result: "observable result"
artifact_pointers:
  - uri: "artifact://project/..."
    version: "commit, digest, or revision"
permission_context:
  grants: ["scoped capabilities"]
  denials: ["known restrictions"]
  pending_approvals: []

```

Do not copy secrets into the handoff. Record the capability reference, scope, or approval ID; let the runtime reattach credentials at the enforcement boundary described in [Chapter 7](#).

The handoff must be persisted before the old context is discarded. Its update and the milestone transition should use the consistency mechanism defined by the runtime—ideally an atomic commit or a version check—so two workers cannot silently publish incompatible successors. The causal record of that commit remains in [Chapter 10's](#) event history.

12.6 Choose Context Transitions Deliberately

[Chapter 5](#) distinguishes context, compaction, memory, and handoff. For long-running work, the harness must choose an explicit transition rather than treating all context pressure as the same problem:

Transition	Preserves	Best used when	Main risk
Continue	Current visible context	Relevant evidence still fits and remains coherent	Accumulated noise and stale assumptions
Compact	A summarized representation inside the ongoing context	Continuity is valuable and a faithful summary fits	Omission or distortion
Reset + handoff	Selected semantic state in a fresh context	A clean reasoning surface is worth reconstruction cost	Missing handoff state
Worker handoff	Artifacts and execution state across workers	Ownership, specialization, or runtime placement changes	Version and permission mismatch
Durable recovery	Recorded results and execution position	Failure, cancellation, deployment, or lease loss	Defined by Chapter 10, not by prompt text

Anthropic explicitly distinguishes compaction from a context reset: compaction summarizes history in place, while reset starts a fresh context and depends on a structured handoff. In one model-specific experiment, reset was important for Sonnet 4.5's observed “context anxiety,” while Opus 4.5 allowed the reset mechanism to be removed.

The same follow-up later removed sprint decomposition for Opus 4.6 ([Anthropic — Harness Design for Long-Running Application Development](#)). These are evidence that scaffolding should be tested per model and task—not proof that reset, compaction, or sprints are always required.

A context reset must not implicitly reset the durable run, sandbox, permissions, or external resources. Those lifecycles have separate identities. Conversely, replacing a sandbox after compromise or corruption should not be described as a mere context reset.

12.7 Resume by Revalidating, Not by Trusting the Note

The successor should treat the handoff as a high-value index whose claims still require reconciliation with authoritative state.

A resume protocol should:

1. Confirm the objective, run identity, configuration versions, artifact revisions, and permission context.
2. Read the latest committed handoff and progress state.
3. Inspect the referenced artifacts rather than relying on pasted summaries.
4. Reproduce the smallest useful baseline: start the service, query the external state, open the document, or run a focused check.
5. Compare the observed state with completed milestones and evidence. Reopen items whose evidence is stale or contradicted.
6. Select one bounded next action and state its expected observable result.
7. After acting, update evidence, progress state, risks, and the next handoff.

Anthropic's coding sessions similarly began by inspecting the working directory, Git history, progress file, feature list, and a basic end-to-end baseline before implementing the next feature ([Anthropic — Effective Harnesses for Long-Running Agents](#)). The general rule is broader than coding: **resume from artifacts, but verify against the environment**.

If the handoff points to missing artifacts, a different version, expired authorization, or a failed baseline, the successor should enter reconciliation or blocked state. It should not continue from an imagined state merely to preserve momentum.

12.8 Keep Self-Verification Action-Local

Self-verification is immediate feedback inside the action loop. Examples include running a focused test after an edit, opening a generated file, exercising a browser path, querying a database after a write, or comparing an output with a schema. Anthropic's long-running coding experiment found that explicit end-to-end browser testing caught failures that code inspection and narrower checks missed ([Anthropic — Effective Harnesses for Long-Running Agents](#)).

This makes self-verification valuable, but it does not make the acting agent the final authority on global completion. Local checks may have incomplete coverage, the agent may misread results, and subjective quality may require calibrated judgment.

An **independent evaluator** is conditional, not mandatory for every milestone. Anthropic's 2026 follow-up found separate evaluation especially useful for subjective work and tasks near the generator's capability boundary, but found it could become unnecessary overhead for work the newer model handled reliably; the updated harness moved from per-sprint evaluation to a final pass as the model and harness changed ([Anthropic — Harness Design for Long-Running Application Development](#)).

Use the evaluation design in [Chapter 11](#) and the loop controls in [Chapter 13](#) to decide when to add an evaluator. Relevant conditions include high impact, subjective or adversarial criteria, weak deterministic checks, behavior near the model's reliability boundary, and evidence that evaluator feedback changes outcomes enough to justify latency and cost. Independence is one design lever, not a substitute for calibrated graders and environment-grounded evidence.

12.9 Finalize on Outcome and Environment State

The agent's statement “done” is a transcript event. It is not the outcome. Anthropic's evaluation terminology makes the distinction concrete: a flight-booking agent may say that a reservation was made, while the outcome is whether the reservation actually exists in the environment database ([Anthropic — Demystifying Evals for AI Agents](#)).

Finalization should therefore evaluate a task-specific success predicate over authoritative artifacts and environment state:

```
finalize only if
  required_outcomes_hold(environment, artifacts)
  and required_evidence_is_current
  and no_blocking_risk_is_open
  and required_approvals_are_satisfied
```

Examples include a database row with the correct fields, a deployed endpoint passing a health and behavior check, a file with the required content and format, or a submitted external transaction with a verifiable receipt. The acceptance map decides which outcomes are necessary; not every task needs the same checks.

A checklist can help the agent navigate, and a progress file can summarize prior work, but neither may be the sole completion oracle. If the outcome cannot be verified, report `unverified` or `blocked` with the missing evidence and next action—not `complete` based on confidence.

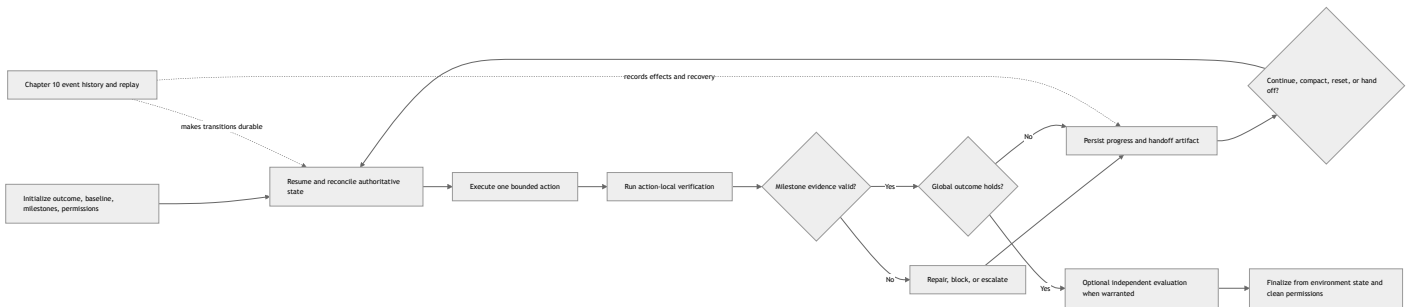
Finalization also emits a terminal artifact: achieved outcomes, verification evidence, residual risks, artifact pointers, and permission cleanup or transfer. Cleanup should revoke temporary capabilities and identify resources that remain live; it must not erase the durable evidence required for audit or recovery.

12.10 Failure Modes and Controls

Failure mode	Observable symptom	Harness control
One-shot overreach	Large partial change, no clean stopping point	Bounded milestones and slice budget
Premature completion	Some visible progress is mistaken for the whole outcome	Acceptance map plus environment-based finalization
Stale progress state	Handoff disagrees with artifacts	Versioned pointers and resume-time reconciliation
Evidence laundering	“Tests passed” without command, result, or scope	Structured evidence records
Half-finished boundary	Broken baseline at context exit	Boundary health check and explicit blocker state
Permission amnesia	Successor repeats a denied action or assumes a grant	Mandatory permission context and revalidation
Artifact loss	Summary references inaccessible work	Addressable artifact store and retention policy
Reset cargo cult	Frequent resets add cost without improving outcomes	Model/task ablation and reset telemetry
Self-certified finish	Agent prose or checklist is the only proof	Outcome predicate over authoritative state
Infinite local repair	Repeated verification-fix loop makes no net progress	Loop budgets and escalation from Chapter 13

The maintenance burden also grows over long projects. OpenAI reports encoding repository “golden principles” into mechanical rules and recurring cleanup tasks because agents reproduce existing patterns, including bad ones (OpenAI — Harness Engineering). The general control is to make quality constraints executable and to schedule maintenance as explicit work, rather than expecting each successor to rediscover architectural intent.

12.11 Reference Lifecycle



The diagram's loop is semantic. Crash recovery, replay, idempotency, and concurrent ownership remain runtime responsibilities from Chapter 10.

12.12 Design Checklist

Before enabling multi-context execution, verify that:

- The target outcome and non-goals are observable.
 - The initializer can establish or explicitly fail the baseline.
 - Milestones have stable IDs, dependencies, verification, and evidence fields.
 - Every handoff contains completed work, verified evidence, open risks, next action, artifact pointers, and permission context.
 - Artifact pointers are stable and versioned where drift matters.
 - Resume reconciles the note with authoritative artifacts and environment state.
 - Context reset, sandbox replacement, and durable recovery are separate operations.
 - Self-verification stays action-local; evaluator use is justified by task risk and measured lift.
 - Global completion is decided by outcome and environment state, not agent prose or a checklist.
 - The Chapter 10 runtime can persist, replay, deduplicate, and recover the underlying execution.
 - Finalization records evidence and cleans up temporary permissions and resources.
-

Key Takeaways

- **Long-running design is horizon extension, not transcript extension.**
- **Chapter 10 supplies durable recovery; Chapter 12 supplies semantic continuity.**
- **The initializer creates a resumable world with baselines, milestones, artifacts, progress state, and permissions.**
- **Every handoff needs six fields:** completed work, verified evidence, open risks, next action, artifact pointers, and permission context.
- **Reset and compaction are model- and task-dependent choices.** Test them instead of hard-coding historical scaffolding.
- **Self-verification provides local feedback.** Independent evaluation is conditional and does not replace environment-grounded evidence.
- **Only the outcome can finish the task.** Agent self-report and checklists are useful projections, never sufficient completion proof.
- **Time-horizon numbers require measurement context.** Include date, task distribution, reliability threshold, harness setup, and method.

Further Reading

- Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
- Anthropic, *Demystifying Evals for AI Agents*, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- METR, *Task-Completion Time Horizons of Frontier AI Models*, updated May 8, 2026. <https://metr.org/time-horizons/>
- Joel Becker, *Clarifying Limitations of Time Horizon*, METR, Jan 22, 2026. <https://metr.org/notes/2026-01-22-time-horizon-limitations/>

Chapter 13: Loop Engineering and Verifier Hierarchies

Agent = Model + Harness. Chapter 1 introduced the inner agent loop: the harness assembles context, the model proposes an action, the runtime executes it, the environment returns an observation, and the cycle repeats. This chapter treats the control around that cycle as an engineering object. It specifies what starts a run, what one iteration may do, what evidence counts as progress or success, and why the run stops.

The phrase *loop engineering* is useful practitioner vocabulary, but it is not a formal standard or a reference architecture. Addy Osmani uses it for systems that find work, dispatch it, check it, persist state, and decide what happens next ([Addy Osmani — Loop Engineering](#)). This chapter keeps that operational insight while defining the loop in the book's existing harness, runtime, policy, and evaluation terms.

13.1 Research Roots and Practitioner Lineage

The underlying action loop predates the label. ReAct interleaves reasoning traces with task-specific actions so that actions obtain information from an external environment and observations can update the plan ([Yao et al. — ReAct](#)). A production harness need not expose private reasoning traces, but it does need the observable control cycle: propose action → enforce policy → execute → record observation → update execution state.

Two later practitioner labels describe particular ways of operating that cycle:

- **Ralph** describes a repository-oriented practice in which a single process performs one task per loop and reloads specifications and a plan on every iteration ([Geoffrey Huntley — Ralph Wiggum as a “software engineer”](#)).
- **Loop engineering** emphasizes triggers, isolated workspaces, reusable instructions, connectors, sub-agents, and state outside one conversation ([Addy Osmani — Loop Engineering](#)).

These are practitioner lineages, not conformance targets. Neither establishes that every agent must reset context each iteration, use multiple agents, run on a schedule, or progress toward unattended autonomy. Treat each pattern as a design option whose value must be demonstrated for the task.

13.2 Start With a Loop Contract

A **loop contract** is the harness configuration that makes a repeated run bounded and testable. Before enabling repetition, define:

- **Trigger**: a human request, event, schedule, or parent run, including deduplication semantics.
- **Goal and acceptance contract**: the requested outcome, allowed scope, constraints, and evidence required for success.
- **Execution state**: the current task, attempt, budget use, policy state, artifacts, and latest accepted checkpoint.
- **Iteration boundary**: the maximum unit of work one pass may attempt and commit.
- **Action envelope**: available tools, identities, resources, side-effect classes, and mandatory approval gates.
- **Verifier plan**: the checks to run, their order, thresholds, abstention behavior, and escalation target.
- **Progress function**: the measurements that distinguish useful change from motion without progress.
- **Stop and suspend rules**: terminal reasons, retry rules, and resumable waiting states.

This contract belongs to the harness and control plane, not to the model's prose. The model may propose that the goal is complete; the harness decides whether the configured evidence supports `success`. Anthropic's production-pattern guidance similarly describes agents as using environmental feedback in a loop, pausing for human input at checkpoints or blockers, and using stopping conditions such as iteration caps to retain control ([Anthropic — Building effective agents](#)).

One bounded iteration can be expressed as:

```
load execution state
→ select one eligible unit of work
→ authorize and execute bounded actions
→ collect artifacts and environment observations
→ run the configured verifier plan
→ checkpoint accepted progress
→ continue, suspend, or terminate with a reason
```

Iteration count alone is not progress. A useful progress function is task-specific: failing tests decrease, required fields become valid, a target environment changes to the expected state, evidence coverage increases, or a human resolves a named ambiguity. Persist the measurements and artifact identities so that a later run can tell whether anything actually changed.

13.3 A Default Verifier Hierarchy

Use the lowest-cost verifier that can supply evidence appropriate to the consequence, then add higher levels when the lower ones leave material uncertainty. The default escalation hierarchy is:

Level	Verifier	Best evidence	Main limitation
1	Schema checks	Required fields, types, ranges, and protocol shape	Valid shape does not imply a correct outcome
2	Deterministic tests and linters	Reproducible behavioral assertions, invariants, static rules	Tests can be incomplete or coupled to an implementation
3	Environment outcome checks	The external system reached the intended state	A good end state may hide an unsafe or disallowed path
4	Same-agent critique	Cheap, context-rich detection and repair of obvious defects	The critique can share the maker's assumptions and blind spots
5	Independent model grader	Rubric-based judgment for open-ended or nuanced properties	Independence does not remove model error or correlated failure
6	Human review	Intent, accountability, expert judgment, and ambiguity resolution	Slow, costly, and not automatically consistent

This is an escalation hierarchy, not a claim that every task should use all six levels or that a later level dominates every earlier one. A human should not replace a checksum, and an independent model should not replace an executable test. Agent evaluations commonly combine code-based, model-based, and human graders; code checks are fast and reproducible, model graders capture nuance but require human calibration, and human grading is slower and more expensive ([Anthropic — Demystifying evals for AI agents](#)).

The old slogan “the maker must not be the checker” is too absolute. Same-agent critique can be an efficient repair step when consequences are low and defects are locally visible. Conversely, a nominally independent checker may share the same base model, training distribution, prompt assumptions, tools, or incomplete rubric. Separation is useful only when its measured reduction in error justifies its cost and when correlated failure is understood.

13.4 Choose the Verifier From Evidence, Not a Slogan

Chapter 11 defines the verifier profile used here: failure classes, false-positive and false-negative rates, abstention, disagreement, coverage, latency, cost, and correlated failure. Choose and compose levels using five task properties:

1. **Consequence.** Higher-impact or difficult-to-reverse actions require stronger evidence, tighter false-accept targets, and often a policy gate or accountable human review.
2. **Task ambiguity.** Precise outputs favor schema and deterministic checks. Open-ended quality or intent may require a calibrated model rubric, expert review, or explicit abstention.
3. **Capability boundary.** Use evaluation results—not intuition—to ask whether the maker reliably handles this task class. Near the measured boundary, additional critique or an evaluator can add value. Far inside it, that evaluator may only add latency and cost. Anthropic reports this exact task-relative effect in a long-running coding harness: as model capability improved, an evaluator became overhead for work the generator handled reliably but remained useful at the edge of capability ([Anthropic — Harness design for long-running application development](#)).
4. **Latency and cost.** Run cheap, high-signal checks first and reserve slower graders for unresolved uncertainty. Track verifier cost and latency separately from maker cost and latency.
5. **Correlated failure.** Estimate whether maker and checker fail on the same examples. Different roles, prompts, or models do not by themselves prove useful independence; use Chapter 11's joint-error and disagreement measurements.

A practical policy might be: schema-check every tool result; run tests and linters for each candidate artifact; check the real environment before claiming an external outcome; use same-agent critique for low-cost repair; invoke an independent grader only for residual rubric-based properties; require human review when consequence or unresolved ambiguity exceeds a threshold.

For every verifier, define at least three results rather than forcing a binary answer: `pass`, `fail`, and `abstain`; a versioned contract may also define `partial`. An abstention must route to another verifier or to human resolution; it must never silently become a pass. Calibrate model graders against expert human judgment, as recommended for subjective research and other open-ended evaluations ([Anthropic — Demystifying evals for AI agents](#)).

13.5 Keep Live Verification Separate From Release Evaluation

A **live verifier** influences the trajectory: it selects retries, provides feedback, accepts a checkpoint, or stops the loop. A **holdout grader** measures the completed system without steering that trajectory. Reusing a release grader's hidden answers or exact rubric as iteration feedback creates a path for evaluator leakage and optimization to the check rather than the underlying task.

Therefore:

- expose only the acceptance evidence required for productive repair;
- keep holdout cases and release thresholds outside the maker's writable state;
- version every schema, test suite, rubric, grader model, and human-review protocol;
- record which verifier result caused each transition;
- periodically compare automated verifier decisions with blinded human labels;
- investigate disagreement and joint error, not only aggregate pass rate.

Outcome checks and trajectory checks answer different questions. The first asks whether the intended state was reached; the second asks whether the route respected policy, budgets, and tool constraints. Anthropic's evaluation guidance explicitly distinguishes graders over outcomes from graders over transcripts and gives examples that combine state checks, required tool calls, turn limits, and quality rubrics ([Anthropic — Demystifying evals for AI agents](#)). A loop needs both when a correct outcome could be reached through an unacceptable path.

13.6 Stop, Suspend, and Resume Are Different States

Every run must terminate with a machine-readable `stop_reason`, or enter a named nonterminal suspension. The minimum terminal set is:

Stop reason	Required condition	Retry semantics
<code>success</code>	The configured acceptance contract passed; self-declaration is insufficient	Start a new run only for a new goal or invalidated evidence
<code>budget_exhausted</code>	Any configured token, time, money, action, or iteration ceiling is reached	Resume only with a newly authorized budget
<code>no_progress</code>	The progress function stays below its threshold for the configured window	Resume after changing evidence, plan, tools, or task framing
<code>repeated_failure</code>	The same normalized failure signature recurs beyond its retry allowance	Resume after changing the suspected cause, not by resetting the counter
<code>policy_denied</code>	The policy decision point denies the proposed action and no allowed alternative remains	A model retry cannot override the decision; change policy or scope through an authorized path
<code>human_cancelled</code>	An authorized human cancels the run	Do not auto-resume; require a new explicit request
<code>environment_terminal</code>	The environment reports an absorbing state such as deleted, expired, closed, or irrecoverably failed	Start a new run only if the environment and task contract permit it

`approval_wait`, missing input, rate-limit backoff, and a temporarily unavailable dependency are normally **suspensions**, not successful or failed terminal states. Persist a checkpoint, the outstanding request, its expiry, and the identity allowed to resolve it. Resume from recorded execution state rather than asking the model to reconstruct the wait from conversation text.

Policy enforcement must remain outside the model loop. NIST defines a policy decision point as the component that computes access decisions and a policy enforcement point as the component that enforces those decisions for protected resources ([NIST — Zero Trust Architecture Glossary](#)). In this book's architecture, `policy_denied` is therefore a harness/runtime outcome, not an objection the model may talk past.

Precedence must also be explicit. For example, `human_cancelled`, `policy_denied`, and `environment_terminal` should be checked before scheduling another retry; `success` requires current evidence; and budget consumption must be committed atomically enough that concurrent workers cannot each spend the same remaining allowance.

13.7 Nested Loops Need Ownership Boundaries

A useful topology separates a fast inner action loop from slower outer control loops:

- the **action loop** executes a bounded unit and consumes immediate environment feedback;
- the **repair loop** uses verifier failures to select another attempt;
- the **task loop** chooses the next unit of work and checks goal-level completion;
- the **release loop** applies holdout evaluation, policy, approval, or deployment controls;
- the **product loop** uses real-world outcomes to revise goals and acceptance contracts.

Do not let an inner loop redefine the success criteria of its parent. A worker may propose a revised plan, but the owner of the parent contract must accept it. Likewise, a child run may return an artifact and evidence; the parent remains responsible for deciding whether that evidence satisfies its own verifier plan.

Evaluator–optimizer loops fit when evaluation criteria are clear and iterative refinement produces measurable value; they are not a default wrapper for every model call ([Anthropic — Building effective agents](#)). Each nested loop should have its own budget, state owner, stop reason, and escalation route so that “retry” cannot expand into an unbounded tree of delegated work.

13.8 Context Reset Is One Recovery Strategy

The Ralph practice deliberately performs one repository task per loop and reloads plan and specification artifacts each time ([Geoffrey Huntley — Ralph Wiggum as a “software engineer”](#)). That can reduce contamination from a long interaction and makes durable artifacts important. It does not prove the stronger claim that memory “lives on disk” or that every iteration should discard context.

Choose context policy from the failure mode:

- **continue the same context** when recent observations and unresolved local state remain useful;
- **compact** when the relevant trajectory can be summarized with traceable references;
- **reset and rehydrate** when stale assumptions or context pressure are causing repeated failure;
- **handoff to another run** when capability, authority, or responsibility changes.

Persist canonical execution state, artifacts, checkpoints, and event history in harness-managed storage. A repository file can be one artifact, but it is not automatically the source of truth for identity, budget, approval, or policy state. Anthropic's long-running harness report describes structured artifacts for handoffs between sessions and also shows that scaffolding should be removed when stronger model capability makes it unnecessary overhead ([Anthropic — Harness design for long-running application development](#)).

13.9 Replace the Autonomy Ladder With a Capability–Control Profile

A single maturity ladder wrongly suggests that every product should progress from manual operation to auto-merge. Design maturity is multidimensional. Record at least these dimensions:

Dimension	Example values	Design question
Trigger autonomy	manual · event · schedule · parent run	Who may start work, and how is duplication prevented?
Action scope	read-only · draft · reversible write · irreversible effect	What can one iteration change?
Verifier strength	hierarchy levels and measured profile	What evidence justifies continuation or success?
Runtime durability	ephemeral · checkpointed · replayable	What survives interruption?
Policy and approval	allowlist · dynamic policy · mandatory gate	Which decisions are outside model authority?
Observability	final artifact · trace · event history · lineage	Can operators reconstruct the decision path?
Recovery	retry · compensate · rollback · human repair	How is partial failure contained?
Human responsibility	operator · reviewer · approver · accountable owner	Where does judgment and accountability remain?

A system can be highly mature in durability and observability while intentionally remaining read-only and manually triggered. Another can schedule low-consequence reversible actions but require human approval for release. The target profile follows product value, failure consequence, and evidence—not a universal push toward maximum autonomy. Anthropic likewise recommends adding agentic complexity only when it demonstrably improves outcomes, noting the latency, cost, and compounding-error tradeoffs ([Anthropic — Building effective agents](#)).

13.10 Operate the Loop as a Measured Control System

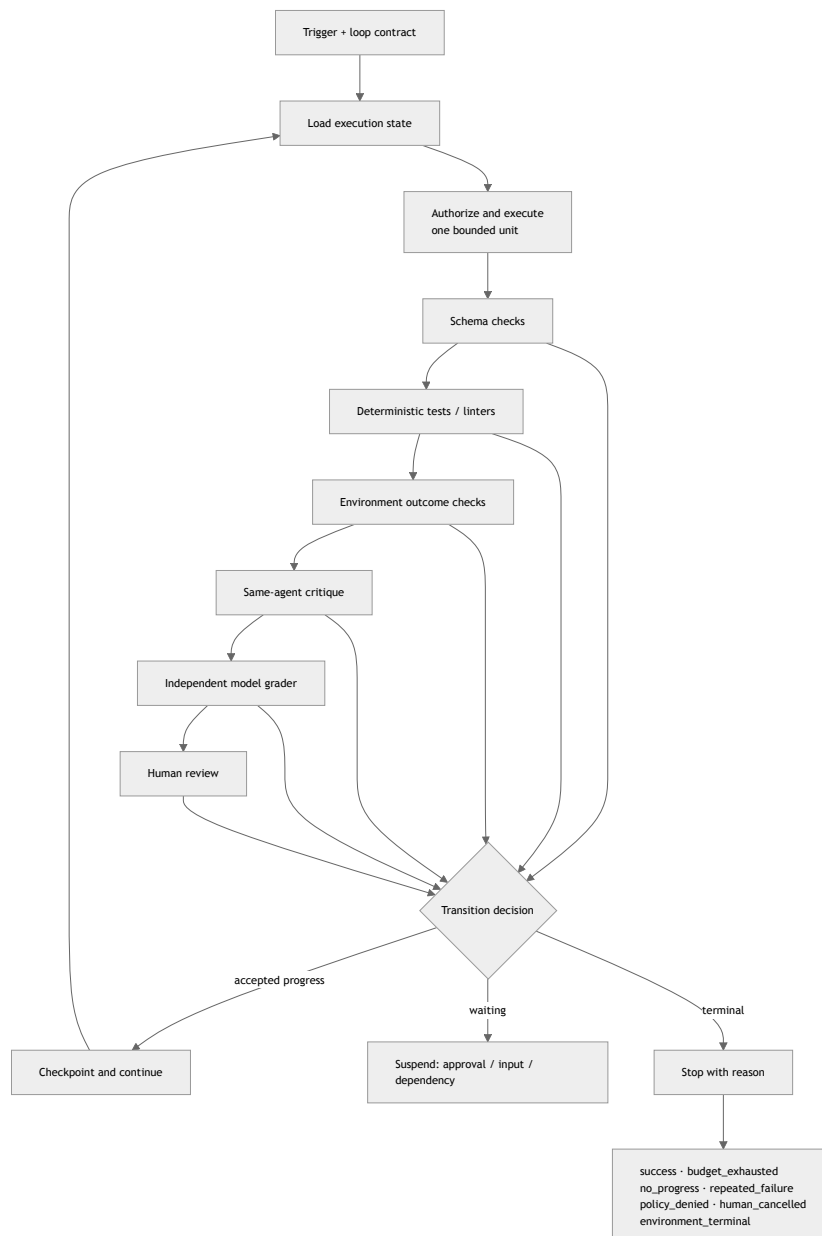
For each run, emit a loop control record containing the contract version, trigger identity, iteration and retry counts, consumed budgets, verifier versions and results, progress measurements, artifacts, policy and approval events, final checkpoint, and `stop_reason`. Chapter 10 supplies durable execution semantics; Chapter 11 supplies verifier calibration; Chapter 12 supplies local self-verification patterns; Chapter 14 supplies human review and approval semantics; Chapters 17–19 connect traces, budgets, policy enforcement, lineage, and fleet operations.

Use those records to answer operational questions:

- Which failure signatures consume the most retries?
- Which verifier levels change decisions rather than merely add cost?
- Where do maker and checker fail together?
- How often does `success` survive holdout and human review?
- Which tasks stop for budget, no progress, denial, cancellation, or terminal environment state?
- Does a context reset improve recovery for the targeted failure class?

Only expand the loop's action scope, trigger autonomy, or retry allowance when the evidence supports that particular change. A loop is engineered when continuation is justified by measured progress and termination is justified by an auditable reason.

Diagram: A Bounded Loop With Escalating Evidence



Key Takeaways

- **“Loop engineering” and “Ralph” are practitioner lineages, not standards.** Use their patterns selectively and measure the result.
- **A loop contract makes repetition bounded.** It defines trigger, state, action envelope, evidence, progress, budgets, and stop or suspend semantics.
- **Use a verifier hierarchy.** Escalate from schema checks through deterministic and environment evidence to critique, independent model grading, and human review only as uncertainty and consequence require.
- **The maker may critique its own work.** Independent checking is valuable when its calibrated error profile and reduced correlated failure justify the cost; it is not an unconditional rule.
- **Every run needs explicit terminal reasons.** Cover success, exhausted budget, no progress, repeated failure, policy denial, human cancellation, and terminal environment state.
- **Waiting is not stopping.** Approval, missing input, and transient dependency waits are durable suspensions with explicit resume authority.
- **There is no universal autonomy ladder.** Mature systems choose a capability–control profile appropriate to their task and risk.

Further Reading

- Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*. <https://arxiv.org/abs/2210.03629>
- Anthropic, *Building effective agents*. <https://www.anthropic.com/engineering/building-effective-agents>
- Anthropic, *Harness design for long-running application development*.
<https://www.anthropic.com/engineering/harness-design-long-running-apps>
- Anthropic, *Demystifying evals for AI agents*. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Geoffrey Huntley, *Ralph Wiggum as a “software engineer”*. <https://ghuntley.com/ralph/>
- Addy Osmani, *Loop Engineering*. <https://addyosmani.com/blog/loop-engineering/>
- NIST, *Zero Trust Architecture Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>

Chapter 14: Human–Agent Interaction

Humans participate in agent systems in several roles: they supply intent, answer questions, approve or deny consequential actions, steer work in progress, cancel runs, and review evidence. These roles need product and runtime support. They cannot be implemented safely as a vague instruction telling the model to “ask when appropriate.”

Mixed-initiative interface research framed the underlying design problem before modern agents: a system acting on a person's behalf must decide when to act, when to defer, how to account for interruption cost, and how to preserve interaction context ([Horvitz — Principles of Mixed-Initiative User Interfaces](#)). Agent systems add external side effects and long-running state, so the interface must also make authority, evidence, timing, and recovery explicit.

14.1 Consultation and Mandatory Approval Are Different Paths

Two human-interaction patterns may look similar in a chat UI but have different security semantics:

Pattern	Trigger	Meaning	Enforcement owner
Agent-requested consultation	The model or workflow proposes a question because intent, facts, or judgment are unclear	Human input becomes an observation that may guide the next model call	Harness/runtime records the question and response; the response does not automatically grant an unrelated capability
Runtime-mandated approval gate	Policy classifies a proposed action as requiring approval before dispatch	An authenticated, scoped decision permits or denies that exact action under stated conditions	A non-bypassable policy enforcement point (PEP) on the action path

HumanLayer's “contact humans with tool calls” is a useful orchestration pattern for the first path: represent a question as structured work, suspend, and return the answer to the loop as an observation ([HumanLayer — 12-Factor Agents](#)). The model can request clarification or recommend that someone review a choice. It may also fail to ask.

The second path therefore cannot depend on a voluntary model call. NIST defines a PEP as the component that enforces access decisions, while its access-control guidance requires enforcement mechanisms not to be bypassable ([NIST — Zero Trust Architecture Glossary](#); [NIST IR 7987](#)). The MCP tools specification likewise recommends that applications show tool inputs and request confirmation for sensitive operations ([MCP — Tools Specification](#)). The runtime must intercept the action even if the model never emits `request_human_approval`.

An answer to a consultation can cause policy to re-evaluate an action, but it is not itself the mandatory approval unless it was collected through the authenticated gate with the required scope. Conversely, the runtime may require approval for an action the agent believes is routine.

14.2 Scale Human Attention with the Stakes

Approval on every action creates habituation and slows useful work; too little review encourages blind trust. Anthropic describes reducing repeated permission prompts by placing file and network operations inside explicit sandbox boundaries, while keeping actions beyond those boundaries subject to confirmation ([Anthropic — Beyond Permission Prompts](#)). The general rule is to spend human attention where the policy-relevant consequences justify it.

Classify an action along at least these dimensions:

- **Reversibility:** can the exact prior state be restored, and has rollback been tested?
- **Blast radius:** one scratch file, one user, one tenant, a production service, or a public audience?
- **External communication:** will the action send, publish, submit, or share something outside the working boundary?
- **Money:** can it create a purchase, transfer, subscription, contract, or metered spend?
- **Production:** can it change live infrastructure, data, access, or availability?
- **Sensitive data:** can it read, transform, expose, retain, or transmit secrets, personal data, regulated data, or tenant-confidential content?

A starting matrix is:

Action profile	Example	Default interaction	Required evidence
Reversible, sandboxed, narrow blast radius	Read allowed files; run tests; edit a disposable workspace	Execute under existing authorization and report as needed	Action/result record; sandbox boundary
Reversible and scoped, but changes durable work	Create a draft or branch; update a non-production artifact	Execute and report, or sample for review according to policy	Diff or versioned artifact; rollback path; checks run
External communication	Send email; post a comment; submit a form	Preview exact audience and payload; normally require approval unless a narrow standing policy covers it	Recipient/destination, normalized payload, identity, policy reason
Money or binding commitment	Purchase, transfer, paid API expansion, contract acceptance	Mandatory approval with amount, recipient/vendor, limits, and expiry	Exact transaction proposal; budget and authorization evidence
Production or broad blast radius	Deploy, modify access, alter live data, rotate credentials	Mandatory approval or stronger change-management gate	Diff/plan, affected resources, tests, rollback, current health
Sensitive-data disclosure or movement	Export customer data; send a secret; change sharing scope	Deny unless explicitly authorized; mandatory approval where policy permits	Data class, source, destination, purpose, minimization/redaction evidence
Irreversible or destructive	Permanent deletion; unrecoverable publish; destructive migration	Mandatory approval plus recovery or exception plan	Exact target, impact analysis, backups/recovery evidence, expiry

This is not a universal allowlist. Reversibility can be deceptive: deleting a local file may be harmless if versioned, while sending an “editable” message can still create an irreversible disclosure. Policy should combine the dimensions with identity, purpose, environment, and current resource state.

14.3 The Mandatory Gate Binds Approval to an Exact Proposal

A mandatory approval should be issued against an immutable or content-addressed action preview. At minimum, the pending record contains:

```

proposed_action
normalized_arguments
resource_identity_and_version
acting_user_agent_and_delegation_identity
policy_id_and_version
risk_class_and_policy_reason
approval_scope_and_expiry

```

The decision record adds the authenticated approver identity, approver authority scope, decision, time, conditions, and the proposal hash. The UI may render friendly labels, but the grant must bind to normalized arguments and stable resource identifiers rather than only to natural-language prose.

A **material change** invalidates the prior approval and sends the proposal through policy again. Examples include changing the amount, recipient, destination, target resource or version, data class, external audience, acting identity or delegation, tool semantics, side-effect scope, policy version, or execution time beyond the grant's expiry. Policy should define materiality for the domain; the model must not decide that two actions are “close enough.” Chapter 6 applies the same rule to retries: approval, authorization, and outcome status must be checked again when their inputs change.

Approval establishes permission to attempt the scoped action. It does not prove the tool executed, that a timeout caused no effect, or that the intended outcome occurred. Keep approval ID, dispatch attempt, and outcome evidence distinct.

14.4 Approval Events Support Recovery; Audit Requires More

Consultation and approval should enter the workflow event history through explicit transitions such as:

```
consultation_requested -> consultation_answered
approval_required -> approval_granted | approval_denied | approval_expired
approval_granted -> approval_revoked | action_dispatched
```

Each event carries the run/action identity, sequence, actor, time, proposal or artifact reference, policy version, and previous state. This lets a durable runtime release compute while waiting and later reconstruct whether it may resume. Temporal's event-history model is one concrete example in which persisted events reconstruct workflow state after failure ([Temporal — Events and Event History](#)). Chapter 10 defines this book's event and replay contract.

An event history is not automatically an audit record. NIST SP 800-53 treats audit-record content, generation, review, protection, access, and retention as explicit controls ([NIST SP 800-53 Rev. 5](#)). Approval events become audit evidence only when the system can establish required **completeness, identity, timestamp, integrity, retention, and access** properties. A sampled trace or mutable application log may link to the event, but it cannot silently inherit those guarantees.

The audit representation should preserve what was shown, who decided, which authority they held, what changed after the decision, which action was dispatched, and what outcome was confirmed. It should also minimize unnecessary prompt, personal, and secret content; auditability is not permission to retain everything forever.

14.5 Design the Review Surface Around Decisions and Evidence

A useful review surface enables a person to understand what is happening, what decision is required, and what evidence supports it. Human–AI interaction guidelines recommend making system capabilities clear, showing status, supporting correction, and making it easy to dismiss or override incorrect output ([Amershi et al. — Guidelines for Human-AI Interaction](#)).

Show, as applicable:

- **plan and current status:** intended milestones, completed work, pending work, and blockers;
- **tool action:** operation name, normalized arguments, resource, acting identity, expected side effects, and idempotency/outcome status;
- **diff or artifact:** the exact version, content hash, rendered change, destination, and rollback option;
- **test and evidence:** checks actually run, results, citations, environment state, and links to complete output;
- **uncertainty:** assumptions, conflicting evidence, missing checks, unknown outcomes, and what would resolve them;
- **policy reason:** why the action was allowed, denied, redacted, sandboxed, or sent for approval; which policy/version applied;
- **decision consequences:** what approval permits, when it expires, and which material changes will require a new decision.

This does not require exposing hidden chain-of-thought. OpenAI's Model Spec notes that some models generate hidden chain-of-thought that is not exposed to developers or users except potentially in summarized form ([OpenAI — Model Spec, Hidden Chain of Thought](#)). A review system should rely on concise plans, observable actions, artifacts, checks, sources, policy decisions, and environment outcomes—not private reasoning text or a model's post-hoc rationale.

Separate **pre-action approval** from **post-action review**. Before dispatch, emphasize the exact proposed effect and approval scope. After execution, emphasize what actually changed, verification evidence, residual uncertainty, and recovery options. A “done” message is insufficient for either surface.

14.6 Steering and Cancellation Need Event Semantics

Steering is not merely appending a message. Record a `steering_requested` event with requester identity, scope, instruction reference, reason, and time. The controller then records whether it was accepted, rejected, or superseded, and the sequence or step at which it became effective. Normally it can affect the next model call and prevent undischarged future actions; it cannot retroactively change the arguments of an action already dispatched.

Cancellation similarly has at least two states: **cancellation requested** and a terminal **cancelled, partially completed, failed, or unknown** outcome. Temporal's workflow model distinguishes a cancellation request from the resulting terminal state, and activity cancellation is cooperative rather than proof of rollback ([Temporal — Workflow Execution](#); [Temporal — Events and Event History](#)).

When a steering or cancellation event arrives:

1. stop scheduling new actions within its scope;
2. record the current model/tool call and whether dispatch already occurred;
3. request cancellation from in-flight operations when their contract supports it;
4. do not assume an already dispatched effect was rolled back;
5. reconcile external state and record succeeded, partial, cancelled, failed, or unknown outcomes;
6. invalidate pending approval if the new direction materially changes its proposal;
7. checkpoint or construct a handoff before resuming under the new instruction.

A long, non-interruptible model call may only observe steering after it returns. A remote tool may ignore cancellation or finish before the request arrives. The UI should show this timing honestly: “request received” is not “action stopped.” Chapter 6 owns cancellation and unknown-outcome handling for individual calls; Chapter 10 owns durable state transitions.

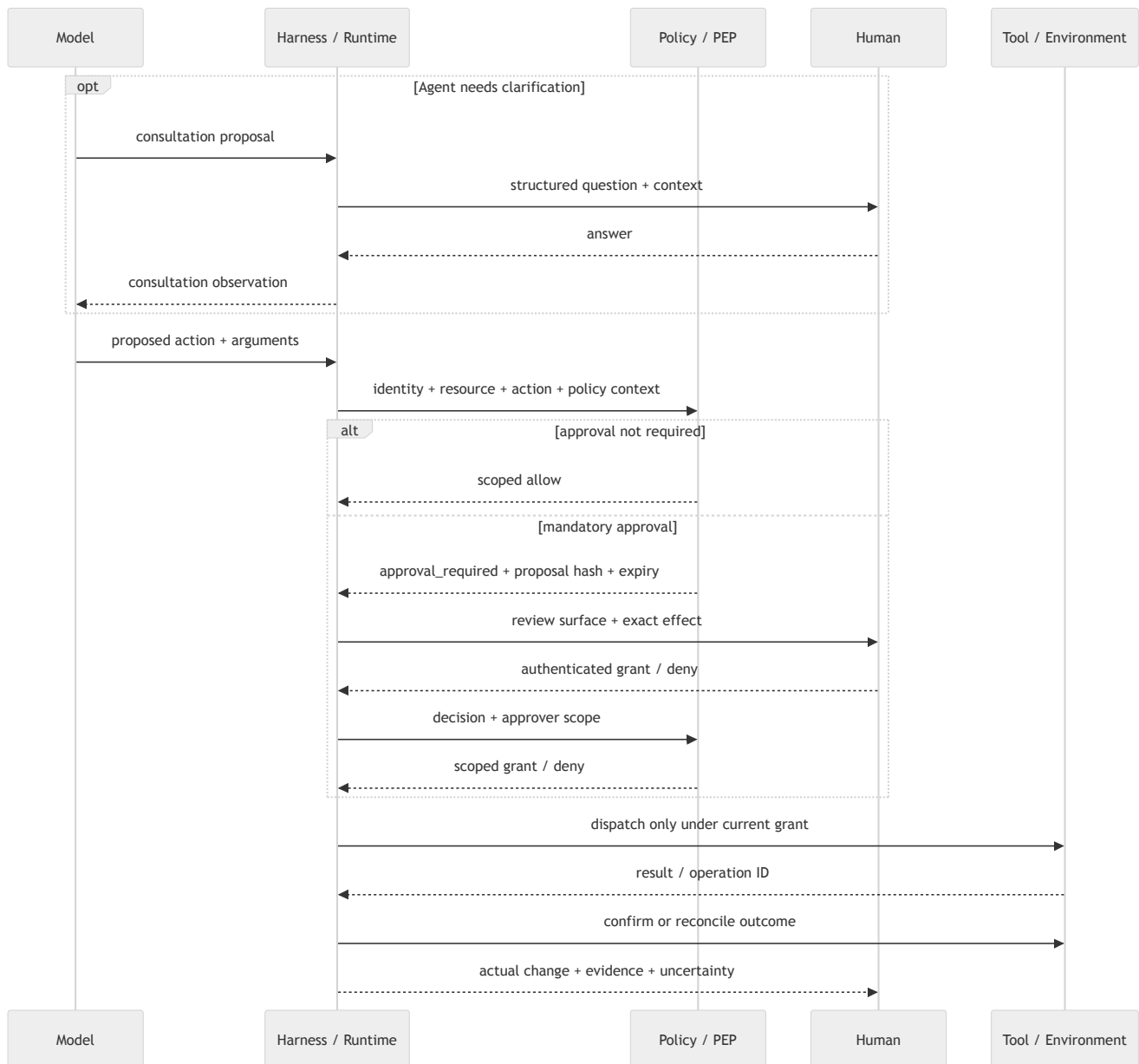
14.7 From One Review to a Fleet Queue

For one run, a reviewer can inspect the pending action directly. Across a fleet, the queue should prioritize work by **risk and evidence**, not merely arrival time or transcript length. Useful escalation signals include mandatory approvals waiting near expiry, sensitive-data or production actions, broad blast radius, external communication or spend, failed deterministic checks, missing artifacts, conflicting evidence, unknown side effects, novel tool/policy combinations, and repeated human overrides.

Low-risk work with strong outcome evidence may be summarized or sampled according to policy; consequential work should retain the exact proposal and review artifacts. Queue status must link back to the run/action IDs and current event-history state so a stale card cannot approve an action that has changed or already completed.

Cross-run identity, queue ownership, policy administration, revocation, and fleet lifecycle belong to [Chapter 19](#). This chapter supplies the human decision and review contract that the fleet control plane must preserve.

14.8 Two Human Paths Around One Action



The consultation path improves intent and judgment. The mandatory path enforces policy. They can appear in the same run, but one cannot substitute for the other.

Key Takeaways

- **Separate consultation from approval:** a model can ask a person for input, but a runtime-mandated gate must not depend on the model volunteering to call it.
- **Bind approval to the exact action:** record normalized arguments, resource, identity, policy/version, risk, scope, and expiry; material change requires a new decision.
- **Scale attention with stakes:** consider reversibility, blast radius, external communication, money, production, and sensitive data together.
- **Approval is permission, not outcome evidence:** dispatch and postcondition confirmation remain separate lifecycle stages.
- **Event history is not automatically audit:** approval events become audit evidence only with defined completeness, identity, integrity, timestamps, retention, and access.
- **Review observable work, not hidden reasoning:** show plan/status, tool actions, diffs or artifacts, tests and evidence, uncertainty, policy reasons, and decision consequences.
- **Steering and cancellation are state transitions:** record when they take effect and reconcile any action already dispatched.

- **Fleet review belongs to the control plane:** Chapter 19 owns cross-run queues, identity, policy administration, revocation, and lifecycle.

Further Reading

- Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>
- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>
- Model Context Protocol, *Tools Specification*, Jun 2025. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- NIST, *Zero Trust Architecture Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>
- NIST, *IR 7987: Policy Machine*. <https://nvlpubs.nist.gov/nistpubs/ir/2014/NIST.IR.7987.pdf>
- NIST, *SP 800-53 Rev. 5*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- Temporal, *Workflow Execution*. <https://docs.temporal.io/workflow-execution>
- Temporal, *Events and Event History*. <https://docs.temporal.io/workflow-execution/event>
- OpenAI, *Model Spec*, Oct 2025. <https://model-spec.openai.com/2025-10-27>

Chapter 15: Computer-Use and Multimodal Agents

Computer use is a specialization of the tool lifecycle, not a different abstraction. The model receives one or more observations of a user interface, proposes a structured computer action, and the application or harness decides whether and how to execute it. The resulting screenshot, structured state, error, or execution record returns as a correlated tool result/observation. OpenAI's current contract uses `computer_call` actions and `computer_call_output`; Anthropic's client-executed computer tool uses `tool_use` and `tool_result` under the same model-proposes/application-executes boundary ([OpenAI — Computer Use](#); [Anthropic — Computer Use Tool](#); [Anthropic — How Tool Use Works](#)).

The distinctive difficulty is not that an image stops being a tool result or that a click stops being a function-like proposal. It is that GUI observations are partial and perishable, grounding is geometric, actions depend on hidden interface state, and a successful input event may not create the intended environment outcome.

15.1 The Structured Computer-Tool Loop

The specialized loop extends [Chapter 6's](#) invocation lifecycle:

1. **Capture an observation bundle.** Record a screenshot and/or structured UI state with identity, time, geometry, and surface metadata.
2. **Ask the model for an action proposal.** The output is structured, such as `click(x, y)`, `type(text)`, `scroll(direction, amount)`, a key sequence, or a custom DOM/role action.
3. **Normalize and validate.** Bind the proposal to the observation it used; validate action type, arguments, coordinate transform, current application/origin, target scope, and policy.
4. **Authorize and approve.** A non-bypassable enforcement point applies allowlists, sandbox constraints, and any mandatory human gate.
5. **Execute through the application.** Browser automation, an OS input adapter, or another UI runtime performs the action—not the model.
6. **Return an execution result and fresh observation.** Correlate them to the proposal and report `succeeded`, `failed`, `partial`, or `unknown` as appropriate.
7. **Verify the environment outcome.** Decide whether the intended UI and backend postconditions hold; otherwise re-observe, recover, or stop.

OpenAI's built-in loop currently returns ordered actions in a `computer_call`, asks application code to execute them, and expects an updated screenshot in a correlated `computer_call_output`. Anthropic similarly requires application code to translate tool requests into computer actions and return screenshots or command output in `tool_result` blocks ([OpenAI — Computer Use](#); [Anthropic — Computer Use Tool](#)). Provider payloads differ, but the responsibility boundary is the same.

15.2 Keep Six Records Distinct

A robust harness must not collapse these objects into a generic “screen state”:

Object	What it represents	Minimum useful metadata	What it does not prove
Screenshot observation	Rendered pixels captured at one instant	observation_id , capture time, image digest, pixel dimensions, viewport, crop, scale/device-pixel ratio, window/tab, current URL or application identity	That the same pixels still exist at dispatch time
DOM/accessibility state	Structured elements, roles, labels, values, bounds, and relationships exposed by the UI automation surface	snapshot/version ID, frame or window, origin, extraction method, element references	Exact appearance, salience, visibility, or clickability unless explicitly checked
Coordinate grounding	A mapping from intended target or element reference into the coordinate space of a particular observation	source observation, target reference, bounding box/point, confidence or ambiguity, coordinate transform	That an input event was dispatched
Action proposal	The model's requested UI operation and arguments	proposal/call ID, source observation ID, action type, normalized arguments, expected effect	Authorization, execution, or success
Execution result	What the executor knows about dispatch and input-event processing	attempt ID, per-action status, timestamps, active surface, error, partial/unknown status	The intended product state was reached
Environment outcome	Verified state after the interaction	UI assertions, URL/origin, backend/API state, durable object IDs, external-effect evidence	That the path complied with policy unless process evidence is also checked

A screenshot can be the payload of a tool result. DOM or accessibility data can be returned in the same result or through a separate tool. A click can be encoded as a provider-defined computer action, a custom function call, or generated code ([OpenAI — Computer Use](#)). These are protocol and interface choices, not mutually exclusive categories.

Keep identity across the chain:

```

observation_id
  -> grounding_id
  -> proposal_id / provider_call_id
  -> dispatch_attempt_id
  -> execution_result_id
  -> post_action_observation_id
  -> outcome_evidence_id

```

This correlation is what lets an eval or incident review answer whether the model chose the wrong target, the coordinate mapping was wrong, the executor acted on stale state, or the application accepted the input but failed to update its backend.

15.3 Choose Observation Channels by the Decision

The harness may expose several complementary observation channels:

- **Screenshot:** preserves rendered layout, styling, overlays, and spatial relationships at capture time.
- **DOM or application tree:** exposes structured content and element identity for supported applications.
- **Accessibility state:** exposes semantic roles, labels, values, checked/expanded state, and hierarchy when the application publishes them.
- **OCR or extracted text:** provides text when structured state is unavailable, but should retain bounding boxes and source-image identity if it will support grounding.
- **Direct environment query:** retrieves authoritative backend or application state for verification rather than perception.

OpenAI's current guide explicitly supports visual computer actions, custom Playwright/Selenium/VNC/MCP harnesses, and code-execution harnesses that can combine screenshots with DOM-based workflows ([OpenAI —](#)

[Computer Use](#)). This supports a hybrid design: use structured state for stable element identity and efficient reading, pixels for geometry and visual conditions, and direct state queries for outcome verification.

No channel is automatically ground truth for every question. A DOM node may exist while an overlay receives the click; a screenshot may show text without exposing the element's role or stable identity. Playwright therefore checks properties such as visibility, stability, whether an element receives events, enabled state, and editability before supported actions ([Playwright — Auto-Waiting and Actionability](#)). A computer-use harness should expose equivalent preconditions where its automation stack permits them.

Observation selection should be task- and model-evaluated. Record which channel and extraction version produced each decision, and archive full observations as artifacts when model context only receives a compact projection.

15.4 Grounding Is a Versioned Geometry Contract

Coordinate grounding translates an intended target into the executor's coordinate space. It may use:

- raw pixel coordinates predicted from a screenshot;
- an element reference resolved through DOM or accessibility state;
- a numbered overlay such as Set-of-Mark, where candidate regions receive discrete labels ([Yang et al. — Set-of-Mark Prompting](#));
- a hybrid lookup that selects an element semantically, checks its current bounds, and clicks a verified point.

Every mapping must declare its geometry:

```
captured_pixel_size, model_visible_size, viewport_size
crop_origin, scroll_container_and_offset, browser_zoom
device_pixel_ratio, resize_scale, coordinate_origin
window_or_frame_id, element_or_mark_id
```

Do not let a provider resize a screenshot invisibly and then apply returned coordinates directly to the original display. Anthropic's current computer-use documentation warns that oversized images may be downscaled before the model sees them, so the model's coordinates refer to the resized image; it recommends client-side resizing with a known scale and calls out Retina device-pixel-ratio handling ([Anthropic — Computer Use Tool](#)). OpenAI likewise recommends `detail: "original"` for current GPT-5.6 computer-use screenshots and requires coordinate remapping if the client downsizes them ([OpenAI — Computer Use](#)).

Grounding expires with its source observation. Before dispatch, re-resolve element references or verify that the relevant geometry and surface identity remain current. A coordinate should never be treated as a durable identifier for a button.

15.5 Observation/Action Races Are Normal

Between capture and dispatch, the interface can change. The central race is:

```
observe state S0 -> propose action for S0 -> environment becomes S1 -> execute against S1
```

Several mechanisms create this race:

Mechanism	Failure example	Control
Stale screenshot	A delayed model response clicks a page that already navigated	Bind action to observation age and surface identity; re-observe after a freshness deadline
Layout shift or animation	The target moves and the coordinate lands on another control	Require stable bounds or re-ground immediately before dispatch
Focus change	Typing goes to the address bar, terminal, or wrong field	Verify active window/frame and focused element before sensitive input
Scroll drift	The wrong container scrolled, or a target left the viewport	Record container and offset; verify target in viewport after scrolling
Modal/overlay	A consent dialog or popup intercepts the click	Detect topmost dialog/overlay and pause or route it through policy
Latency/navigation	A click is accepted while the next action assumes the old page	Wait for declared navigation/network/UI condition, then capture a new observation
Partial batch execution	Action 1 succeeds, action 2 fails, action 3 would now be unsafe	Record per-action status; stop dependent actions and return <code>partial</code> plus a fresh observation

OpenAI's current built-in contract can batch ordered actions in one `computer_call` and capture a screenshot after the batch ([OpenAI — Computer Use](#)). That is a latency optimization, not evidence that every batch is atomic. This chapter therefore recommends batching only actions whose preconditions remain valid together and splitting consequential, navigation-producing, focus-sensitive, or weakly reversible actions with observation barriers.

Do not “fix” a race by blindly replaying a click or submit. A lost response or timeout can leave the environment changed even when the executor did not observe success. Use Chapter 6's `partial` and `unknown` states, inspect the UI and backend, and deduplicate or escalate before another consequential attempt.

15.6 Validate the Action at Dispatch Time

Computer-use dispatch specializes, but does not bypass, the validation stack from Chapter 6. Before execution, the policy enforcement path should check:

- proposal and observation identity, freshness, and correlation;
- action schema and semantic bounds;
- application, process, window, tab, frame, URL, domain, and origin as applicable;
- current focus, target visibility/stability, overlay state, and coordinate transform;
- actor, tenant, credential scope, policy version, and action allowlist;
- whether the action transmits data, communicates externally, spends money, changes access, modifies production, or is difficult to reverse;
- mandatory approval bound to the exact normalized proposal;
- retry, deduplication, deadline, cancellation, and action-budget state.

The result envelope should be per action, even when a provider batches proposals:

```
status: denied | not_executed | succeeded | failed | partial | unknown
proposal_id, attempt_id, source_observation_id
surface_before, surface_after
normalized_action, executor_error
post_action_observation, postcondition_evidence
```

The executor should capture a fresh observation after a state-changing action or batch and verify the expected transition. “Mouse event delivered” is an execution result; “form submitted once and order exists with the expected fields” is an environment outcome.

15.7 Safety Must Surround the Model

Rendered pages, documents, emails, chats, PDFs, and tool output are untrusted input. OpenAI's computer-use guide says on-screen instructions are not user permission and recommends an isolated environment plus domain/action allowlists; Anthropic warns that content in webpages or images can override instructions in some circumstances and

recommends a minimally privileged VM/container, restricted domains, and human confirmation for meaningful consequences ([OpenAI — Computer Use](#); [Anthropic — Computer Use Tool](#)).

For high-risk UI actions, combine:

1. **Action allowlist:** expose or dispatch only approved action types and scoped targets; deny unexpected keyboard shortcuts, downloads, shell launches, clipboard reads, or credential entry by default.
2. **Sandbox:** use a separate browser profile, container, or VM with minimal filesystem, network, credential, device, and host integration, as defined in [Chapter 7](#).
3. **Domain and origin checks:** evaluate the actual current origin at navigation and dispatch, including redirects, iframes, popups, and newly opened tabs; a remembered domain from the prior screenshot is not current authority.
4. **Mandatory approval:** insert the runtime gate from [Chapter 14](#) immediately before sending/posting, transmitting sensitive data, purchasing, deleting, changing access, accepting terms, or another policy-classified consequence. Approval binds the exact target, payload, identity, origin, and expiry.
5. **Post-action verification:** independently check the new UI and, where possible, authoritative backend state before reporting success or attempting a retry.

Instruction priority remains a behavioral defense: it can tell the model to treat page content as data and to surface suspicious instructions. It is not an allowlist, sandbox, origin check, approval record, or PEP. [Chapter 2](#) defines that boundary. Provider-side prompt-injection classifiers are also defense in depth; Anthropic explicitly says its computer-use precautions remain necessary even with its classifier layer ([Anthropic — Computer Use Tool](#)).

15.8 Visual Accounting Is Provider- and Model-Specific

There is no portable conversion between “one screenshot” and “one page of text.” Image preprocessing, detail modes, resizing, patch/tile rules, token multipliers, and prices vary by provider, model, snapshot, and API contract ([Anthropic — Vision](#); [OpenAI — Images and Vision](#)).

Two dated examples show why. They are not equivalences:

Contract verified July 31, 2026	Input example	Documented accounting basis
Anthropic Vision, Claude Opus 5 high-resolution tier	1000×1000 image	<code>ceil(width/28) × ceil(height/28) = 1,296</code> visual tokens; the same contract lists a 2,576-pixel long-edge and 4,784-visual-token native limit for Claude 4.7-and-later high-resolution models (Anthropic — Vision)
OpenAI Vision, GPT-5.6 , detail: "original"	1000×1000 image	<code>ceil(width/32) × ceil(height/32) = 1,024</code> original patches; the current contract says GPT-5.6 <code>original</code> does not resize to a pixel-dimension or patch-budget limit (OpenAI — Images and Vision)

For every visual cost or context claim, record:

provider, endpoint, model and snapshot/tool version
verification date, image detail/fidelity mode
source and model-visible dimensions, crop and resize transform
patch/tile rule and multiplier, observed input usage, price contract

Optimize from measured task performance and actual usage. Cropping, downscaling, structured UI state, zoom, or fewer observation barriers can reduce cost and latency, but each may reduce legibility or increase race risk. Archive old screenshots as artifacts and keep only decision-relevant observations in model context, following [Chapter 5](#); do not discard evidence required for evaluation or incident review.

Round-trip latency is also contract-specific. Anthropic describes its screenshot-based computer tool as slower because actions require screenshot round trips, while OpenAI's current tool can return ordered action batches before the next screenshot ([Anthropic — Tool Combinations](#); [OpenAI — Computer Use](#)). Measure capture, upload, inference, approval wait, dispatch, application settle, and verification separately.

15.9 Recovery and Stop Conditions

A computer-use loop should stop or escalate when:

- the verified environment outcome holds;
- policy denies the next action or mandatory approval is denied/expired;
- the current origin, account, application, or target falls outside scope;

- a modal, CAPTCHA, authentication challenge, suspicious instruction, or sensitive-data request requires human handling;
- execution is partial or unknown and reconciliation cannot establish state;
- observation, action, token, cost, or wall-time budget is exhausted;
- the loop repeats without environment progress;
- the user steers or cancels the run.

Stopping because the model emits prose or no longer requests a tool is not sufficient completion evidence. Reconcile any in-flight action and apply the outcome predicate before finalization. Chapters 6 and 14 define unknown effects, steering, cancellation, and approval expiry; [Chapter 13](#) defines the general loop stop hierarchy.

15.10 Evaluate Final UI and Backend State

Computer-use evals should grade the layers separately:

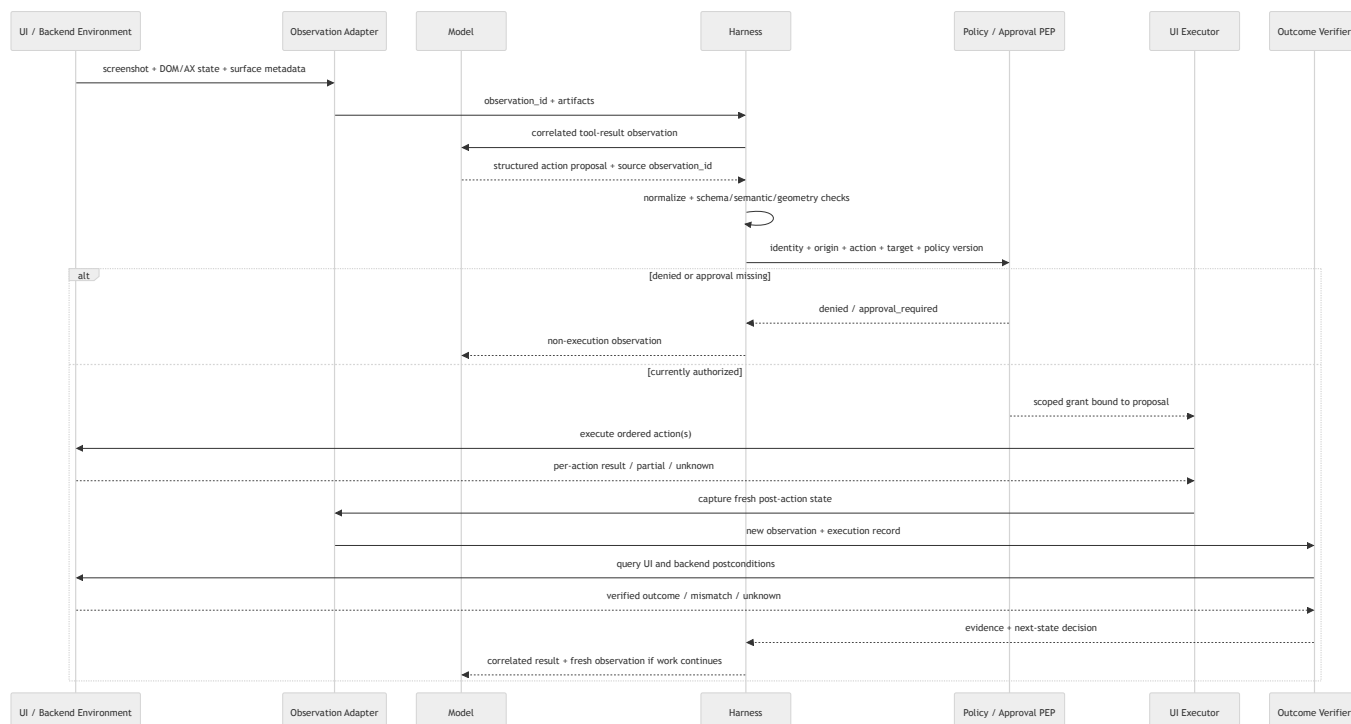
- **Process:** allowed origins, forbidden actions, approval use, sensitive-data handling, budgets, and race recovery.
- **Artifact:** downloaded or edited files, screenshots, exported reports, or other addressable outputs.
- **UI environment state:** visible dialog state, selected value, current page, application state, or rendered confirmation.
- **Backend/external state:** database record, submitted form, sent message, account permission, order, or other durable effect.
- **User outcome:** whether the intended task was actually satisfied.

A final screenshot is evidence, not necessarily the outcome. A confirmation banner can be stale or misleading; backend state can change without a visible confirmation. Anthropic's agent-eval guidance specifically describes browser tasks graded with URL/page-state checks and backend verification, and distinguishes transcript from final environment outcome ([Anthropic — Demystifying Evals for AI Agents](#)).

WebArena supplies self-hosted web tasks with functional correctness evaluators, while OSWorld supplies desktop tasks with execution-based evaluation scripts over application and system state ([WebArena](#); [OSWorld](#)). Treat them as primary benchmark designs, not substitutes for workload-specific tasks. Apply [Chapter 11's](#) isolated baselines, repeated trials, complete trajectories, configuration pinning, grader calibration, and per-slice reporting.

Report slices for observation channel, screen resolution/detail contract, grounding method, application/origin, action type, modal/navigation presence, latency, partial/unknown execution, policy gate, and outcome-verification method. Otherwise an aggregate score can hide that the agent succeeds only on static pages or only when backend verification is absent.

15.11 The Full Computer-Use Loop



The screenshot and structured state are observation payloads. The UI operation is a structured proposal. The application executes it, and only postcondition evidence can establish the environment outcome.

15.12 Design Checklist

Before releasing a computer-use workflow, verify that:

- Every proposal names the observation and coordinate transform it depends on.
 - Screenshot, DOM/accessibility state, grounding, proposal, execution result, and environment outcome have separate identities.
 - The executor validates surface, origin, focus, scroll, overlay, and geometry at dispatch time.
 - Batched actions have compatible preconditions and produce per-action results.
 - Partial and unknown execution stop dependent actions and trigger reconciliation.
 - High-risk actions pass through allowlists, sandboxing, origin checks, mandatory approval, and post-action verification.
 - On-screen instructions are treated as untrusted content; instruction priority is not used as enforcement.
 - Visual accounting records provider, model, version, resolution, detail mode, date, usage, and price contract.
 - Completion graders inspect final UI and backend state, not only the trajectory or screenshot.
 - Evals restore isolated baselines and report race, grounding, resolution, policy, and outcome slices.
-

Key Takeaways

- **Computer use is a structured tool loop.** The model proposes; the application validates, authorizes, executes, and returns an observation.
- **Pixels and tool results are not opposites.** A screenshot is commonly the payload of a correlated tool result.
- **Gestures and function calls are not opposites.** Click, type, scroll, and key actions are structured proposals under provider or custom schemas.
- **Keep perception, grounding, execution, and outcome separate.** Their failure modes and evidence differ.
- **Observation/action races are expected.** Staleness, layout, focus, scroll, modals, latency, and partial execution require explicit state handling.
- **Safety surrounds the model.** Allowlists, sandboxing, origin checks, mandatory approvals, and verification create the boundary; instruction priority only assists behavior.
- **Visual accounting is contract-specific.** Never translate screenshots into a universal number of text pages.
- **Environment outcome decides success.** Grade final UI and backend state, with process checks for policy-sensitive paths.

Further Reading

- OpenAI, *Computer Use*. <https://developers.openai.com/api/docs/guides/tools-computer-use>
- Anthropic, *Computer Use Tool*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool>
- Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
- Microsoft Playwright, *Auto-Waiting and Actionability*. <https://playwright.dev/docs/actionability>
- Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
- Anthropic, *Vision*. <https://platform.claude.com/docs/en/build-with-claude/vision>
- OpenAI, *Images and Vision*. <https://developers.openai.com/api/docs/guides/images-vision>
- Anthropic, *Tool Combinations*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-combinations>
- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
- Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>

Chapter 16: Infrastructure Noise in Agent Evals

Chapter 11 defined an evaluation result as evidence about a complete, versioned model-plus-harness configuration. This chapter examines one easily hidden part of that configuration: the infrastructure in which trials run. For an agent that installs dependencies, calls services, runs tests, and changes environment state over many turns, CPU, memory, storage, network, concurrency, and time limits can alter both whether the run completes and which strategies are feasible. Anthropic's infrastructure study makes the boundary concrete: two agents with different resource budgets or time limits are not taking the same test ([Anthropic — Quantifying infrastructure noise in agentic coding evals](#)).

The phrase **infrastructure noise** is claim-relative. If an eval claims to isolate a model or harness change, uncontrolled infrastructure variation is a nuisance variable or confounder. If it claims to measure an end-to-end product under a declared production contract, rate limits, timeouts, and resource ceilings are part of the tested system and their failures count toward reliability. Decide which claim you are making before deciding what to hold constant or exclude.

16.1 Define the Estimand Before Running Trials

An **estimand** is the quantity the experiment is designed to estimate. Infrastructure studies commonly ask one of three different questions:

1. **Model-plus-harness comparison under a fixed environment:** hold infrastructure constant and estimate the outcome difference between systems A and B.
2. **Infrastructure sensitivity:** hold the model, harness, tasks, and graders constant and estimate what changes when one infrastructure factor changes.
3. **Production-system reliability:** sample the infrastructure conditions users will actually encounter and estimate the complete system's outcome, latency, cost, and failure distribution.

These questions require different treatment of failures. An unexpected cluster setup failure may invalidate a model-capability trial. A timeout imposed by the declared product contract is a system reliability failure, not removable “noise.” Preserve Chapter 11's distinction among task verdict, infrastructure-invalid trial, and in-scope reliability failure.

Write the target claim and the treatment contrast before looking at scores:

Claim: changing X from A to B changes metric Y

Population: declared task suite and slices

Fixed configuration: model, harness, graders, and named infrastructure fields

Experimental unit: one trial

Pairing/blocking keys: task × replicate × environment block

Primary effect: absolute difference in outcome rate

Uncertainty: declared interval or model, clustered at the task/block level

Invalid-trial policy: versioned inclusion, exclusion, and retry rules

Without this contract, “same benchmark” can conceal different experiments.

16.2 The Anthropic Resource Experiment Is a Named Case Study

Anthropic ran Terminal-Bench 2.0 on a Google Kubernetes Engine cluster under six resource configurations, ranging from using each task's resource specification as both allocation floor and hard ceiling (1×) to uncapped resources. The same Claude model, harness, and task set were used; the intended treatment was resource configuration ([Anthropic — Quantifying infrastructure noise in agentic coding evals](#)).

The reported results belong to that setup:

Comparison	Experimental setting	Reported result
Terminal-Bench 1× → 3× ceiling	Same model, harness, task set; increasing headroom over per-task specifications	Infrastructure errors fell from 5.8% to 2.1% ($p < 0.001$); success scores fluctuated within the experiment's noise ($p = 0.40$)
Terminal-Bench 3× → uncapped	Same six-configuration study	Infrastructure errors fell another 1.6 percentage points while success increased by almost 4 percentage points
Terminal-Bench 1× → uncapped	Extremes of the same study	Success differed by 6 percentage points ($p < 0.01$)
SWE-bench 1× → 5× RAM	Crossover experiment over 227 problems with 10 samples each	Score increased by 1.54 percentage points

Anthropic interpreted the Terminal-Bench curve as two empirical regimes. Up to roughly 3×, additional headroom mainly reduced container failures; above that point, additional resources enabled approaches involving large dependencies, expensive subprocesses, or memory-intensive tests. The `bn-fit-modify` example showed some models installing a Python data-science stack under generous limits but exhausting memory during installation under tight limits, while a standard-library solution remained possible ([Anthropic — Quantifying infrastructure noise in agentic coding evals](#)).

This case also illustrates why allocation and enforcement must be reported separately. Kubernetes distinguishes resource **requests** used for scheduling from resource **limits** enforced by the runtime; CPU and memory limits also have different enforcement behavior ([Kubernetes — Resource Management for Pods and Containers](#)). A label such as “8 GB machine” is not a complete execution contract.

16.3 Three Percentage Points Is Not a Universal Threshold

Anthropic concluded that, until resource methodology is standardized, leaderboard differences below 3 percentage points deserved skepticism **for the studied agentic-coding setting until configurations were documented and matched**. The article tied that advice to an observed spread just below 2 points across moderate Terminal-Bench resource configurations, naive binomial intervals of roughly 1–2 points, and a 6-point spread at allocation extremes ([Anthropic — Quantifying infrastructure noise in agentic coding evals](#)).

Do not convert that case-specific diagnostic into any of these claims:

- differences below 3 points are never real;
- differences above 3 points are significant;
- 3 points is an acceptable regression budget;
- every benchmark has 3 points of infrastructure noise;
- matching RAM alone removes all infrastructure confounding.

A universal cutoff ignores sample size, paired structure, task mix, baseline rate, variance, multiple comparisons, grader error, and the infrastructure variables actually changed. For a new eval, estimate its own effect size and uncertainty under its own design. NIST's guidance on intervals for a binomial proportion makes the interval depend on the observed count and sample size rather than a fixed percentage-point rule ([NIST/SEMATECH — Confidence Intervals for a Proportion](#)).

16.4 Freeze and Publish an Infrastructure Manifest

Extend Chapter 11's `EvalConfiguration` with an infrastructure manifest. Record configured values and, where available, observed values:

Variable family	Minimum fields to pin or record
Hardware and resources	Cloud/provider, region/zone, node or VM type, CPU architecture, vCPU request and limit, memory request and limit, accelerators, ephemeral-storage request and limit, scheduler class, autoscaling behavior
Disk and filesystem	Local/remote storage type, capacity, IOPS/throughput policy, filesystem and mount options, workspace location, free space at start, snapshot/reset mechanism
Network	Allowed destinations, DNS/proxy configuration, egress path, bandwidth/connection limits, region to each dependency, network emulation, outage or incident status
Container image	Image registry, repository, immutable digest, base-image digest, OS/runtime versions, entrypoint, sandbox/runtime version
Dependencies	Lockfile and package-manager versions, repository/index snapshot, system packages, compilers, test runners, browser/drivers, installation policy
Service and rate limits	Model/provider endpoint and region, account tier, quotas, requests/tokens per interval, connection pools, retry and backoff policy, observed throttling
Cache state	Image, package, build, repository, DNS, retrieval, and application caches; cold/warm treatment; priming sequence; reset or sharing scope
Parallelism and contention	Evaluation concurrency, workers per node, agent subprocess limits, neighboring workloads, task scheduling/order, queue policy
Timeouts and clocks	End-to-end, model, tool, command, install, test, network, queue, idle, and cleanup timeouts; retry budget; clock/timezone controls

Do not identify an image only by a mutable tag. Docker documents that a tag can later resolve to a different image and that a digest pins the image version; it also documents how build caches and unpinned packages can change what a later build installs ([Docker — Building best practices](#)). Record dependency lock hashes and repository snapshots in addition to the image digest because trials may install software after startup.

“Warm” is not a single state. Name which cache is warm, who primed it, whether it is shared across trials, and whether the agent can observe artifacts from another trial. If cache state is not the treatment, reset or balance it across variants. If it is the treatment, define cold and warm protocols precisely.

16.5 Pair by Task and Environment When Possible

For an A/B comparison, run both variants on the same `task_id` and the same declared baseline. Place them in matched environment blocks: same image and dependencies, hardware/resource class, network policy, cache treatment, timeout policy, grader version, and a narrow run window. Repeat enough trials per task to represent model non-determinism, and randomize or counterbalance A/B order within blocks.

The resulting observations are paired because each A result has a natural B counterpart for the same task and block. NIST defines paired observations in exactly this way—measurement `i` in one sample is naturally matched to measurement `i` in the other—and analyzes their within-pair differences ([NIST/SEMATECH — Analysis of Paired Observations](#)). NIST’s experimental-design guidance likewise recommends holding controllable nuisance factors constant within blocks and randomizing the factors that cannot be controlled ([NIST/SEMATECH — Randomized Block Designs](#)).

“Same environment” does not mean reusing contaminated state. Start each trial from the same immutable or recoverable baseline but give it an isolated namespace, filesystem, credentials, and external-resource scope. A matched block should equalize intended conditions without letting one trial’s artifacts help the other.

If pairing is impossible—for example, variants use incompatible hardware, providers, or time windows—report an unpaired comparison and name the confounds. Region, time of day, account tier, image, concurrency, or provider incident can remain plausible alternative explanations; Anthropic specifically identifies cluster health, hardware, concurrency, egress bandwidth, and time-varying API conditions as possible sources of variance, while noting that it had not formally quantified the time-of-day observation ([Anthropic — Quantifying infrastructure noise in agentic coding evals](#)). Do not describe the observed difference as the causal effect of the model or harness alone.

16.6 Preserve a Raw Trial Ledger

A leaderboard point estimate is not sufficient evidence. Store one immutable record per trial:

```

InfrastructureTrial {
  task_id, trial_id, replicate_id, pair_or_block_id, variant,
  start_time, run_order, model_and_harness_version,
  task_suite_and_grader_version, infrastructure_manifest_hash,
  node_or_worker_id, image_digest, dependency_lock_hash,
  configured_resources, observed_peak_resources,
  cache_treatment, concurrency, rate_limit_events,
  phase_timings, retries, timeout_reason,
  task_verdict, validity, failure_category,
  artifact_ids, trajectory_id, environment_outcome_id
}

```

Publish or preserve raw trial rows alongside the aggregate, subject to privacy and security controls. At minimum report the number of tasks, trials per task and variant, pass/fail/partial counts, invalid trials, retries, and missing records. Anthropic recommends multiple trials because agent outputs vary and emphasizes stable environments for coding evals ([Anthropic — Demystifying evals for AI agents](#)).

Use a versioned failure taxonomy:

- model/agent substantive failure;
- CPU, memory, accelerator, process, or disk exhaustion;
- storage or filesystem failure;
- network, DNS, proxy, or egress failure;
- container-image, dependency, package-index, or setup failure;
- provider error, rate limit, or backoff exhaustion;
- timeout, queue delay, or cancellation;
- concurrency contention or worker eviction;
- cache contamination or failed reset;
- harness, grader, fixture, cleanup, or logging failure.

Classification depends on the claim. A rate limit inside the declared production contract is a valid reliability failure. A grader outage outside the tested system may make the trial invalid. Record the reason code before applying retry or exclusion policy so that infrastructure errors cannot disappear into a rerun.

16.7 Report Effect Size and Uncertainty Together

For a binary outcome, start with the absolute pass-rate difference in **percentage points**, with raw numerators and denominators. Relative lift or a rate ratio may supplement it, but always include the baseline: “1.5×” can describe very different absolute changes at 2% and 60% baselines.

For paired repeated trials, compute task-level differences so that tasks with more retained attempts do not silently dominate:

```

d_task = mean(outcome_B for task and matched blocks)
        - mean(outcome_A for task and matched blocks)
primary effect = mean(d_task across the declared task population)

```

Choose an interval or statistical model that respects the design: paired task bootstrap, an appropriate matched binary procedure, or a hierarchical model when trials are nested under tasks and environment blocks. Report the sampling unit, confidence or credible level, method, assumptions, and sensitivity to invalid-trial handling. Chapter 11 explains why treating correlated attempts as independent overstates precision.

Every result table should contain:

- raw task and trial counts for each variant;
- absolute effect size and, if useful, a relative effect with its baseline;
- uncertainty interval and any pre-specified hypothesis-test result;
- failure-category counts and deltas;
- latency, cost, and resource-use distributions;
- invalid, missing, retried, and excluded trials by reason;

- per-task or per-slice results, not only a pooled score;
- the complete configuration or its content-addressed manifest.

A `p`-value does not replace the effect size or interval. Likewise, a narrow interval does not repair an uncontrolled confound or a broken grader. If many infrastructure variants or slices are compared, declare primary contrasts before looking at results and account for exploratory multiplicity in the interpretation.

16.8 Diagnose Why a Score Changed

Separate at least four mechanisms:

1. **Reliability stabilization:** fewer otherwise-invalid or in-scope infrastructure failures reach the grader.
2. **Strategy enablement:** the agent can now install, search, compile, test, or brute-force approaches that the tighter configuration prevented.
3. **Latency-budget interaction:** faster hardware, warmer caches, less contention, or different rate limits let more work fit inside the same timeout.
4. **Uncontrolled confounding:** another configuration or time-varying condition changed with the intended treatment.

Compare both outcome and trajectory evidence: failure reasons, resource peaks, install and test behavior, subprocess count, service throttling, queue time, and phase latency. In Anthropic's Terminal-Bench case, the `1x – 3x` range was dominated by declining infrastructure errors, while the rise above `3x` outpaced that decline and coincided with resource-intensive strategies ([Anthropic — Quantifying infrastructure noise in agentic coding evals](#)). That is evidence about that experiment, not a universal breakpoint.

Do not relabel every infrastructure-sensitive success as score inflation. If the product genuinely provides the larger resource budget, exploiting it may be valid capability. The report should say whether it measures efficiency under constraint, maximum task success under a budget, or reliability under production conditions.

16.9 Run a Sensitivity Matrix, Not One “Clean” Configuration

A pinned baseline supports comparison but does not show robustness. Before release or publication, test the important configuration boundaries:

Factor	Example controlled contrast	Evidence to inspect
Hardware/resources	Declared request/limit floor vs calibrated ceiling	Outcome, OOM/throttling, peak use, strategy change
Disk/network	Local vs remote disk; normal vs declared bandwidth/latency condition	I/O errors, install/test latency, timeout interaction
Image/dependencies	Same image digest; then one planned dependency upgrade	Reproducibility, setup failures, behavioral deltas
Rate limits	Declared quota tiers or controlled throttling	Retry/backoff behavior, queue time, incomplete tasks
Warm caches	Precisely defined cold vs warm treatment	Hit rate, latency/cost, contamination checks
Parallelism	Low vs production concurrency at fixed per-trial limits	Contention, tail latency, eviction, outcome
Timeouts	Production timeout vs a diagnostic longer timeout	Near-boundary completions, hung phases, strategy changes

Screen one factor at a time when diagnosis is the goal; use a pre-specified factorial or blocked design when interactions matter. Do not change model, harness, image, resources, concurrency, and timeout together and then attribute the entire score delta to one component.

16.10 Publish a Reconstructible Result

An infrastructure-aware report should include:

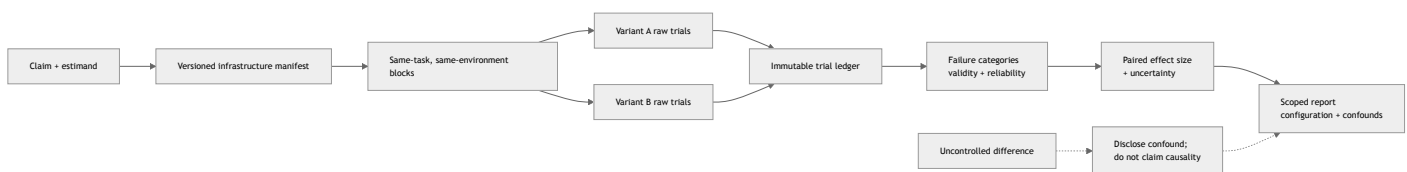
- claim, estimand, task population, slices, and decision owner;
- model, agent harness, evaluation harness, tool, grader, and policy versions;
- experiment design, pairing/blocking, order randomization, run dates, and locations;
- complete infrastructure manifest, including resource allocation **and enforcement**;
- raw trial ledger or a privacy-preserving derivation with checksums;

- point estimates, raw counts, effect sizes, uncertainty, and failure categories;
- invalid-trial, retry, missing-data, and exclusion policies with sensitivity analysis;
- named confounds and the limits of causal or cross-lab comparison;
- release decision, exception owner, and monitoring or rerun trigger.

OpenAI's evaluation playbook similarly asks reports to disclose the tested system, harness and tool access, elicitation method, budgets, safeguards, and validity checks because these determine what claim the result can support ([OpenAI — A shared playbook for trustworthy third-party evaluations](#)).

Cross-lab reproduction does not require identical vendor names, but it does require equivalent experimental contracts or an explicit statement of differences. If equivalence cannot be established, publish separate results rather than normalizing away the discrepancy. A benchmark score becomes useful evidence when another reviewer can reconstruct what ran, classify what failed, and understand which uncertainty belongs to sampling and which belongs to configuration.

Diagram: From Infrastructure Variables to a Scoped Claim



Key Takeaways

- **Infrastructure noise is claim-relative.** A nuisance variable in a model comparison can be an in-scope reliability condition in a product eval.
- **The 3-percentage-point advice is a named Anthropic case result.** It is not a universal significance threshold, regression budget, or estimate of every benchmark's noise.
- **Publish the full execution contract.** Hardware/resources, disk/network, image and dependencies, rate limits, cache state, parallelism, and timeouts all belong in the manifest.
- **Pair by task and environment when possible.** Use matched baselines and blocks, then analyze within-pair differences; otherwise disclose the unpaired confounds.
- **Preserve raw trials and failure categories.** Distinguish substantive failure, in-scope system reliability failure, and infrastructure-invalid trials before retrying or excluding anything.
- **Report effect size with uncertainty.** Include raw counts, sampling unit, per-slice effects, intervals, invalid-trial sensitivity, latency, cost, and resource use—not only a leaderboard point estimate.
- **Keep every multiplier and score shift attached to its setup.** The 3×, 6-point, and 1.54-point results describe specific Anthropic experiments, not portable constants.

Further Reading

- Gian Segato, *Quantifying infrastructure noise in agentic coding evals*, Anthropic. <https://www.anthropic.com/engineering/infrastructure-noise>
- Mikaela Grace et al., *Demystifying evals for AI agents*, Anthropic. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- NIST/SEMATECH, *Randomized Block Designs*. <https://www.itl.nist.gov/div898/handbook/pri/section3/pri332.htm>
- NIST/SEMATECH, *Analysis of Paired Observations*. <https://www.itl.nist.gov/div898/handbook/prc/section3/prc311.htm>
- NIST/SEMATECH, *Confidence Intervals for a Proportion*. <https://itl.nist.gov/div898/handbook/prc/section2/prc241.htm>
- Kubernetes, *Resource Management for Pods and Containers*. <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- Docker, *Building best practices*. <https://docs.docker.com/build/building/best-practices/>
- OpenAI, *A shared playbook for trustworthy third-party evaluations*. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>

Chapter 17: Trace-Driven Iteration

A production trace can show where time went, which components interacted, and which errors appeared along one observed execution path. That makes traces valuable for debugging and for discovering candidate regression cases. It does not make them the system of record for every other purpose.

Chapter 10 defines a **trace** as observability data composed of spans, timestamps, attributes, events, links, and status. OpenTelemetry exposes those objects through its tracing API and allows implementations to sample what is recorded or exported ([OpenTelemetry — Trace API](#); [OpenTelemetry — Trace SDK](#)). This chapter asks how to use that fallible, potentially incomplete signal to improve a versioned model-plus-harness system safely.

17.1 Trace Is One Evidence Product, Not Four

Keep these products separate:

Product	Required property	Primary use	Why a sampled trace cannot replace it
Trace	Queryable spans and causal/correlation metadata under a declared sampling policy	Observability, performance diagnosis, failure discovery	Sampling may drop an entire execution or individual detail
Event history	Ordered, durable accepted events under a recovery/replay contract	Reconstruct execution state and resume	Trace order and retention are not authoritative workflow state
Eval trajectory	Every model, tool, observation, artifact, and policy event required by declared graders for one trial	Grade process behavior	Production instrumentation may omit or redact grader-required evidence
Audit record	Defined content, identity, timestamps, completeness, integrity, access, protection, and retention	Accountability, investigation, compliance	A trace may be mutable, sampled, access-scoped differently, or retained for too short a period

Temporal's event history is an append-only workflow record used for recovery, while OpenTelemetry sampling can intentionally discard attributes, events, status, or complete spans ([Temporal — Events and Event History](#); [OpenTelemetry — Trace SDK](#)). NIST treats audit generation, content, protection, review, access, and retention as explicit controls rather than properties inherited from an arbitrary log ([NIST SP 800-53 Rev. 5](#)). Chapter 11 likewise requires a complete eval trajectory for the evidence a grader consumes.

The products should link to one another without collapsing. A span may carry an event-history ID; an eval trajectory may be built from unsampled instrumentation plus artifacts; an audit pipeline may copy selected protected records. The lineage is useful precisely because the guarantees remain explicit.

17.2 Correlate Durable IDs with Trace and Span IDs

Chapter 10 defines application identities that outlive an observability backend:

```
session_id -> run_id -> step_id -> action_id -> attempt_id
call_id attaches to the step or attempt that issued the protocol exchange
```

Trace identities answer different questions. W3C Trace Context standardizes propagation of `trace-id`, parent/span identity, and trace flags across service boundaries ([W3C — Trace Context](#)). Do not substitute those transport-oriented IDs for durable workflow identities.

Durable identity	Trace correlation	Important distinction
<code>session_id</code>	Attribute on relevant root and child spans	One session may contain many runs and traces
<code>run_id</code>	Attribute on every span for the run; often correlated with one root trace or a linked series of traces	A long-running run can outlive trace retention or cross trace boundaries
<code>step_id</code>	Attribute on the orchestration span representing that logical transition	Repeating a loop step under explicit semantics does not make <code>span_id</code> the source of workflow position
<code>action_id</code>	Attribute on the intended-effect span and all delivery attempts	Remains stable across retries; it is not a tool-call transport ID
<code>attempt_id</code>	Attribute on one model/tool execution-attempt span	Changes for each attempt under the same logical step or action
<code>call_id</code>	Attribute on the provider/protocol model or tool span and correlated result	Identifies one exchange; it does not supply idempotency for an external effect
<code>trace_id</code> / <code>span_id</code>	Native observability correlation	May be sampled, regenerated at boundaries, or absent from durable recovery data

Use a parent–child relationship for synchronous causal nesting. Use span **links** for work joined from a queue, fan-out/fan-in branches, retries tied to an earlier attempt, or operations that start after the original trace ended; OpenTelemetry allows links to spans in the same or another trace ([OpenTelemetry – Trace API](#)). Preserve `causation_id` and `correlation_id` in the durable event envelope as well, because a trace backend is not the source of truth for accepted state.

17.3 Design Spans for Questions, Not Maximum Payload

A span represents one operation. OpenTelemetry lets instrumentation set attributes, add timestamped events, set status, record exceptions, and link related spans ([OpenTelemetry – Trace API](#); [OpenTelemetry – Exceptions](#)). Use each field deliberately:

- **Attributes** hold stable, queryable facts: durable IDs, operation type, component and configuration versions, model/tool name and schema version, tenant-safe classification, token/cost counters, retry number, policy outcome, and artifact references. Avoid raw secrets, full prompts, high-cardinality payloads, or mutable prose as primary keys.
- **Events** record point-in-time facts inside the operation: retry scheduled, approval requested, first byte received, timeout observed, cancellation requested, result normalized, or postcondition checked. An event on a span is still observability data, not automatically an accepted workflow event.
- **Status** summarizes the operation's tracing status. OpenTelemetry defines `Unset`, `Ok`, and `Error`; it does not encode the application's complete state machine ([OpenTelemetry – Trace API](#)). Keep `partial`, `cancelled`, `unknown outcome`, policy denial, and business success as explicit domain attributes/events and durable state.
- **Errors** should use a stable category and safe description, plus a reference to full diagnostics when needed. OpenTelemetry's exception convention records an unhandled exception as an event when it causes the span to end in error ([OpenTelemetry – Exceptions](#)). A caught exception followed by successful recovery should not make the entire run look failed without qualification.
- **Links** represent non-tree relationships rather than forcing every asynchronous dependency into a false parent–child chain.

OpenTelemetry's GenAI semantic conventions define emerging names for model, agent, and tool spans, but the conventions remain versioned and parts may be marked Development ([OpenTelemetry – GenAI Spans](#)). Pin the convention version, document local extensions, and map provider fields at the adapter boundary. Do not bake a changing semantic-convention draft into durable business state.

A minimal span envelope might be:

```
trace_id, span_id, parent_span_id, links
session_id, run_id, step_id, action_id, attempt_id, call_id
operation, component_version, model_or_tool_version
started_at, ended_at, status, error_type
policy_decision, token_and_cost_counters, artifact_refs
sampling_policy_version, content_capture_mode
```

17.4 Sampling Policy Is Part of the Interpretation

OpenTelemetry sampling exists to reduce collection overhead and volume; a sampler may `DROP`, `RECORD_ONLY`, or `RECORD_AND_SAMPLE`, and an unrecorded span discards its attributes, events, and status ([OpenTelemetry — Trace SDK](#)). Therefore a dashboard built from exported traces answers a question about the sampled population unless the estimator and inclusion probabilities justify a broader claim.

Define and version the sampling policy:

- baseline probability or rule for ordinary successful runs;
- higher-priority capture for errors, policy denials, approval flows, unknown side effects, novel versions, and canary traffic;
- head- versus tail-sampling behavior and what happens if downstream services disagree;
- content-capture mode, redaction point, storage location, retention, and access class;
- how metrics or ledgers that require complete counts are produced outside sampled trace export;
- how operators discover blind spots when the collector, exporter, or instrumentation fails.

Priority sampling improves diagnostic coverage; it does not create a statistically representative dataset automatically. If failure traces are oversampled, their raw fraction is not the production failure rate. If successful runs are rarely sampled, absence of a failure pattern is weak evidence. Maintain unsampled counters or another complete operational record for billing, hard policy events, and durable recovery instead of asking traces to provide guarantees their sampling policy removed.

17.5 Three Activities That Must Not Be Confused

Activity	Question	Evidence	Output
Trace grader	Did a complete evaluation trajectory follow a declared process rule?	Unsampled trial trajectory, events, tool arguments, approvals, budgets, artifacts	Versioned grader verdict for one trial
Outcome grader	Did the artifact, environment, or user/business state satisfy the success criterion?	Tests, state queries, independent API checks, artifact inspection, user/business events	Versioned outcome verdict or score
Incident investigation	What happened in production, what was affected, why, and how should the system contain and recover?	Traces plus event history, logs, audit records, configuration, external-system state, human reports	Timeline, impact, containment/recovery actions, contributing causes, follow-ups

Anthropic's agent-eval guidance separates transcript/trajectory evidence from the outcome and warns against judging only what the agent said ([Anthropic — Demystifying Evals for AI Agents](#)). NIST's incident-response guidance treats preparation, detection, response, and recovery as organizational risk-management activities, not as a grader pass/fail decision ([NIST SP 800-61 Rev. 3](#)).

A production trace analyzer may flag “missing approval span.” That is a hypothesis for investigation unless the relevant record is known complete. A trace grader in an eval can fail a trial only when the evaluation harness guaranteed that the required approval evidence would be captured. An outcome grader can still pass the final artifact while a process grader fails a policy violation; neither verdict replaces incident triage for an actual production event.

17.6 Turn Production Failures into Governed Regression Cases

Production failures are valuable sources of candidate eval tasks, but copying traces directly into a suite creates privacy, rights, duplication, and validity risks. Use a governed intake pipeline:

1. **Triage the live event first.** Preserve volatile evidence and handle containment, notification, and recovery through the incident process; do not delay response while designing an eval.
2. **Establish use rights.** Check user notice or consent where required, contract terms, data-processing purpose, third-party content licenses, and intellectual-property restrictions before repurposing production material. NIST's AI RMF core calls for documented management of privacy and third-party data/software rights across AI-system risk work ([NIST — AI RMF Core](#)).
3. **Minimize and redact.** Remove secrets, credentials, personal and tenant data, unnecessary payloads, and production identifiers; preserve protected evidence separately when investigation or legal obligations require it.

4. **Reconstruct complete evidence.** Join the sampled trace with authorized event-history records, artifacts, configuration versions, and external outcomes. Mark missing facts; do not invent a complete trajectory from a partial trace.
5. **Create an isolated fixture.** Replace live side effects with a reproducible environment, synthetic or licensed data, scoped identities, and deterministic reset/teardown.
6. **Deduplicate and cluster.** Several alerts may be one underlying failure. Keep lineage to every source incident while preventing repeated copies from silently overweighting one class.
7. **Review distribution shift.** Decide whether the case belongs in a representative task distribution, a historical-regression slice, a high-risk policy suite, or incident-only testing. Do not change the headline distribution without versioning and disclosure.
8. **Write separate process and outcome assertions.** State what trace evidence must be complete and what independent environment result establishes success.
9. **Adjudicate and version.** Review task fairness, solvability, leakage, rights, grader behavior, and expected failure before accepting the case.

OpenAI's evaluation playbook identifies contamination, broken tasks, reward hacking, refusals, and sandbagging as validity hazards and recommends reporting the tested system, harness, budget, and validity checks ([OpenAI — Trustworthy Third-Party Evaluations](#)). Escaped production failures can strengthen a regression suite, but a growing archive of only failures is not a representative sample of ordinary traffic. Chapter 11's suite versioning, slices, isolation, and grader calibration still apply.

17.7 Trace Review Produces Hypotheses, Not Automatic Fixes

Trace review should classify the likely boundary before proposing a change:

Symptom	Candidate causes to test	Evidence to collect
Wrong tool or malformed call	Tool description/schema, context selection, model capability, router choice	Proposal, catalog/version, validation errors, relevant context, comparison trials
Repeated loop or premature stop	Stop state, error normalization, context compaction, no-progress detector, model behavior	Step/action IDs, error classes, state transitions, budgets, final outcome
Correct transcript, wrong external state	Tool timeout, duplicate/partial effect, stale observation, missing postcondition check	Action/attempt IDs, operation ID, event history, environment query
Policy or approval failure	Missing PEP, stale policy, identity/delegation mismatch, UI or event bug	Policy input/version, decision, approval proposal hash, dispatch evidence
Latency or cost regression	Model/routing change, cache miss, retrieval/tool latency, retry fan-out, infrastructure change	Child spans, sampling-adjusted metrics, configuration diff, controlled eval

The model's text is one component, not the default root cause. Repair may require a prompt, but it may instead require a typed tool, state transition, sandbox rule, policy gate, context transformation, infrastructure fix, or grader change. Record the hypothesis and falsifying test before editing the harness; otherwise teams can overfit a vivid trace without fixing the failure class.

17.8 Meta-Harness Changes Need a Release Pipeline

A “self-improving agent” should mean that production evidence informs an external, governed improvement process—not that the deployed agent rewrites its active prompt, tools, memory policy, runtime, or grader without bounds. OpenAI's tax-agent case study describes expert corrections becoming reviewed findings, targeted evals, and bounded coding tasks rather than directly rewriting the deployed agent ([OpenAI — Building Self-Improving Tax Agents with Codex](#)).

Apply the same release discipline to human-authored and meta-harness-generated changes:

1. **Bound the proposal.** Name the failure class, component, owner, expected effect, and files/configuration allowed to change.
2. **Create a version.** Pin the model, prompts, tool schemas/implementations, retrieval configuration, memory policy, sandbox image, runtime, policies, budgets, and graders affected.
3. **Run offline evals.** Require the targeted regression, broader holdout suite, critical policy slices, grader calibration checks, and cost/latency/reliability comparisons.

4. **Obtain approval.** A responsible owner reviews the change, evidence, known limitations, deployment scope, canary plan, and rollback trigger. The candidate cannot approve itself.
5. **Canary safely.** Start with a bounded traffic slice, limited authority, isolated tenants or tasks where appropriate, and enhanced outcome monitoring.
6. **Promote or roll back.** Use declared criteria, preserve the decision and lineage, and restore the last compatible version when a trigger fires.

Production traffic may generate traces, findings, candidate tests, or change proposals. It must not grant a process unbounded authority to mutate active production configuration and immediately evaluate itself on the same traffic. Separate proposal, evaluation, approval, deployment, and outcome monitoring identities; prevent the candidate from editing its holdout tasks or release grader.

17.9 Model–Harness Dependence Creates a Testing Responsibility

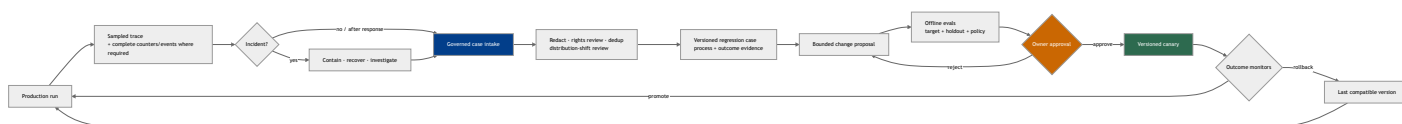
Performance belongs to a tested **model-plus-harness configuration**, not to either component in isolation. OpenAI's evaluation guidance explicitly asks reports to disclose harness choices because tools, budgets, safeguards, and elicitation setup can change the result a score supports ([OpenAI — Trustworthy Third-Party Evaluations](#)).

This is a dependency claim, not a prediction that model and harness must inevitably “co-evolve” in one direction. A new model may make a planner unnecessary, require a different tool interface, or expose a new failure. A harness change may improve one model and regress another. Anthropic describes an ablation-style practice: remove or alter a component and rerun realistic evals to determine whether it remains worth its cost for the current task and model ([Anthropic — Harness Design for Long-Running Application Development](#)).

The testing responsibility is concrete:

- version model and harness together in every trace, eval result, and release packet;
- rerun relevant suites when either changes, including routing, fallback, tools, context, policy, or infrastructure;
- use ablations and controlled comparisons rather than assuming a component remains load-bearing;
- report task slices where coupling appears instead of generalizing one benchmark result;
- keep rollback compatibility and migrations for durable runs created under an earlier configuration.

17.10 The Governed Iteration Loop



Key Takeaways

- **Trace is sampled observability data:** it cannot automatically serve as complete event history, eval trajectory, or audit record.
- **Correlate; do not conflate IDs:** session/run/step/action/attempt/call identities remain durable, while trace/span IDs serve observability propagation and correlation.
- **Use the complete span model:** attributes, events, status, errors, and links answer different diagnostic questions.
- **Version sampling policy:** priority capture changes the observed distribution and does not replace complete counters, policy events, or recovery data.
- **Separate three activities:** trace grading, outcome grading, and incident investigation have different questions, evidence, and outputs.
- **Govern production-to-eval intake:** review redaction, consent and licensing, deduplication, distribution shift, task validity, and grader evidence.
- **Treat trace findings as hypotheses:** test the responsible model, context, tool, runtime, policy, infrastructure, or grader boundary before changing it.
- **Do not self-modify in place:** meta-harness proposals require offline eval, independent approval, versioning, canary deployment, monitoring, and rollback.
- **Model–harness dependence creates regression responsibility:** it is an empirical property of a configuration, not an inevitable historical trend.

Further Reading

- OpenTelemetry, *Trace API*. <https://opentelemetry.io/docs/specs/otel/trace/api/>
- OpenTelemetry, *Trace SDK*. <https://opentelemetry.io/docs/specs/otel/trace/sdk/>
- OpenTelemetry, *Exceptions*. <https://opentelemetry.io/docs/specs/otel/trace/exceptions/>
- OpenTelemetry, *Semantic Conventions for Generative AI Spans*.
<https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
- W3C, *Trace Context*. <https://www.w3.org/TR/trace-context/>
- Temporal, *Events and Event History*. <https://docs.temporal.io/workflow-execution/event>
- NIST, *SP 800-53 Rev. 5*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- NIST, *SP 800-61 Rev. 3*. <https://csrc.nist.gov/pubs/sp/800/61/r3/final>
- NIST, *AI RMF Core*. <https://airc.nist.gov/airmf-resources/airmf/5-sec-core/>
- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- OpenAI, *A Shared Playbook for Trustworthy Third-Party Evaluations*, May 2026.
<https://openai.com/index/trustworthy-third-party-evaluations-foundations/>
- OpenAI, *Building Self-Improving Tax Agents with Codex*, 2026. <https://openai.com/index/building-self-improving-tax-agents-with-codex/>
- Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026.
<https://www.anthropic.com/engineering/harness-design-long-running-apps>

Chapter 18: AgentOps — Cost, Privacy, and Production Operations

The earlier chapters define how a model-plus-harness system executes, recovers, and produces evaluation evidence. Production adds a different responsibility: operators must keep that exact configuration within cost, privacy, reliability, and release boundaries while real runs are in flight.

This book uses **AgentOps** as a local umbrella term for that operating discipline. It is not the name of one mandatory product or an assertion that every vendor uses the same taxonomy. [Chapter 10](#) separates execution state, event history, traces, and audit records; [Chapter 11](#) defines release evidence; and [Chapter 17](#) explains how sampled traces support diagnosis without becoming a system of record. This chapter preserves those boundaries.

18.1 Cost Needs a Complete Ledger, Not Just a Trace

A **cost ledger** is the complete, deduplicated set of billable and externally spendable events for a declared accounting scope. A trace may link to those events and make them easier to diagnose, but it is not automatically the ledger and is not the audit record.

OpenTelemetry sampling can decide not to record a span, not to export a recorded span, or to drop its attributes, events, and status. It also permits collection limits that drop span fields ([OpenTelemetry — Trace SDK](#)). Therefore:

- a sampled trace cannot prove complete spend;
- a trace backend that loses or delays exports cannot close a billing period by itself;
- only **complete and unlost cost events or cost spans** may feed the cost ledger;
- if cost spans are used, their collection path must be unsampled for the accounting scope, loss-detected, idempotently ingested, and reconciled;
- the trace should carry `cost_event_id` or provider-call references so an operator can join diagnostic spans to ledger entries without collapsing their guarantees.

A useful cost event contains at least:

```
cost_event_id, session_id, run_id, step_id, action_id, attempt_id, call_id
tenant_id, provider_call_id, model_or_tool_version, billing_item
input_units, output_units, cached_units, reasoning_units
price_catalog_version, amount, currency, external_spend
retry_of, fallback_from, status, occurred_at, recorded_at
```

`amount` must be computed under a pinned price/catalog version or copied from an authoritative provider record. Late events, duplicate delivery, credits, and price corrections need explicit adjustment entries rather than mutation that erases history. Reconcile ledger totals against provider invoices and tool/vendor receipts, and alert on missing call IDs or unexplained differences.

Cost reporting should state both the unit of analysis and the population: per model call, tool call, run, successful run, tenant, release version, or accounting window. A low cost per request can conceal expensive failures if abandoned and retried runs are excluded.

18.2 Define Latency Percentiles Where They Are Used

This chapter uses the following local definitions for a declared population and time window:

- **p50** is the median observed latency;
- **p95** is the smallest reported threshold at or below which at least 95% of observations fall;
- **p99** is the corresponding 99% threshold.

Every chart or SLO must say what was measured: end-to-end run latency, model-call latency, tool latency, queue time, approval wait, or another interval. It must also name the time window, release/configuration, sample count, timeout treatment, and whether cancelled or failed runs are included. Do not average independently computed percentiles as though that recreated the combined distribution; aggregate compatible observations or merge an appropriate histogram/sketch.

Report latency beside outcome and cost. A faster run that times out more often or skips required verification is not an operational improvement.

18.3 Four Cache Types, Four Correctness Contracts

“Cache” is not one mechanism. This book uses the canonical distinctions introduced in [Chapter 3](#):

Cache	Reuse boundary	What is reused	Correctness condition
Per-request KV cache	Within one generation/request	The model's internal key/value state for an already processed prefix	Same live generation and compatible model/runtime state
Provider prompt cache	Across separate provider requests	Provider-managed computation for an exact reusable prompt prefix	Provider-specific exact-prefix, model, retention, and eligibility rules
Application response cache	Across application requests	A completed response for an exact normalized request	Exact request plus compatible authorization, tenant, versions, freshness, and policy
Semantic cache	Across application requests	A completed response for a meaning-similar request	Similarity and authorization, tenant, version, freshness, provenance, and risk gates all pass

Provider prompt caching does not reuse the final response: the provider still generates an output for the current request. As a dated product example rather than a portable contract, OpenAI's documentation, checked on **2026-07-31**, describes cache hits for exact prompt prefixes and provider-specific eligibility and retention behavior ([OpenAI — Prompt caching](#)). As another dated product example, Microsoft Azure API Management's documentation, checked on **2026-07-31**, describes semantic caching that can return a stored response for meaning-similar prompts and exposes a similarity threshold ([Microsoft — Semantic caching in Azure API Management](#)). Product behavior may change; these examples do not redefine the four categories.

Semantic similarity is never proof of authorization or factual equivalence. A semantic cache key and admission/lookup gate must include or bind all of the following:

tenant_id
user_id and authorization_scope
model_id and model version
normalized prompt/request and prompt-template version
tool schemas, tool implementations, and relevant tool-result versions
retrieval index, corpus snapshot, filters, and retrieval configuration
policy and safety-rule version
TTL / freshness deadline
invalidation dependencies and invalidation epoch
response provenance: source run, response, citations/artifacts, and creation time

The cache must re-check the current caller's authorization and current policy at lookup time. Tenant separation is mandatory; a shared embedding index does not make cross-tenant response reuse safe. Invalidation must cover source-document changes, permission changes, deletions, model/prompt/tool/retrieval/policy releases, and discovered bad responses. TTL is a maximum freshness bound, not a substitute for invalidation.

Disable semantic response reuse by default for sensitive, personalized, or high-impact decisions. Enable a use case only after tests measure false-hit behavior, stale-answer behavior, cross-scope denial, provenance display, invalidation propagation, and safe miss/fallback behavior. Cache hits and misses should still emit complete cost events and policy-relevant records.

18.4 A Run Budget Is a Vector

A single token cap is not a production budget. Each run needs a versioned budget vector with at least:

Dimension	Examples of what counts	Enforcement question
Tokens	input, output, reasoning, cached read/write units	Can the next call start and still leave a reserve for a safe conclusion?
Dollars	model, retrieval, storage, grader, and tool charges	Will the planned action exceed the run or tenant monetary ceiling?
Tool calls	attempted calls, including failed calls and branches	Has the loop or fan-out reached its call limit?
Wall time	queue, model, tool, approval, retry, and cleanup time	Is there enough deadline remaining to finish or stop safely?
External spend	purchases, transfers, paid API actions, cloud provisioning	Is the side effect separately authorized within amount and destination limits?
Retry/fallback	retries, hedges, model fallbacks, recovery attempts	Is another attempt allowed, and is its side-effect outcome known?

The runtime, not the model's prose, owns the counters and rejects work that would cross a hard boundary. The model may receive remaining-budget state to plan, but it cannot grant itself more budget. Soft thresholds can trigger compaction, cheaper routing, reduced fan-out, or human escalation; hard thresholds transition to a declared terminal or paused state.

Reserve budget for verification, durable state writes, cleanup, and a useful handoff. Count retries and fallbacks against both their own limit and every resource they consume. When an earlier side effect is `unknown`, do not spend the retry budget on blind repetition; resolve or escalate it under [Chapter 10](#)'s recovery state machine. Budget events that enforce hard ceilings must be complete durable state, not reconstructed from sampled traces.

18.5 Privacy Is an End-to-End Data Lifecycle

The NIST Privacy Framework is a voluntary tool for identifying and managing privacy risk through enterprise risk management, not a product-specific telemetry checklist ([NIST — Privacy Framework](#)). For an agent harness, translate that risk discipline into controls across the entire data path:

Lifecycle point	Required harness control	Evidence to retain
Collection and context	Minimize fields and documents; prefer handles or summaries; classify sensitivity before model/tool use	Purpose, data class, selected fields, source, lawful/organizational basis where required
Redaction	Remove secrets and unnecessary personal data before prompts, tool arguments, traces, eval fixtures, or support exports; keep access-controlled references when full data is needed	Redaction policy/version, detector result, exception/approval
Retrieval	Enforce tenant and user authorization before search and again before materialization; carry purpose and scope into filters	Principal, tenant, query/filter, corpus/index version, allow/deny decision
Trace capture and storage	Declare sampling separately from content capture; default to metadata; encrypt, access-control, and isolate trace stores; never treat sampling as redaction	Capture mode, sampling policy, destination, access log, retention class
Retention	Assign TTL/retention by data class and evidence purpose; expire caches, traces, memory, artifacts, and derived eval data deliberately	Retention decision, expiry, legal/incident hold where applicable
Egress	Allowlist provider/tool destinations, constrain region/account where required, redact/minimize payloads, and approve sensitive transfers	Destination, purpose, fields sent, credential audience, approval/policy decision
Deletion	Propagate deletion or revocation to application caches, semantic indexes, retrieval stores, memory, traces, artifacts, and downstream processors according to policy	Tombstone/request ID, affected objects, completion or documented exception

Sampling reduces telemetry volume; it does not make retained content non-sensitive. Redaction reduces exposed content; it does not replace retrieval ACLs or egress policy. Retention expiry does not prove deletion unless downstream copies and derived artifacts are in scope and deletion completion is recorded.

18.6 Monitor Five Different Operational Questions

Do not put every signal into a single “agent health” score. Separate the questions, data completeness requirements, and owners:

Signal class	What it asks	Example signals
Service health	Is the serving system available and within resource limits?	availability, request/error rate, queue depth, saturation, model/tool latency, p50/p95/p99
Agent outcome	Did the run achieve the independently verified task result?	outcome-grader pass, environment-state success, incomplete/unknown outcomes, recovery success
Safety/policy event	Did a protected rule trigger or fail?	denial, approval, sandbox violation, cross-tenant attempt, sensitive egress, policy-version mismatch
Cost	What complete spend was incurred and how quickly are budgets consumed?	ledger amount per run/outcome/tenant/release, missing cost events, invoice variance, budget exhaustion
Human escalation	Can people review and resolve the work in time?	queue age, approval expiry, unresolved unknown side effects, override rate, handoff completion

Service uptime cannot establish task success; the agent's statement that it is done cannot establish an external outcome; a sampled trace count cannot establish total cost or policy-event frequency. Each SLO must name its authoritative source and completeness assumption. Dashboards may correlate the five classes by `run_id` and release version, while alerts route to the owner who can act.

Production failures can become [Chapter 11](#) regression candidates only after the [Chapter 17](#) intake process addresses redaction, consent/licensing, deduplication, distribution shift, and evidence reconstruction. Incident investigation still uses traces together with durable histories, ledger records, audit records, configuration, and external-system state.

18.7 Release the Whole Model–Harness Configuration

The unit of release is a versioned configuration, not “the prompt” or “the model.” Its manifest must pin at least:

- model/provider and inference settings
- prompt/instruction templates and context policy
- tool schemas and tool implementations
- retrieval index/corpus snapshot and retrieval configuration
- memory schema and memory policy
- sandbox image, capabilities, network policy, and resource limits
- policy/approval rules and runtime/router versions
- grader versions and evaluation contract
- run budgets, retry/fallback policy, and cache policy

Any change to one of these components creates a new release candidate and reruns the affected eval slices. OpenAI's evaluation guidance emphasizes reporting the exact tested system, harness, tools, budget, safeguards, and validity checks because those choices determine what an evaluation claim supports ([OpenAI — Trustworthy Third-Party Evaluations](#)).

Carry forward [Chapter 11](#)'s **release evidence packet**: claim and scope, immutable configuration, task-suite and slice versions, isolated trial trajectories, grader calibration, outcome/process/safety results, uncertainty, cost and latency, known limitations, exception owner/expiry, approval, canary plan, post-release monitors, and rollback trigger. A release gate passes only when that packet supports the declared claim.

Deploy the approved unit as a canary, correlate every run and ledger entry with its release version, and compare outcome, safety, cost, service health, and escalation signals. Rollback must restore a compatible complete configuration, preserve durable-run migration rules, and be executable when the declared trigger fires. The candidate cannot approve itself or edit its holdout graders.

18.8 Governance and Vendor Claims Need Scope and Dates

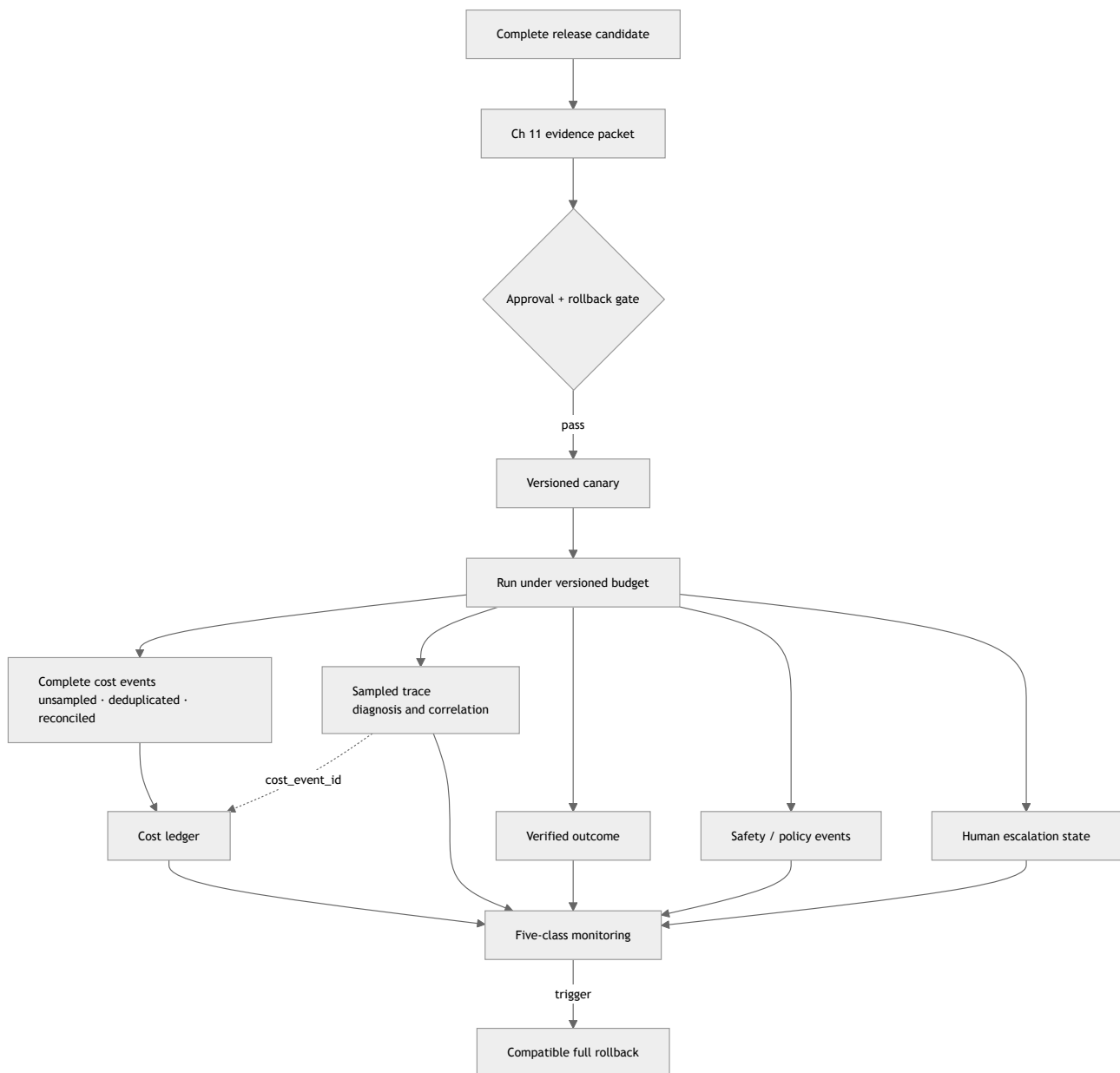
NIST AI RMF is voluntary risk-management guidance, and OWASP's GenAI project publishes engineering risk guidance; neither is a jurisdiction-specific legal determination ([NIST — AI Risk Management Framework](#); [OWASP — Top 10 for LLM Applications](#)). ISO/IEC 42001:2023 specifies an AI management-system standard, while Regulation (EU) 2024/1689 is the official text of the EU AI Act ([ISO — ISO/IEC 42001:2023](#); [European Union — Regulation \(EU\)](#)

2024/1689). A certification, framework mapping, or product feature does not by itself prove that a particular deployment complies with every applicable obligation.

Scope note — effective 2026-07-31. Legal obligations depend on the deployment's jurisdiction, provider/deployer role, use case, affected people, contracts, and the provisions in force on the relevant date. Confirm current official text and qualified counsel for the actual deployment. This chapter is engineering guidance, not legal advice.

Treat rapidly changing vendor “AgentOps,” gateway, observability, cache, evaluation, and governance products only as dated implementation examples. Record the product, documented behavior, region/tier, source URL, and verification date; keep the architecture and release contract independent of the vendor label.

Diagram: Production Evidence and Control Loops



The trace helps explain a run; complete domain records enforce budgets, accounting, policy, recovery, and evidence requirements.

Key Takeaways

- **A sampled trace is not a cost ledger or audit record:** only complete, unlost, deduplicated cost events or cost spans can close the ledger, and sampling breaks that completeness.
- **Define percentiles locally:** state the population, window, interval, configuration, sample count, and timeout/failure treatment for p50/p95/p99.

- **Keep four cache contracts distinct:** per-request KV, provider prompt, application response, and semantic caches reuse different things at different scopes.
- **Semantic cache reuse is authorization-sensitive:** bind tenant, user/auth scope, model, prompt, tool, retrieval, policy version, TTL, invalidation, and provenance; disable sensitive, personalized, and high-impact reuse by default.
- **Budget a vector, not only tokens:** enforce tokens, dollars, tool calls, wall time, external spend, and retry/fallback in durable runtime state.
- **Privacy spans the lifecycle:** minimization, redaction, retrieval ACLs, trace capture/storage, retention, egress, and deletion need separate controls and evidence.
- **Monitor five questions separately:** service health, agent outcome, safety/policy events, cost, and human escalation have different authoritative sources.
- **Release and roll back the complete configuration:** carry Chapter 11's evidence packet and Chapter 17's approval, canary, monitoring, and rollback discipline forward.

Further Reading

- OpenTelemetry, *Trace SDK*. <https://opentelemetry.io/docs/specs/otel/trace/sdk/>
- OpenAI, *Prompt caching* (product documentation; checked 2026-07-31). <https://developers.openai.com/api/docs/guides/prompt-caching>
- Microsoft, *Enable semantic caching for LLM APIs in Azure API Management* (product documentation; checked 2026-07-31). <https://learn.microsoft.com/en-us/azure/api-management/azure-openai-enable-semantic-caching>
- NIST, *Privacy Framework*. <https://www.nist.gov/privacy-framework>
- NIST, *AI Risk Management Framework*. <https://www.nist.gov/itl/ai-risk-management-framework>
- OWASP, *Top 10 for LLM Applications*. <https://genai.owasp.org/llm-top-10/>
- ISO, *ISO/IEC 42001:2023*. <https://www.iso.org/standard/42001>
- European Union, *Regulation (EU) 2024/1689*. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
- OpenAI, *A Shared Playbook for Trustworthy Third-Party Evaluations*. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>

Chapter 19: Agent Fleets and the Control Plane

Earlier chapters govern one model–harness system: how it receives authority, executes a run, survives failure, asks for approval, and produces evidence. A fleet adds a different coordination problem. Many teams may publish many versions, tenants may share infrastructure without sharing authority, and work may move across runtimes or be delegated to other agents. Operators need one coherent answer to what may run, on whose behalf, under which release and policy, and how that authority can be withdrawn.

This book uses **agent control plane** as an adopted distributed-systems architectural convention, not as the name of one product or the only industry-standard decomposition. Kubernetes, for example, documents a control plane that manages cluster state while worker nodes run workloads; the analogy is useful, but an agent fleet is not a Kubernetes cluster and need not copy its component boundaries ([Kubernetes — Components](#)). Here, the **action plane** performs model calls, retrieval, tool calls, code execution, and agent-to-agent exchanges. The **control plane** manages fleet-wide registry, identity, policy administration and decisions, placement, lifecycle, review queues, and evidence indexes. Enforcement still occurs at explicit, non-bypassable points on the action path.

19.1 The Control Plane Governs; the Runtime Executes

Keep four responsibilities distinct:

Responsibility	Owns	Does not become
Model	Proposals: text, plans, candidate actions, and interpretations	The executor, credential holder, policy authority, or source of final truth
Harness	Model-adjacent context assembly, tool mediation, validation, loop control, and evidence capture	The fleet-wide source of release, tenant, or identity truth
Runtime	Durable execution state, event history, queues, retries, scheduling, workers, sandboxes, and cancellation	The policy author or complete audit system merely because it stores events
Control plane	Registry, identities, policy administration and decisions, desired state, placement constraints, revocation, review queues, and lineage indexes	A mandatory single process, vendor product, or centralized hop for every action

Central administration can coexist with distributed enforcement. A control-plane API may publish a desired release, signed policy bundle, or revocation epoch; local gateways, sidecars, tool proxies, and sandbox brokers then enforce that state close to the protected resource. Conversely, putting a service on the network and calling it a “control plane” does not give it authority. Its decisions matter only when every relevant action reaches an enforcement point that can block it.

The control plane also does not replace [Chapter 10](#)'s runtime contract. If a worker crashes, event history and checkpoints recover the run. The control plane may decide where and whether it is allowed to resume, but it should not reconstruct workflow state from a dashboard or sampled trace.

19.2 The Registry Stores Chapter 18's Release Unit

Do not invent a second, weaker “agent record.” The immutable object registered and promoted by the fleet is the complete **release manifest** defined in [Chapter 18](#): model/provider and inference settings; prompt and context policy; tool schemas and implementations; retrieval snapshot and configuration; memory schema and policy; sandbox image, capabilities, network policy, and limits; policy/approval rules and runtime/router versions; grader and evaluation contract; and budgets, retry/fallback, and cache policy.

The registry adds fleet-management bindings around that manifest rather than copying its fields into mutable columns:

```
release_id, manifest_digest, release_manifest_ref
publisher_id, accountable_sponsor_id, owner_team_id
release_state, approval_ref, evidence_packet_ref
eligible_tenants, eligible_workload_classes, placement_constraints
endpoints_and_protocol_versions, dependency_release_ids
desired_deployments, observed_deployments
created_at, deprecates_at, revoked_at, revocation_reason
```

Resolve a display name to an immutable `release_id` before a run starts, and stamp that ID into the run. Registration is not approval: `draft`, `evaluated`, `approved`, `deployed`, `suspended`, `deprecated`, `revoked`, and `retired` are distinct fleet states. A release-state transition should name the evidence packet and accountable approver; rollback selects another complete, compatible manifest rather than swapping only the prompt or model.

Discovery metadata is useful but insufficient. A public capability card may help another agent find an endpoint, while the internal registry determines whether that exact release is approved for this tenant, purpose, workload class, and environment. Dependency edges must be queryable so revoking a tool, policy bundle, sandbox image, publisher, or model can identify affected releases and running work.

19.3 Identity Is a Chain, Not One API Key

A fleet action needs several identities and constraints. “The agent did it” is too vague, and user identity alone loses which release and workload actually acted. The control plane should bind at least:

```
release_version_id      # immutable Chapter 18 release
sponsor_id              # accountable person or organization
delegator_chain         # subject and every acting/delegating principal
tenant_id               # isolation and policy domain
purpose_id              # bounded task/workflow justification
runtime_id              # durable execution service and version
workload_id             # concrete process/service identity
credential_audience    # intended resource/service
issued_at, not_before, expires_at
credential_id, revocation_epoch, revocation_status
session_id, run_id
```

Keep the release identity, runtime identity, and workload identity separate. One release may have many concurrent workloads; one runtime may host many releases; replacing a process must not silently change the release or delegator. As one implementation building block rather than a required standard, SPIFFE defines a SPIFFE ID that uniquely identifies a workload and an SVID with which that workload proves the identity ([SPIFFE — Concepts](#)). It does not supply the sponsor, purpose, release manifest, or delegation semantics needed by this book, so those remain explicit control-plane claims.

Credentials should be short-lived, audience-bound, and presented only at the protected boundary. Raw credentials must not enter model context, messages, generated code, or general-purpose artifact storage. The PEP verifies issuer, subject/actor chain, tenant, purpose, audience, scope, time bounds, revocation state, release status, and the requested resource/action before use.

19.4 Delegation Can Only Narrow Authority

This book adopts a monotonic delegation rule:

```
child_effective_authority =
    parent_effective_authority
    n approved_child_release_policy
    n tenant_and_environment_policy
    n delegated_actions_and_resources
    n purpose, audience, budget, and time bounds
```

Every hop appends an actor to the chain and can remove actions, resources, destinations, budget, or time. It cannot add authority that the parent did not possess. If broader authority is genuinely required, an authorized principal must make a new grant through the policy and approval path; the child cannot infer it from task text.

OAuth 2.0 Token Exchange distinguishes a subject from an actor and defines `audience`, `resource`, and `scope` parameters, but it explicitly leaves token trust and much deployment policy out of scope ([RFC 8693 — OAuth 2.0 Token Exchange](#)). It can encode part of a delegation flow; it does not itself guarantee this book’s narrowing invariant, revocation propagation, or purpose binding. The issuer and PDP must enforce those properties.

Most importantly, **a child's output carries information, not the child's permissions**. When parent A accepts a report, plan, artifact, or tool result from child B, A does not inherit B's credential and B does not acquire A's authority.

If A wants to turn that output into a privileged action, A submits a new action proposal under A's current identity and policy. This is the fleet-scale form of preventing trust escalation and confused-deputy behavior.

19.5 Separate Policy Administration, Decision, and Enforcement

Three functions must remain visible even if one product implements several of them:

Function	Contract	Typical outputs
Policy administration	Humans and governed automation author, review, version, test, approve, distribute, and retire policy bundles	Immutable policy version, signatures, rollout status, revocation epoch
Policy decision point (PDP)	Evaluates identity, tenant, release, resource, action, purpose, delegation, environment, budget, and policy	<code>allow</code> , <code>deny</code> , <code>redact</code> , <code>require_approval</code> , <code>sandbox</code> , narrower scope/budget, reason and decision ID
Policy enforcement point (PEP)	Intercepts the real request and applies the PDP decision before the protected operation occurs	Blocked or transformed request, approval gate, sandbox route, enforcement event

There is a terminology trap here. NIST's Zero Trust glossary defines a **Policy Administrator (PA)** as the component that executes a policy engine's decision by commanding a PEP, and treats the policy engine plus PA as a PDP; that request-time PA is not the same as this book's broader policy-authoring and policy-lifecycle function ([NIST — Zero Trust Architecture Glossary](#)). This chapter therefore writes **policy administration** for authoring/lifecycle and **PDP** for the complete request-time decision service.

A gateway, sidecar, tool proxy, network egress proxy, retrieval materializer, or sandbox broker **may** be a PEP. It qualifies only for the actions it non-bypassably mediates. For example, a tool proxy is not the PEP for shell egress if generated code can reach the network around it. NIST IR 7987 describes a PEP as performing a reference-mediation function; this book makes complete, tamper-resistant mediation an explicit fleet invariant ([NIST IR 7987 Rev. 1 — Policy Machine](#)). Consequential operations should fail closed when current identity, policy, approval, or revocation state cannot be established.

A prompt is **not** a PEP. “Never transfer money without approval” can help the model propose safely, but it cannot prevent a compromised or mistaken model from emitting a call. The mandatory approval from [Chapter 14](#) becomes real only when a PEP verifies an approval bound to the exact immutable action proposal before execution.

19.6 Lifecycle, Placement, and the Kill Switch

Release lifecycle, run lifecycle, and sandbox/worker lifecycle are related but not interchangeable. Revoking a release prevents new runs, yet an already-issued credential and an active run may continue unless their own states are reconciled. A controller should compare desired and observed state and make each consequence explicit:

- scheduling admits only an approved release for the tenant, purpose, workload class, region, and required isolation;
- a run owns durable states such as queued, active, waiting for approval, suspended, cancel-requested, completed, failed, and canceled;
- workers and sandboxes use leases so abandoned execution can be detected and recovered without duplicating the logical run;
- upgrades use the complete release unit and [Chapter 18](#)'s canary, monitoring, rollback, and durable-run compatibility rules.

A fleet **kill switch is a coordinated state transition**, not a natural-language “stop” message. Depending on scope—credential, run, release, publisher, tenant, tool, or fleet—the control plane must:

1. advance the revocation epoch and revoke or invalidate affected credentials;
2. publish a deny rule to every relevant PEP and stop admission of new work;
3. request suspension or cancellation through the run lifecycle, stop renewing leases, and terminate sandboxes where policy requires;
4. reconcile in-flight actions whose side-effect outcome is `unknown` rather than blindly retrying them;
5. record acknowledgements, unreachable enforcement points, exceptions, and final observed state.

RFC 7009 standardizes an OAuth token-revocation request that invalidates a token and may also affect related tokens or the underlying grant, but access-token revocation support and propagation behavior depend on the implementation ([RFC 7009 — OAuth 2.0 Token Revocation](#)). Therefore credential revocation is one required kill-switch

actuator, not the whole mechanism. PEP denial closes still-open action paths, while runtime cancellation controls already-running work. A message to the model is at most an additional steering signal.

19.7 Trace, Event History, Audit, and Lineage Are Different Products

These records may share identifiers or storage infrastructure, but their contracts do not collapse:

Evidence product	Canonical meaning	Completeness and use
Trace	Spans with timing, parent/link relationships, attributes, events, and status	Diagnostic and performance evidence; may be sampled or dropped. OpenTelemetry explicitly allows sampling decisions that discard span data (OpenTelemetry — Trace API ; OpenTelemetry — Trace SDK)
Event history	Ordered durable execution events for reconstructing workflow state and recovering progress	Runtime system of record for the declared workflow scope. Temporal, as a concrete example, durably persists an append-only Event History to recover after failure (Temporal — Events and Event History)
Audit record	Protected accountability record for a declared control scope	Requires the necessary principal, action, object, outcome, timestamps, integrity, retention, access, and completeness controls; NIST SP 800-53's Audit and Accountability family is the relevant control catalog, not a claim that ordinary logs already meet it (NIST SP 800-53 Rev. 5)
Lineage	Provenance graph connecting identities, versions, inputs, artifacts, decisions, actions, and outcomes	Supports causal queries across products; W3C PROV-DM supplies a stable entity/activity/agent and derivation vocabulary, while this chapter defines the fleet-specific graph (W3C — PROV-DM)

Temporal documentation also uses “audit log for debugging” informally for Event History ([Temporal — Events and Event History](#)). That does not make every event history this book's protected audit record. Likewise, an unsampled trace is still not automatically durable workflow history, and copying audit rows into a graph does not create lineage unless the causal edges and version identities are represented.

19.8 The Minimum Fleet Lineage Graph

Lineage should join existing authoritative records by immutable identifiers, not duplicate their mutable prose. For every consequential action, retain edges sufficient to traverse:

```
session_id → run_id → step_id → action_id → attempt_id → call_id
run_id → release_id → manifest_digest → component artifacts
run_id → sponsor / delegator chain / tenant / purpose / runtime / workload
action_id → policy_decision_id → policy_version + decision inputs
action_id → approval_id → immutable proposal digest + reviewer + expiry
call_id → input/output artifact IDs → content hashes and source versions
call_id → cost_event_id → Chapter 18 cost ledger
action_id → enforcement_event_id → PEP identity and result
records → protected_audit_record_id + trace_id/span_id where available
action_id → external_effect_id → verified outcome or explicit unknown
```

The links matter more than collecting every payload. A reviewer should be able to ask which release and policy produced an expensive or harmful outcome, which runs depend on a revoked artifact, or whether every approved side effect has both a PEP event and an independently verified result. Preserve content hashes and protected references; minimize or redact sensitive prompt, memory, and tool content under [Chapter 18](#)'s privacy lifecycle.

Cost is part of lineage without turning traces into accounting. Link the complete `cost_event_id` from Chapter 18's ledger to `call_id`, `run_id`, `tenant_id`, and `release_id`; never infer total fleet spend from sampled spans. Similarly, link the protected audit record rather than assuming the lineage index itself satisfies audit retention and integrity.

19.9 Fleet Review Is a Cross-Run Control Surface

[Chapter 14](#) defines the single-action approval contract: a mandatory approval is a runtime gate bound to the exact immutable proposal, not a conversational “looks good.” The fleet control plane owns the **cross-run review queue** that brings those proposals to the right reviewers.

Queue items should carry `tenant_id`, `release_id`, `run_id`, `action_id`, risk class, policy reason, evidence links, uncertainty, proposed side effect, resource version, proposal digest, requested approval scope, expiry, and current

run/event state. Prioritize by risk, deadline, reversibility, uncertainty, and evidence quality—not arrival order or transcript length. Route by tenant, domain, region, separation-of-duties rule, and reviewer authority.

The queue is a view, not the authorization source. Approval is valid only when the PEP verifies its immutable binding. If the action arguments, resource version, identity chain, release, policy, risk class, scope, or expiry changes, mark the card stale and require a new decision. Cancellation or steering updates durable run state; closing a browser card does not cancel work. Queue SLOs should track age, expired proposals, reassignment, overrides, unresolved `unknown` side effects, and reviewer workload without turning silence into approval.

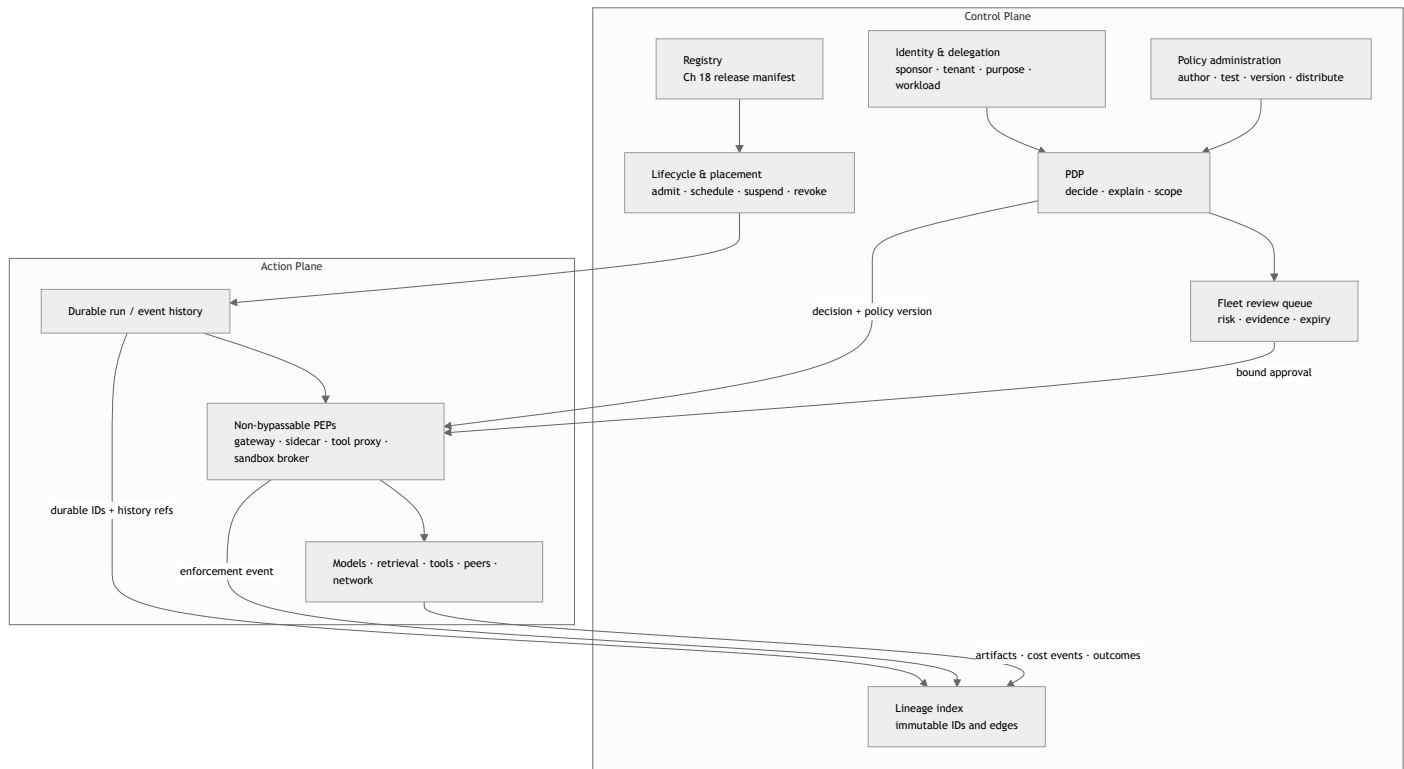
19.10 Dated Standards Snapshot

No source below defines this entire control plane. Record protocol and product status with a date because these surfaces change:

Surface	Status verified 2026-07-31	What it covers here—and what it does not
A2A	Latest released specification shown as 1.0.0	Agent discovery through Agent Cards, messages, tasks, cancellation, and interoperability. The specification explicitly leaves the scope, representation, validity, and revocation semantics of in-task authorization to implementations, so it is not the fleet authority or release registry (A2A — Protocol Specification)
MCP	Versioned authoritative protocol revision 2025-06-18	Exposes resources, prompts, and tools between hosts, clients, and servers. It is a capability/data integration protocol, not a standard for release manifests, sponsor/delegator identity, PDP/PEP topology, or fleet lifecycle (MCP — Specification 2025-06-18)
W3C PROV-DM	W3C Recommendation dated 2013-04-30 , described by W3C as stable reference material	Mature, domain-agnostic provenance vocabulary; it does not define the fleet-specific nodes, enforcement guarantees, or retention policy (W3C — PROV-DM)
NIST ZTA terminology	Current NIST reference vocabulary, checked 2026-07-31	Useful PDP/PEP/PA definitions for access control; not an agent-control-plane product specification (NIST — Zero Trust Architecture Glossary)

Vendor registries, gateways, identity brokers, runtimes, and observability products may implement parts of this design. When using one as evidence, record product name, documented feature, release/tier/region, source URL, and verification date. Do not let a changing product label redefine the architecture.

Diagram: Decision, Enforcement, and Evidence Across a Fleet



The control plane may be distributed. The invariant is that an approved release and identity reach a PDP, and every protected action reaches a PEP that can enforce the current decision and revocation state.

Key Takeaways

- **Control plane is this book's convention, not a universal standard:** it governs fleet-wide state while runtimes and harnesses execute individual runs.
- **Reuse the complete release unit:** the registry points directly to Chapter 18's immutable release manifest and evidence packet.
- **Carry an identity chain:** release, sponsor, delegators, tenant, purpose, runtime, workload, audience, expiry, and revocation must remain distinguishable.
- **Delegation only narrows:** a child result transfers information, never permission; privileged follow-up needs a new policy decision.
- **Separate policy administration, PDP, and PEP:** a gateway or proxy is a PEP only on paths it non-bypassably mediates; a prompt is never a PEP.
- **Kill switches need three actuators:** revoke credentials, deny at PEPs, and transition durable run/worker lifecycles.
- **Trace, event history, audit, and lineage have different guarantees:** correlate them with durable IDs without treating one as another.
- **Lineage joins authoritative records:** include release, identity, policy, approval, PEP action, artifact, cost event, protected audit reference, and verified outcome.
- **Fleet review extends Chapter 14:** prioritize cross-run work by risk and evidence, while the exact approval remains enforced at runtime.

Further Reading

- Kubernetes, *Components*. <https://kubernetes.io/docs/concepts/overview/components/>
- NIST, *Implementing a Zero Trust Architecture — Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>
- NIST, *NIST IR 7987 Rev. 1: Policy Machine: Features, Architecture, and Specification*. <https://nvlpubs.nist.gov/nistpubs/ir/2015/nist.ir.7987r1.pdf>
- IETF, *RFC 8693: OAuth 2.0 Token Exchange*. <https://www.rfc-editor.org/rfc/rfc8693.html>
- IETF, *RFC 7009: OAuth 2.0 Token Revocation*. <https://www.rfc-editor.org/rfc/rfc7009.html>
- SPIFFE, *SPIFFE Concepts*. <https://spiffe.io/docs/latest/spiffe-about/spiffe-concepts/>
- OpenTelemetry, *Trace API and Trace SDK*. <https://opentelemetry.io/docs/specs/otel/trace/api/> and <https://opentelemetry.io/docs/specs/otel/trace/sdk/>
- Temporal, *Events and Event History*. <https://docs.temporal.io/workflow-execution/event>
- NIST, *SP 800-53 Rev. 5*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- W3C, *PROV-DM: The PROV Data Model*. <https://www.w3.org/TR/prov-dm/>
- A2A Protocol, *Protocol Specification*. <https://a2a-protocol.org/latest/specification/>
- Model Context Protocol, *Specification 2025-06-18*. <https://modelcontextprotocol.io/specification/2025-06-18>

Chapter 20: Outlook

This volume began with a deliberately precise boundary: an agent is not a model with authority added by fluent prose. It is a model embedded in an external system that selects inputs, governs actions, preserves state, and checks results. Products, model capabilities, protocols, and preferred scaffolds will change. The boundary tells us which claims still require an external guarantee when they do.

This final chapter therefore does not predict one inevitable agent architecture. It closes the two-volume argument, separates durable principles from dated implementation evidence, identifies open engineering questions, and provides a navigation map from model mechanisms in *LLM Foundations* to the corresponding responsibilities in this book.

20.1 The Standing Boundary

The final principle is the same one stated in *LLM Foundations*, Chapter 14:

The model predicts or proposes. The external system owns context selection, durable state, permissions, execution, verification, and consequences.

“External system” is not one mandatory service. Chapter 1 divides it into the agent harness, runtime, product/application, platform/control plane, and evaluation harness. A deployment may combine those layers or assign parts to providers, but it must still name the component that supplies each guarantee.

Responsibility	Model's legitimate role	External owner and required guarantee
Context selection	Interpret the supplied representation; suggest missing information	The agent harness selects, scopes, serializes, and provenance-tracks the input for each call
Durable state	Propose a transition or summarize progress	Runtime and product persist execution/business state; context and memory are not the sole source of truth
Tool surface and execution	Select a tool and propose structured arguments	Harness exposes contracts; dispatcher/runtime validates, authorizes, executes, correlates, and normalizes results
Permissions	State intent, request access, or ask for review	Product/platform policy decides; a non-bypassable PEP enforces the current decision and mandatory approval
Verification	Produce a critique, estimate, or candidate judgment	Runtime, product, evaluation harness, and humans check artifacts, environment state, process rules, and outcomes
Consequences	Predict or describe an intended effect	Runtime and integrated systems create, record, reconcile, compensate, or roll back effects and remain accountable for them

A provider-executed tool, hosted memory feature, or model-generated critique may move an implementation boundary. It does not remove the need to identify who authorizes the operation, which state is authoritative, what evidence establishes the outcome, and who handles failure. Fluency, confidence, schema validity, or a model's statement that work is complete cannot supply those guarantees by themselves.

20.2 Durable Design Principles

The following principles should survive changes in model families and product labels:

- 1. Separate proposal from effect.** Parse, validate, authorize, approve when required, execute, and confirm outcomes as distinct lifecycle stages.
- 2. Treat context as a selected view.** Context is finite input to one call; memory, artifacts, execution state, event history, traces, eval trajectories, cost ledgers, audit records, and lineage have different contracts.
- 3. Prefer the least dynamic control regime that satisfies the task.** Deterministic workflows, bounded hybrid nodes, and model-directed loops are choices to evaluate, not rungs on a mandatory autonomy ladder.
- 4. Put hard guarantees on unavoidable paths.** Prompt instructions and model self-restraint are defense in depth; permissions, sandboxing, egress, budgets, and mandatory approvals require external enforcement.
- 5. Verify the relevant outcome.** A successful API response, attractive artifact, clean transcript, or passing local check establishes only the property it measured.
- 6. Design for interruption and ambiguity.** Persist identities, checkpoints, budgets, approval state, idempotency records, and `partial` or `unknown` outcomes so recovery does not become blind repetition.

7. **Evaluate and release the complete configuration.** Model, prompts, tools, retrieval, memory, runtime, sandbox, policy, graders, caches, and budgets jointly determine the claim a result supports.
8. **Keep evidence guarantees explicit.** Correlate trace, event history, eval trajectory, cost ledger, audit record, and lineage without treating one as a synonym for another.

These are ownership and evidence principles, not promises that one implementation is optimal forever. A stronger model may let a team remove a planner, reset, evaluator, or tool-selection aid after controlled evaluation. It cannot approve its own authority, retroactively make an unknown side effect safe, or turn sampled observability into a complete record.

20.3 Durable Principles Versus Time-Bound Claims

Readers should classify a claim before relying on it:

Claim class	Example	Required treatment
Durable responsibility boundary	The model proposes; an external system authorizes and executes	State as a standing design principle and test the claimed external guarantee
Workload-dependent design choice	Reset context, use an independent grader, defer tools, or choose a routing cascade	Name the task, model-plus-harness configuration, budget, comparison, and measured outcome
Protocol or standard status	A specification version, conformance level, or section maturity	Cite the official version, publication/status, and verification date; do not infer implementation coverage
Vendor product behavior	Cache matching, retention, hosted tools, model limits, price, or region support	Cite current official product documentation with model/tier/region and verification date
Research measurement or trend	A benchmark score or fitted capability horizon	Preserve the task distribution, harness, metric, uncertainty, update date, and limitations; do not rewrite it as a forecast

The following snapshot illustrates the labeling discipline. It is **not** a ranking or a forecast; every row was verified on **2026-07-31**:

Item	Status or maturity at verification	What the status does not establish
OpenTelemetry Trace SDK	The specification labels the SDK Stable except where otherwise specified ; individual sections can carry another status (OpenTelemetry — Trace SDK)	That exported traces are complete, unsampled, suitable for replay, or compliant audit records
MCP tools contract, version 2025-06-18	A dated protocol specification for discovering and invoking exposed tools, including optional tool-list change notification (MCP — Tools Specification)	Current authorization, non-bypassable enforcement, resource ownership, or verified external outcomes
NIST AI RMF 1.0	A published voluntary risk-management framework; NIST's page states that 1.0 is being revised (NIST — AI Risk Management Framework)	Legal compliance, product certification, or one required agent architecture
OpenAI prompt caching	Active provider product documentation checked on the stated date; matching, breakpoints, retention, and accounting are provider/model-specific (OpenAI — Prompt caching)	A portable cache contract or permission to reuse an application response across users or tenants
METR Time Horizon 1.1	A research measurement page last updated 2026-05-08 , fitting reliability against human-expert duration on its declared software-task distribution (METR — Task-Completion Time Horizons)	How long an arbitrary deployed agent may run autonomously or a forecast for all occupations and environments
Anthropic long-running harness case	A dated vendor engineering case in which decomposition and evaluator scaffolding changed as the tested model and task boundary changed (Anthropic — Harness Design for Long-Running Application Development)	A universal rule that future models will absorb a particular harness component

When a dated row changes, update the row and its verification date. Do not change the standing boundary merely because a provider moved a mechanism behind its API.

20.4 Open Questions by Responsibility

These are questions for engineering and research, not claims that a particular future solution will win. Progress should be demonstrated with a stated contract, representative evidence, and failure analysis.

Category	Open questions	Evidence that would support progress
Context	How should a harness measure information lost by selection, compaction, reset, retrieval, and memory consolidation? How can dynamic tool/context assembly preserve authorization, provenance, invalidation, and provider-cache correctness?	Resume tests, adversarial omission cases, provenance coverage, cross-scope negative tests, and measured quality/cost/latency on declared task slices
State	How should long-running configurations migrate while old runs remain recoverable? How can branches, concurrent workers, compensation, deletion, and nondeterministic model/tool results compose under an honest replay contract?	Recovery drills across version changes, invariant checks, duplicate/unknown-effect tests, deterministic reuse of recorded outputs, and explicit migration/rollback evidence
Tools	How can schemas express effect class, idempotency, approval materiality, postconditions, cancellation, and partial results portably? How should computer-use and peer-agent interfaces expose enough state for safe verification?	Cross-implementation contract tests, injected timeout/race cases, postcondition recall, safe-retry tests, and outcome evidence independent of model text
Security	How should identity, delegated authority, retrieval ACLs, sandboxing, egress, supply-chain provenance, PDP decisions, and distributed PEPs compose without gaps? How should residual prompt-injection risk be measured rather than declared solved?	Non-bypass tests, revocation latency, least-privilege analysis, adversarial content/tool suites, cross-tenant tests, and protected records of decisions and effects
Eval	How can suites remain representative as tasks, models, and user distributions change? How should grader integrity, correlated maker/checker failures, contamination, infrastructure variance, and delayed business outcomes affect release claims?	Versioned tasks and graders, isolated repeated trials, calibration/adjudication sets, per-slice uncertainty, raw failure accounting, and post-release outcome joins
Operations	How should systems optimize cost, latency, reliability, privacy, and human attention per successful policy-compliant outcome? How can complete ledgers and policy events coexist with sampled telemetry, retention, deletion, and incident response?	Complete reconciled counters, five-class monitoring, budget enforcement tests, privacy lifecycle evidence, canary comparisons, rollback drills, and incident timelines
Identity and standards	Which portable contracts should represent agent version, sponsor, delegator, tenant, purpose, credential audience, approval, lineage, revocation, and audit semantics across organizations? Which parts belong in protocols versus local policy?	Versioned conformance profiles, multi-vendor interop tests, explicit maturity levels, negative authorization tests, provenance continuity, and revocation/kill-switch demonstrations

The categories interact. A semantic cache is simultaneously a context, security, and operations problem; an asynchronous peer-agent task is simultaneously state, tools, identity, and eval. Classification does not remove the coupling—it makes each missing guarantee visible.

20.5 Two-Volume Navigation

The table below is the shortest route from a model mechanism in *LLM Foundations* to the engineering responsibility it creates in *Agent Harness*. The repository's [complete Foundations](#) → [Harness source map](#) provides the maintained version with fuller boundary notes.

Foundations mechanism	Model-side boundary	Harness responsibility	Continue in Harness
1. Tokens and model outputs	Produces token probabilities or structured output, not an external effect	Parse proposals, correlate calls, govern dispatch, and verify effects	Ch 1, Ch 6
2. Tokenization	Input cost is representation- and model-specific	Measure actual contracts and enforce context/visual budgets	Ch 3, Ch 15
3. Next-token prediction	Fluent continuation is not proof of fact, permission, execution, or completion	Separate proposal from effect and grade independent evidence/outcomes	Ch 1, Ch 11
4. Attention	Operates on current input; does not create durable memory	Select/order context and persist state, memory, artifacts, and provenance externally	Ch 3, Ch 5, Ch 10
5. Training data and scaling	Broad capability does not establish workload readiness	Evaluate and release the exact model-plus-harness configuration	Ch 11, Ch 18
6. Inference and sampling	Attempts vary; constrained decoding can establish shape, not semantics or authority	Run repeated trials and validate semantics, permissions, and outcomes	Ch 6, Ch 11
7. Post-training	Instruction following is learned behavior, not access control	Compose model-visible guidance but enforce protected actions externally	Ch 2, Ch 7
8. Prompting and in-context learning	Prompt structure steers behavior; untrusted text can still steer proposals	Track assembly provenance, separate data from authority, and enforce outside the prompt	Ch 2, Ch 4, Ch 7
9. Context window and caches	Context is finite; per-request KV state is not every cross-request cache	Budget/transform context and scope each provider, response, and semantic cache correctly	Ch 3, Ch 5, Ch 18
10. Knowledge, hallucination, and uncertainty	Plausibility and verbal confidence are not verified evidence	Retrieve governed sources, preserve citations/provenance, support abstention, and grade grounding/outcomes	Ch 4, Ch 11, Ch 13
11. Embeddings and retrieval	Similarity and minimal RAG do not operate a production data lifecycle	Own ingestion, ACLs, freshness, deletion, index versioning, reranking, and retrieval eval	Ch 4
12. Reasoning, tools, and agents	Tool/computer actions are proposals and results return as context	Complete invocation, authorization, retry, computer-use, approval, and outcome lifecycles	Ch 6, Ch 7, Ch 14, Ch 15
13. Evaluation of model behavior	Model-level tasks and graders do not establish system reliability or external state	Evaluate complete trajectories, artifacts, environments, infrastructure, policies, and releases	Ch 11, Ch 16, Ch 17
14. Operational mental model	Model prediction stops before authority, persistence, execution, and consequences	Assign responsibility across harness, runtime, product, control plane/PEPs, and evaluation harness	Ch 1, Ch 18, Ch 19, Ch 20

20.6 A Rule for Future Revisions

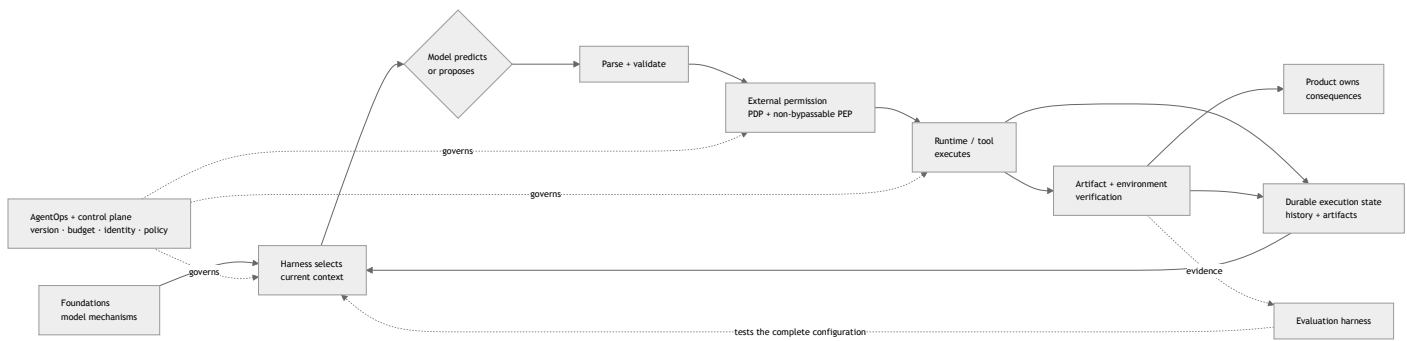
When a model, product, protocol, or benchmark changes, ask four questions:

1. **Which measured component became more or less load-bearing?** Rerun the relevant task and risk slices; do not infer the answer from a release announcement.
2. **Did an implementation move, or did a guarantee disappear?** A provider-hosted mechanism still needs an owner, contract, evidence, and failure path.
3. **Which statement is now stale?** Update version, status, date, scope, and source; preserve historical case results as historical cases.
4. **Does the responsibility boundary still hold?** If the model still only predicts or proposes while another system owns the effect, state, authority, verification, or consequence, the boundary has not moved.

The field can change quickly without making the book's conclusion complicated. Use the model for what it does well. Put authority and durable evidence in the systems capable of providing them. Measure the complete configuration,

and describe only the claim that the evidence supports.

Diagram: One Boundary, Many External Owners



The model remains central to capability. The surrounding system remains accountable for turning that capability into governed, recoverable, and verified work.

Key Takeaways

- **The standing boundary is durable:** the model predicts or proposes; external systems own context selection, durable state, permissions, execution, verification, and consequences.
- **External ownership is distributed:** agent harness, runtime, product, control plane/PEPs, and evaluation harness supply different guarantees.
- **Principles, designs, standards, products, and measurements age differently:** label version, maturity, scope, date, and evidence accordingly.
- **Open questions fall into seven responsibility groups:** context, state, tools, security, eval, operations, and identity/standards.
- **The two volumes form one map:** Foundations explains the model mechanism; Harness implements and tests the responsibility created at its boundary.
- **Capability improvement is not authority transfer:** remove scaffolding only when controlled evidence supports the change, while preserving external guarantees.

Further Reading

- OpenTelemetry, *Trace SDK*. <https://opentelemetry.io/docs/specs/otel/trace/sdk/>
- Model Context Protocol, *Tools Specification*, version 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- NIST, *AI Risk Management Framework*. <https://www.nist.gov/itl/ai-risk-management-framework>
- OpenAI, *Prompt caching* (product documentation; checked 2026-07-31). <https://developers.openai.com/api/docs/guides/prompt-caching>
- METR, *Task-Completion Time Horizons of Frontier AI Models*, Time Horizon 1.1, updated 2026-05-08. <https://metr.org/time-horizons/>
- Anthropic, *Harness Design for Long-Running Application Development*, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>

References

Full bibliography for *Agent Harness: A Practitioner's Textbook*.

Foundations (prerequisite)

- *LLM Foundations for Harness Engineering* — the prerequisite stage-1 volume this book builds on. [../llm-foundations/](https://llm-foundations/)
-

Foundations

- OpenAI, *Harness Engineering: Leveraging Codex in an Agent-First World*, Feb 2026. <https://openai.com/index/harness-engineering/>
 - *Agent Harness Engineering: A Survey*, OpenReview / TMLR submission, 2026. <https://openreview.net/pdf?id=3hXEPbG0dh>
 - Justin Young et al., *Effective Harnesses for Long-Running Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
 - Prithvi Rajasekaran, *Harness Design for Long-Running Application Development*, Anthropic, Mar 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
 - Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, Mar 2026. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/>
 - Birgitta Böckeler, *Harness Engineering for Coding Agent Users*, Thoughtworks / martinfowler.com, Apr 2026. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>
 - Erik Schluntz and Barry Zhang, *Building Effective Agents*, Anthropic, Dec 2024. <https://www.anthropic.com/engineering/building-effective-agents>
 - Shunyu Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv, Oct 2022. <https://arxiv.org/abs/2210.03629>
 - Kyle Brunet, *Skill Issue: Harness Engineering for Coding Agents*, HumanLayer, Mar 2026. <https://www.humanlayer.dev/blog/skill-issue-harness-engineering-for-coding-agents>
-

Context, Memory, and Working State

- Anthropic Applied AI Team, *Effective Context Engineering for AI Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
 - Yichao 'Peak' Ji, *Context Engineering for AI Agents: Lessons from Building Manus*, Manus, Jul 2025. <https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus>
 - Charles Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.08560>
 - Prateek Chhikara et al., *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.19413>
 - Kevin Lin et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*, arXiv, Apr 2025. <https://arxiv.org/abs/2504.13171>
-

Tools and Constraints

- Ken Aizawa, *Writing Effective Tools for Agents — with Agents*, Anthropic, Sep 2025. <https://www.anthropic.com/engineering/writing-tools-for-agents>
- Adam Jones and Conor Kelly, *Code Execution with MCP: Building More Efficient Agents*, Anthropic, Nov 2025. <https://www.anthropic.com/engineering/code-execution-with-mcp>
- David Dworken and Oliver Weller-Davies, *Beyond Permission Prompts: Making Claude Code More Secure and Autonomous*, Anthropic, Oct 2025. <https://www.anthropic.com/engineering/claude-code-sandboxing>

- Simon Willison, *The lethal trifecta for AI agents: private data, untrusted content, and external communication*, Jun 2025. <https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/>
 - *Agent2Agent (A2A) Protocol*, Google / Linux Foundation, 2025. <https://github.com/a2aproject/A2A>
 - Linux Foundation, *Linux Foundation Launches the Agent2Agent Protocol Project*, Jun 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>
 - OpenAI, *Connect Private MCP Servers to OpenAI Products*, Jun 2026. <https://developers.openai.com/blog/connect-private-mcp-servers-to-openai-products>
 - OpenAI, *Programmatic Tool Calling*, 2026. <https://developers.openai.com/api/docs/guides/tools-programmatic-tool-calling>
-

Specs and Workflow Design

- Dex Horthy, *12-Factor Agents*, HumanLayer, Apr 2025. <https://www.humanlayer.dev/blog/12-factor-agents>
-

Reasoning and Planning Patterns

(See also *ReAct* under Foundations, the base reasoning-and-acting pattern.)

- Noah Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.11366>
 - Aman Madaan et al., *Self-Refine: Iterative Refinement with Self-Feedback*, arXiv, Mar 2023. <https://arxiv.org/abs/2303.17651>
 - Zhibin Gou et al., *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*, arXiv, May 2023. <https://arxiv.org/abs/2305.11738>
 - Shunyu Yao et al., *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.10601>
 - Binfeng Xu et al., *ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models*, arXiv, May 2023. <https://arxiv.org/abs/2305.18323>
 - Andy Zhou et al., *Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models*, arXiv, Oct 2023. <https://arxiv.org/abs/2310.04406>
-

Multi-Agent Systems

- Mert Cemri et al., *Why Do Multi-Agent LLM Systems Fail?* (introducing the MAST failure taxonomy), UC Berkeley / arXiv, Mar 2025. <https://arxiv.org/abs/2503.13657>
-

Reliability and Operational Safety

- Temporal, *Durable Execution Meets AI: Why Temporal Is the Perfect Foundation for AI*, 2025. <https://temporal.io/blog/durable-execution-meets-ai-why-temporal-is-the-perfect-foundation-for-ai>
 - LangChain, *Durable Execution* (LangGraph documentation), 2025. <https://docs.langchain.com/oss/python/langgraph/durable-execution>
 - Martin Fowler, *CircuitBreaker*, martinoflower.com, Mar 2014. <https://martinfowler.com/bliki/CircuitBreaker.html>
 - Thinkst, *Canarytokens* (free tripwire tokens). <https://canarytokens.org/>
 - Anthropic Safeguards Research Team, *How We Contain Claude*, May 2026. <https://www.anthropic.com/engineering/how-we-contain-claude>
 - Google Cloud, *Agent Executor: Google's Distributed Agent Runtime*, 2026. <https://cloud.google.com/blog/products/ai-machine-learning/agent-executor-googles-distributed-agent-runtime/>
-

Agent Capability Measurement

- Thomas Kwa et al., *Measuring AI Ability to Complete Long Tasks*, METR / arXiv, Mar 2025. <https://arxiv.org/abs/2503.14499>
 - METR, *Task-Completion Time Horizons of Frontier AI Models*, updated May 8, 2026. <https://metr.org/time-horizons/>
 - Joel Becker, *Clarifying Limitations of Time Horizon*, METR, Jan 22, 2026. <https://metr.org/notes/2026-01-22-time-horizon-limitations/>
-

Loop Engineering

- Addy Osmani, *Loop Engineering*, addyosmani.com, Jun 2026. <https://addyosmani.com/blog/loop-engineering/>
 - *Loop Engineering*, O'Reilly Radar, 2026. <https://www.oreilly.com/radar/loop-engineering/>
 - Andrew Ng, *Three Loops for Building 0-to-1 AI Products*, The Batch, Jun 2026. <https://www.deeplearning.ai/the-batch/>
 - *The Anthropic leader who built Claude Code ditched prompting — now he writes loops*, The New Stack, 2026. <https://thenewstack.io/loop-engineering/>
 - *The Agentic Loop: A Practical Field Guide*, DEV Community, 2026. <https://dev.to/truongpx396/the-agentic-loop-a-practical-field-guide-mnc>
 - *Loop Engineering Guide (2026)*, AI Builder Club. <https://www.aibuilderclub.com/blog/loop-engineering-guide-2026>
 - *Loop Engineering Crash Course*, The AI Agent Factory (Panaversity). <https://agentfactory.panaversity.org/docs/loop-engineering-crash-course>
-

Evaluation and Observability

- Mikaela Grace et al., *Demystifying Evals for AI Agents*, Anthropic, Jan 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
 - Gian Segato, *Quantifying Infrastructure Noise in Agentic Coding Evals*, Anthropic, Feb 2026. <https://www.anthropic.com/engineering/infrastructure-noise>
 - Vivek Trivedy, *Improving Deep Agents with Harness Engineering*, LangChain, Feb 2026. <https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>
 - OpenTelemetry, *Semantic conventions for generative AI spans*. <https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/>
 - Anthropic Safeguards Research Team, *Agentic Misalignment: Summer 2026 Update*, 2026. <https://alignment.anthropic.com/2026/agentic-misalignment-summer-2026/>
 - OpenAI, *Trustworthy Third-Party Evaluations: Foundations*, 2026. <https://openai.com/index/trustworthy-third-party-evaluations-foundations/>
 - OpenAI, *Building Self-Improving Tax Agents with Codex*, 2026. <https://openai.com/index/building-self-improving-tax-agents-with-codex/>
-

Runtimes, Harnesses, and Reference Implementations

- Harrison Chase, *Agent Frameworks, Runtimes, and Harnesses*, Oh My!, LangChain, Oct 2025. <https://blog.langchain.com/agent-frameworks-runtimes-and-harnesses-oh-my/>
 - Jeremy Hadfield et al., *How We Built Our Multi-Agent Research System*, Anthropic, Jun 2025. <https://www.anthropic.com/engineering/multi-agent-research-system>
-

Instructions and Prompting

- Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, OpenAI, Apr 2024. <https://arxiv.org/abs/2404.13208>

- OpenAI, *Custom Code Review Rules for Codex*, Jul 2026. <https://developers.openai.com/blog/custom-code-review-rules-for-codex>
-

Model Selection, Routing, and Reasoning

- Isaac Ong et al., *RouteLLM: Learning to Route LLMs with Preference Data*, 2024. <https://arxiv.org/abs/2406.18665>
 - Lingjiao Chen, Matei Zaharia, and James Zou, *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*, 2023. <https://arxiv.org/abs/2305.05176>
 - Charlie Snell et al., *Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters*, 2024. <https://arxiv.org/abs/2408.03314>
 - DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, 2025. <https://arxiv.org/abs/2501.12948>
 - LiteLLM (BerriAI), *Python SDK and Proxy Server (AI Gateway)*. <https://github.com/BerriAI/litellm>
 - Portkey, *AI Gateway*. <https://github.com/Portkey-AI/gateway>
-

Human-Agent Interaction

- Saleema Amershi et al., *Guidelines for Human-AI Interaction*, CHI 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>
 - Eric Horvitz, *Principles of Mixed-Initiative User Interfaces*, CHI 1999. <https://www.microsoft.com/en-us/research/publication/principles-of-mixed-initiative-user-interfaces/>
-

Computer-Use and Multimodal Agents

- Anthropic, *Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku*, Oct 2024. <https://www.anthropic.com/news/3-5-models-and-computer-use>
 - OpenAI, *Computer-Using Agent (Operator)*, Jan 2025. <https://openai.com/index/computer-using-agent/>
 - Jianwei Yang et al., *Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V*, 2023. <https://arxiv.org/abs/2310.11441>
 - Boyuan Zheng et al., *GPT-4V(ision) is a Generalist Web Agent, if Grounded (SeeAct)*, 2024. <https://arxiv.org/abs/2401.01614>
 - Shuyan Zhou et al., *WebArena: A Realistic Web Environment for Building Autonomous Agents*, 2023. <https://arxiv.org/abs/2307.13854>
 - Tianbao Xie et al., *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*, 2024. <https://arxiv.org/abs/2404.07972>
-

Cost, Privacy, and Governance Frameworks

- AWS, *AgentOps: Operationalize Agentic AI at Scale with Amazon Bedrock AgentCore*, 2026. <https://aws.amazon.com/blogs/machine-learning/agentops-operationalize-agentic-ai-at-scale-with-amazon-bedrock-agentcore/>
 - OWASP, *Top 10 for Large Language Model Applications*, 2025. <https://genai.owasp.org/llm-top-10/>
 - NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
 - ISO/IEC 42001:2023 — *Information technology — Artificial intelligence — Management system*, ISO, 2023. <https://www.iso.org/standard/42001>
 - *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*, European Union, Jun 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>
 - Fu Bang, *GPTCache: An Open-Source Semantic Cache for LLM Applications*, NLP-OSS @ EMNLP 2023. <https://github.com/zilliztech/GPTCache>
-

Agent Fleets, Identity, and Control Plane

- NIST, *Identity and Authorization for Software Agents*, Feb 2026. <https://www.nist.gov/news-events/news/2026/02/new-concept-paper-identity-and-authority-software-agents>
 - Google Cloud, *Agent Registry Overview*, 2026. <https://docs.cloud.google.com/agent-registry/overview>
 - AWS, *AWS Agent Registry in Amazon Bedrock AgentCore (Preview)*, Apr 2026. <https://aws.amazon.com/about-aws/whats-new/2026/04/aws-agent-registry-in-agentcore-preview/>
 - AWS, *Introducing Amazon Bedrock AgentCore Identity: Securing Agentic AI at Scale*, 2026. <https://aws.amazon.com/blogs/machine-learning/introducing-amazon-bedrock-agentcore-identity-securing-agentic-ai-at-scale/>
 - Microsoft, *Agent Registry in the Microsoft 365 Admin Center*, 2026. <https://learn.microsoft.com/en-us/microsoft-365/admin/manage/agent-registry?view=o365-worldwide>
-

Protocols, Standards, and Provider Contracts

The mutable provider documentation in this section was verified on **2026-07-31**. Product behavior, availability, pricing, retention, and accounting remain provider- and date-specific.

- OpenAI, *Prompt Caching*. <https://developers.openai.com/api/docs/guides/prompt-caching>
 - Anthropic, *Prompt Caching*. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
 - Eric Wallace et al., *The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions*, 2024. <https://arxiv.org/abs/2404.13208>
 - Zuxin Liu et al., *IHEval: Evaluating Language Models on Following the Instruction Hierarchy*, 2025. <https://arxiv.org/abs/2502.08745>
 - Model Context Protocol, *Tools Specification*, 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
 - OpenAI, *Computer Use*. <https://developers.openai.com/api/docs/guides/tools-computer-use>
 - Anthropic, *Computer Use Tool*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool>
 - Anthropic, *How Tool Use Works*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>
 - Anthropic, *Vision*. <https://platform.claude.com/docs/en/build-with-claude/vision>
 - OpenTelemetry, *Trace API*. <https://opentelemetry.io/docs/specs/otel/trace/api/>
 - Temporal, *History Service Architecture*. <https://github.com/temporalio/temporal/blob/main/docs/architecture/history-service.md>
 - NIST, *SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations*. <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
 - NIST, *Audit — Glossary*. <https://csrc.nist.gov/glossary/term/audit>
 - NIST, *Zero Trust Architecture Glossary*. <https://pages.nist.gov/zero-trust-architecture/glossary.html>
 - Anthropic, *Contextual Retrieval*, Sep 2024. <https://www.anthropic.com/engineering/contextual-retrieval>
 - Microsoft Azure API Management, *Enable Semantic Caching for Azure OpenAI APIs*. <https://learn.microsoft.com/en-us/azure/api-management/azure-openai-enable-semantic-caching>
 - Microsoft Azure API Management, *Cache Lookup Policy*. <https://learn.microsoft.com/en-us/azure/api-management/cache-lookup-policy>
-

Additional Primary Sources Used Inline

Mutable documentation in this section was verified on **2026-07-31**.

- A2A Protocol, *Specification*. <https://a2a-protocol.org/latest/specification/>
- Patrick Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. <https://arxiv.org/abs/2005.11401>

- Tianyu Gao et al., *Enabling Large Language Models to Generate Text with Citations*, 2023. <https://arxiv.org/abs/2305.14627>
- Nelson F. Liu et al., *Lost in the Middle: How Language Models Use Long Contexts*, 2023. <https://arxiv.org/abs/2307.03172>
- Mark Chen et al., *Evaluating Large Language Models Trained on Code*, 2021. <https://arxiv.org/abs/2107.03374>
- OpenAI, *Function Calling*. <https://developers.openai.com/api/docs/guides/function-calling>
- OpenAI, *Images and Vision*. <https://developers.openai.com/api/docs/guides/images-vision>
- OpenAI, *Using the Latest Model*. <https://developers.openai.com/api/docs/guides/latest-model>
- OpenAI, *Your Data / Data Residency Controls*. <https://developers.openai.com/api/docs/guides/your-data>
- OpenAI, *Models*. <https://developers.openai.com/api/docs/models>
- OpenAI, *Model Spec*, 2025-10-27. <https://model-spec.openai.com/2025-10-27>
- Anthropic, *Handle Tool Calls*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls>
- Anthropic, *Tool Reference*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-reference>
- Anthropic, *Tool Combinations*. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-combinations>
- Anthropic, *Extended Thinking*. <https://platform.claude.com/docs/en/build-with-claude/extended-thinking>
- Model Context Protocol, *Server Features Overview*, 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18/server/index>
- IETF, *RFC 9110: HTTP Semantics*, Jun 2022. <https://www.rfc-editor.org/rfc/rfc9110.html>
- Temporal, *Workflow Execution*. <https://docs.temporal.io/workflow-execution>
- Temporal, *Events and Event History*. <https://docs.temporal.io/workflow-execution/event>
- Temporal, *Activities*. <https://docs.temporal.io/activities>
- Microsoft Azure Architecture Center, *Event Sourcing Pattern*. <https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing>
- Microsoft Azure AI Search, *Security Filters for Trimming Results*. <https://learn.microsoft.com/en-us/azure/search/search-security-trimming-for-azure-search>
- Elasticsearch, *Aliases*. <https://www.elastic.co/guide/en/elasticsearch/reference/current/aliases.html>
- Kubernetes, *Components*. <https://kubernetes.io/docs/concepts/overview/components/>
- Kubernetes, *Resource Management for Pods and Containers*. <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- Docker, *Building Best Practices*. <https://docs.docker.com/build/building/best-practices/>
- Microsoft Playwright, *Auto-Waiting and Actionability*. <https://playwright.dev/docs/actionability>
- Jeffrey Dean and Luiz André Barroso, *The Tail at Scale*, 2013. <https://research.google/pubs/the-tail-at-scale/>
- Geoffrey Huntley, *Ralph Wiggum as a “software engineer”*. <https://ghuntley.com/ralph/>
- OpenTelemetry, *Tracing SDK*. <https://opentelemetry.io/docs/specs/otel/trace/sdk/>
- OpenTelemetry, *Exceptions*. <https://opentelemetry.io/docs/specs/otel/trace/exceptions/>
- W3C, *Trace Context*. <https://www.w3.org/TR/trace-context/>
- W3C, *PROV-DM: The PROV Data Model*. <https://www.w3.org/TR/prov-dm/>
- SPIFFE, *SPIFFE Concepts*. <https://spiffe.io/docs/latest/spiffe-about/spiffe-concepts/>
- IETF, *RFC 8693: OAuth 2.0 Token Exchange*, Jan 2020. <https://www.rfc-editor.org/rfc/rfc8693.html>
- IETF, *RFC 7009: OAuth 2.0 Token Revocation*, Aug 2013. <https://www.rfc-editor.org/rfc/rfc7009.html>
- NIST, *AI RMF Core*. <https://airc.nist.gov/airmf-resources/airmf/5-sec-core/>
- NIST, *SP 800-61 Rev. 3: Incident Response Recommendations and Considerations for Cybersecurity Risk Management*. <https://csrc.nist.gov/pubs/sp/800/61/r3/final>

- NIST, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, AI 100-2e2025. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-2e2025.pdf>
 - NIST, *IR 7987: Policy Machine*. <https://nvlpubs.nist.gov/nistpubs/ir/2014/NIST.IR.7987.pdf>
 - NIST, *IR 7987 Rev. 1: Policy Machine — Features, Architecture, and Specification*. <https://nvlpubs.nist.gov/nistpubs/ir/2015/nist.ir.7987r1.pdf>
 - NIST, *Privacy Framework*. <https://www.nist.gov/privacy-framework>
 - NIST/SEMATECH, *Confidence Intervals for a Proportion*. <https://itl.nist.gov/div898/handbook/prc/section2/prc241.htm>
 - NIST/SEMATECH, *Analysis of Paired Observations*. <https://www.itl.nist.gov/div898/handbook/prc/section3/prc311.htm>
 - NIST/SEMATECH, *Randomized Block Designs*. <https://www.itl.nist.gov/div898/handbook/pri/section3/pri332.htm>
-

Companion Volume

- *LLM Foundations for Harness Engineering*. The model-internals companion to this book. [../llm-foundations/](https://llm-foundations.com/)
-

Reading List

- *Awesome Harness Engineering*, walkinglabs. <https://github.com/walkinglabs/awesome-harness-engineering>
- *Learn Harness Engineering — English Edition*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/en/>
- *Learn Harness Engineering — Chinese Edition*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/zh/>
- *Learn Harness Engineering — Skills (English)*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/en/skills/>
- *Learn Harness Engineering — Skills (Chinese)*, Walking Labs. <https://walkinglabs.github.io/learn-harness-engineering/zh/skills/>

Glossary

Canonical vocabulary used throughout this book. Chapter links lead to the full argument and its inline evidence; key protocol and standards definitions are also cited here.

System Boundaries

Model — The probabilistic component that consumes the current input representation and generates tokens or structured output. It proposes; it does not itself own durable state, credentials, external execution, or consequences ([Ch 1](#)).

Agent — A model participating in a goal-directed loop with surrounding software that can assemble context, use tools, maintain state, and inspect outcomes.

Agent harness — The model-adjacent system that assembles inputs, parses model outputs, and drives one or more agent loops. It is narrower than the whole product, runtime, or fleet platform ([Ch 1](#)).

Runtime — The execution substrate for scheduling, queues, durable steps, checkpoints, sandbox processes, cancellation, and retry semantics ([Ch 10](#)).

Product / application — The user-facing workflow, domain logic, review surface, and business state in which agent work has meaning.

Platform / control plane — Shared fleet infrastructure for registry, identity, policy administration and decision, lifecycle, and multi-tenant governance. Enforcement still requires a non-bypassable PEP on or next to each protected data path ([Ch 19](#)).

Evaluation harness — The test system that creates task environments, invokes the agent harness for trials, collects complete grading evidence, applies graders, and aggregates results. It is distinct from the production agent harness being evaluated ([Anthropic — Demystifying Evals for AI Agents](#)).

Harness engineering — Iterating on prompts, context policies, tools, runtime controls, state, verification, and operations as one system rather than optimizing a prompt in isolation.

Context engineering — Selecting, ordering, transforming, and provenance-tracking the representations supplied to a model call ([Ch 3](#)).

ETCLOVG — A survey taxonomy—Execution, Tools, Context, Lifecycle, Observability, Verification, Governance—used as an organizing lens, not an industry standard ([Ch 1](#)).

Context, Retrieval, State, and Evidence

Context — The token or multimodal representation visible to one model call. It is finite and is not durable workflow state.

Context window — The model/provider-specific maximum input-plus-output span for one call; practical reliability may degrade before the hard limit ([Foundations Ch 9](#)).

Per-request KV cache — Key/value states reused for previously processed tokens during one generation. It belongs to the inference mechanism described in [Foundations Ch 9](#).

Provider prompt cache — A provider contract for reusing eligible prompt prefixes across independent requests. Matching, TTL, billing, and data controls are provider-specific ([OpenAI](#); [Anthropic](#)). It is not a synonym for per-request KV cache.

Application response cache — Reuse of a final response for an identical, scope-compatible request under application-controlled keys and invalidation.

Semantic cache — Reuse of a response for a semantically similar request. Similarity is only one gate; tenant, user/auth scope, model, prompt, tool, retrieval, policy version, freshness, and risk must also be compatible ([Azure semantic caching](#); [Azure cache policy](#)).

Retrieval pipeline — Source ingestion → parse/chunk → metadata and ACL → versioned index → query → dense/lexical retrieval → fusion/reranking → context assembly → citation/provenance → outcome evaluation ([Ch 4](#)).

Evidence item — A retrieved unit whose content travels with identifiers such as `source_id`, `source_version`, `chunk_id`, `content_hash`, `index_version`, and `acl_scope` so claims can be traced and permissions rechecked.

Grounding — The relationship between an answer or action and supplied evidence. Retrieval correctness, grounding/citation correctness, and final task outcome are separate measurements.

Compaction — A lossy context transformation that preserves selected decisions, constraints, open work, artifact pointers, provenance, and uncertainty in a smaller representation ([Ch 5](#)). Foundations Ch 9 explains why context is finite; it does not define a compaction algorithm.

Memory — An information product stored for possible use in future calls. It needs scope, provenance, authority, freshness, update, access, and forgetting rules.

Execution state — Authoritative structured workflow state such as current step, pending action, budget, lease, and approval status ([Ch 10](#)).

Event history — An ordered durable record of accepted workflow events sufficient for recovery under a stated replay contract. Temporal documents its Event History as the recovery record for workflows ([Temporal](#)).

Checkpoint — Recoverable state at a known history position, plus the references, versions, and resume contract needed to continue. It is not the complete causal history.

Artifact — An addressable work product such as a file, diff, report, dataset, build, screenshot, or test result.

Trace — Observability data composed of spans, events, attributes, status, links, and timestamps; it may be sampled ([OpenTelemetry Trace API](#)). A trace is not automatically an event history, eval trajectory, cost ledger, or audit record.

Eval transcript / trajectory — The complete model-, tool-, observation-, artifact-, and outcome-level record retained to grade one trial. It may be derived from tracing only when grader-required evidence is complete ([Ch 11](#)).

Audit record — A protected accountability record with defined content, identity, timestamps, integrity, access, review, and retention. NIST treats these as explicit control requirements ([NIST SP 800-53 Rev. 5](#)).

Lineage — The provenance graph linking identities, versions, inputs, artifacts, policy decisions, approvals, actions, and verified outcomes.

Context reset — Starting a new model context from a structured handoff while durable execution state and artifacts remain external.

Context firewall — Isolation of a worker's intermediate context from a parent. It reduces context pollution but does not raise the worker's trust or authority; returned claims still need evidence and verification.

Tools, Authorization, and Runtime Enforcement

Tool call / action proposal — Structured model output naming a tool and arguments. Application code decides whether and how to execute it ([Anthropic — How Tool Use Works](#)).

Invocation lifecycle — Proposal → stream completion → parse → schema validation → semantic validation → authorization → mandatory approval when required → idempotency/dedup → execution → timeout/cancellation handling → result normalization → observation → outcome confirmation ([Ch 6](#)).

Schema validity — Whether an action matches its declared machine-readable shape. It does not establish semantic correctness, authorization, or safe consequences.

Semantic validity — Whether normalized arguments make sense for the current task, resource, and state.

Execution result — What an executor returned about an attempted action. It can be successful even when the intended environment outcome did not occur, or unknown after a timeout.

Outcome — The verified environment or business state after an action or trial, distinct from the model's claim and the executor's immediate response.

Idempotency key — A caller-supplied identifier used by a compatible service to deduplicate equivalent mutation attempts. It reduces duplicate effects but does not replace outcome verification.

MCP (Model Context Protocol) — A client/server protocol through which servers expose tools and other capabilities under their implementation and access controls. Tool lists may change, so discovery does not replace authorization at invocation time ([MCP Tools Specification](#)).

A2A (Agent-to-Agent protocol) — A protocol boundary for delegation and coordination among agentic applications; it is not the same boundary as exposing a tool to one agent runtime.

Sandbox — An isolated execution environment with explicit filesystem, network, credential, process, and persistence boundaries. It limits blast radius and creates a region where approved actions can proceed ([Ch 7](#)).

Capability grant — Scoped authority to perform an operation against a resource under stated identity, purpose, environment, audience, and expiry conditions.

Delegated authorization — Short-lived, audience- and purpose-bound authority derived from a user or service. Further delegation may narrow but must not silently widen it.

Instruction priority — The intended behavioral ordering of model-visible instructions. It is defense in depth, not authorization or an execution guarantee; hierarchy-evaluation research still measures conflict failures ([Instruction Hierarchy](#); [IHEval](#)).

Model-visible policy — Instructions and explanations placed in model context to steer behavior and help the model propose compliant actions.

Executable policy — Rules evaluated and enforced by external software on a protected action path.

Policy decision point (PDP) — The component that computes a policy decision from subject, action, resource, purpose, environment, and policy ([NIST Zero Trust Glossary](#)).

Policy enforcement point (PEP) — The non-bypassable component that applies a policy decision to a request for a protected resource. A prompt is not a PEP ([NIST Zero Trust Glossary](#)).

Advisory hook — A lifecycle hook that adds context, warns, or asks the model to reconsider, without necessarily preventing dispatch.

Blocking enforcement — A control through which every relevant action must pass, where denial prevents dispatch and alternate paths cannot bypass it.

Approval request / consultation — A model or workflow request for human input. It is useful orchestration but is not by itself a hard safety gate.

Mandatory approval gate — A runtime/PEP gate that policy inserts before protected dispatch whether or not the model requested it. Approval binds the reviewed identity, resource, normalized arguments, policy version, constraints, and expiry ([MCP Tools Specification](#)).

Lethal trifecta — Practitioner terminology for the combination of private-data access, untrusted content, and external communication. It is a threat-model heuristic, not a formal security standard ([Ch 7](#)).

Routing and Workflow Control

Router — A policy that selects a route target before running the chosen target, using declared quality, cost, latency, capability, and governance features.

Route target — A model-plus-adapter-plus-tool-contract-plus-reasoning-policy configuration, not merely a model name.

Quality cascade — Trying a lower-cost target, evaluating the result, and escalating to a stronger target when the acceptance signal rejects it.

Reliability fallback — Switching after a timeout, rate limit, provider error, or other availability failure, but only after compatibility and side-effect checks.

Hedging — Starting more than one compatible attempt to reduce tail latency, with explicit winner, cancellation, cost, and side-effect semantics.

Human escalation — Routing uncertainty, incompatibility, policy conflict, or high consequence to a person instead of silently substituting another model.

Compatibility gate — A check that fallback targets agree on modalities, context limits, tool protocol/schema, structured output, continuation state, safety policy, latency envelope, and data region ([Ch 8](#)).

Reasoning policy — Provider-specific controls and budgets for reasoning effort, continuation, token accounting, and allowed disclosure. It is not a portable representation of hidden reasoning.

Deterministic workflow — External code owns sequencing and state; model calls fill bounded steps.

Model-directed loop — The model selects the next proposed action while the harness owns state, authorization, execution, stop rules, and outcome checks.

Hybrid workflow — Deterministic structure around bounded model-selected branches or loops.

Orchestrator-workers — A pattern in which an orchestrator proposes decomposition and delegates bounded work; external state and handoff contracts preserve evidence and recovery ([Ch 9](#)).

Evaluation and Verification

Task / trial — A task defines inputs, environment, constraints, and success criteria; a trial is one attempt at that task.

Grader — A code-based, model-based, or human component that measures one declared aspect of a trial.

Process / trajectory grading — Checking actions, policy use, tool arguments, budgets, or recovery behavior without assuming the final artifact is correct.

Artifact grading — Checking an addressable output such as code, a report, or a dataset.

Environment-state grading — Inspecting the actual external state after execution rather than trusting the agent's final message.

User / business outcome grading — Measuring the delayed product result that matters to users or the organization.

pass@k / pass^k — `pass@k` measures the probability of at least one success among `k` attempts; `passk` measures all-`k` consistency. Use the metric that matches the product contract and report uncertainty ([Foundations Ch 13](#); [Ch 11](#)).

Verifier hierarchy — Schema checks → deterministic tests/linters → environment outcome checks → same-agent critique → independent model grader → human review. Choose by consequence, ambiguity, correlated failure, latency, and cost; an independent agent is not universally required ([Ch 13](#)).

Infrastructure noise — Score variation caused by hardware, resource limits, disk/network behavior, images/dependencies, caches, parallelism, or timeouts rather than the tested agent change ([Anthropic — Infrastructure Noise](#)).

Release evidence packet — The pinned model-plus-harness configuration, per-slice trial results, uncertainty, grader versions and calibration, failures, policy checks, and approval needed for a release decision.

Long-Running Work, Human Interaction, and Computer Use

Durable execution — Persisting progress so a run can recover after process, worker, network, or infrastructure failure. Replay reuses recorded model/tool outputs rather than regenerating them ([Ch 10](#)).

Handoff — A transfer package containing completed work, verified evidence, open risks, next action, artifact pointers, and permission context.

Stop rule — Explicit termination on success, budget exhaustion, no progress, repeated failure, policy denial, human cancellation, or terminal environment state.

Action budget — Hard limits covering tokens, dollars, tool calls, wall time, external spend, and retries/fallbacks.

Steering event — A recorded instruction that changes future planning after a defined acceptance point; it does not retroactively cancel an action already dispatched.

Cancellation event — A recorded request to stop future work and cancel compatible in-flight operations; already committed effects may require compensation.

Computer-use tool loop — The model proposes a structured computer action; the application executes it; a screenshot or other UI state returns as tool result/observation ([OpenAI](#); [Anthropic](#)).

Screenshot observation — Pixel evidence captured at a particular time. It may be stale by execution time and does not itself prove the post-action outcome.

Surface identity — The application, process, window/tab, frame, URL/origin, viewport, geometry, and capture version to which an observation and proposed UI action are bound.

Observation/action race — The interface changes after observation but before or during dispatch, so an action grounded in state `S0` executes against state `S1`. Freshness checks and observation barriers control the race ([Ch 15](#)).

Visual / coordinate grounding — Mapping an intended UI target to a coordinate, element, or action in the current interface state.

Accessibility tree — A structured view of UI roles, labels, states, and relationships; useful when available but not guaranteed to match all visible or actionable state.

Operations, Fleets, and Governance

Circuit breaker — A runtime control that trips after a failure threshold so calls to an unhealthy dependency fail fast instead of causing a retry storm.

Kill switch — A human- or policy-triggered stop implemented through run lifecycle, credential revocation, and PEP denial rather than a natural-language instruction alone.

Span telemetry — Trace spans and attributes for model calls, tool calls, retrieval, context assembly, policy, cost, and outcome.

Trace-to-eval loop — Turning a redacted, consent/licensing-reviewed production failure into a reproducible regression task with complete outcome assertions.

Cost ledger — Complete, unsampled cost events correlated to runs, actions, providers, tools, and external spend. Sampled traces cannot by themselves support a complete ledger.

Release unit — The versioned combination of model, prompts, tool schemas/implementations, retrieval configuration/index, memory policy, sandbox image, policy, graders, and budgets.

Agent identity chain — The linked agent version, sponsor, delegator, tenant, purpose, runtime/workload identity, credential audience, expiry, and revocation state.

Agent registry — A governed source of truth for agent definitions, versions, owners, capabilities, requested authority, attestations, dependencies, and deployment state.

Agent fleet — A governed collection of agent definitions, releases, runs, runtimes, identities, tools, and owners rather than one local agent instance.

AgentOps — Operating discipline for service health, agent outcomes, safety/policy events, cost, privacy, release, incidents, and human escalation ([Ch 18](#)).

Non-repudiation — Integrity-protected evidence that attributes a request or authorization to workload and policy identities. It does not prove that a human understood an approval or that a model rationale was truthful.

Foundations → Harness Source Map

This map shows where a model-side concept from *LLM Foundations* becomes an engineering responsibility in this book. The handoff is not a claim that the harness changes the model mechanism; it identifies the surrounding controls required to use that mechanism in a working system.

Foundations concept	Model-side boundary	Harness responsibility	Continue in
Tokens and model outputs	The model produces token probabilities and structured output; it does not directly create an external effect.	Parse proposals, preserve correlation IDs, authorize dispatch, execute, and verify the outcome.	Ch 1 , Ch 6
Tokenization	Text and multimodal inputs consume model-specific budgets; visible characters or image dimensions are not portable token estimates.	Measure with the selected provider/model contract and enforce context and visual-input budgets.	Ch 3 , Ch 15
Next-token prediction	Fluent continuation is not proof of factuality, authorization, execution, or task completion.	Separate proposal from effect and grade evidence or environment state rather than confidence of wording.	Ch 1 , Ch 11
Attention	Attention operates over the current model input and does not create durable memory.	Select, order, compact, retrieve, and provenance-track context; store durable state outside the model.	Ch 3 , Ch 5 , Ch 10
Data and scaling	Training coverage and scale affect capability but do not establish task-specific readiness.	Evaluate the exact model-plus-harness release on representative task and risk slices.	Ch 11 , Ch 18
Inference and sampling	Sampling makes attempts variable; constrained decoding can enforce shape but not semantic validity or permission.	Run repeated trials, validate semantics, and gate side effects independently of output syntax.	Ch 6 , Ch 11
Post-training	Learned interaction conventions and instruction following are behavioral properties, not access-control guarantees.	Represent instruction priority in context, but enforce protected actions through runtime policy and PEPs.	Ch 2 , Ch 7
Prompting and in-context learning	Prompt structure steers behavior; untrusted text can still steer the model through prompt injection.	Track assembly provenance, treat retrieved/tool content as untrusted, minimize authority, and enforce outside the prompt.	Ch 2 , Ch 4 , Ch 7
Context window and caches	Context is finite; per-request KV state differs from a provider's cross-request prompt-cache contract. OpenAI and Anthropic both document provider-specific eligibility and retention behavior (OpenAI , Anthropic).	Budget and transform context, preserve cache-compatible prefixes where useful, and keep response/semantic caches separately scoped and invalidated.	Ch 3 , Ch 5 , Ch 18
Knowledge, hallucination, and uncertainty	A plausible answer, citation-shaped string, or verbal uncertainty is not verified evidence.	Retrieve authorized evidence, bind citations to source versions, support abstention, and grade grounding separately from outcomes.	Ch 4 , Ch 11 , Ch 13
Embeddings and retrieval	Embedding similarity and minimal RAG do not operate ingestion, ACLs, freshness, deletion, or provenance.	Own the production retrieval data path and evaluate retrieval, grounding, citation, and task outcome as separate layers.	Ch 4
Reasoning, tools, and agents	The model can propose a structured tool or computer action; the application executes it and returns a result (Anthropic tool-use protocol).	Complete the invocation lifecycle, enforce authority, control retries and side effects, and confirm the resulting environment state.	Ch 6 , Ch 7 , Ch 15
Evaluating model behavior	Model-level tasks, trials, graders, and pass metrics do not by themselves measure tool execution, environment state, or operational reliability.	Build an evaluation harness that runs the production harness, records transcripts and outcomes, isolates environments, and reports system-level slices.	Ch 11 , Ch 16 , Ch 17
Operational mental model	The model predicts; it does not own context selection, durable state, permissions, real tool effects, or consequences.	Assign those responsibilities explicitly across harness, runtime, product, control plane, enforcement points, and evaluation harness.	Ch 1 , Ch 19 , Ch 20

Reading the Boundary

Three rules prevent most category errors:

1. A **model output** can propose an action; an **execution result** says what the executor returned; a verified **outcome** says what changed in the environment.
2. **Context** is the current call's input; **memory** is a future-retrieval product; **execution state** and **event history** support workflow progress and recovery.
3. A model-visible instruction can reduce undesirable behavior; only a non-bypassable runtime control can provide an enforcement guarantee.

See the [Glossary](#) for the canonical definitions used in both volumes.